

Class05: Data Visualization with GGLOT

Lea Sounthong

Class Work:

Title Here

Subtitle

###Subtitle

Stuff in bold not in bold *something italics*

Indent text

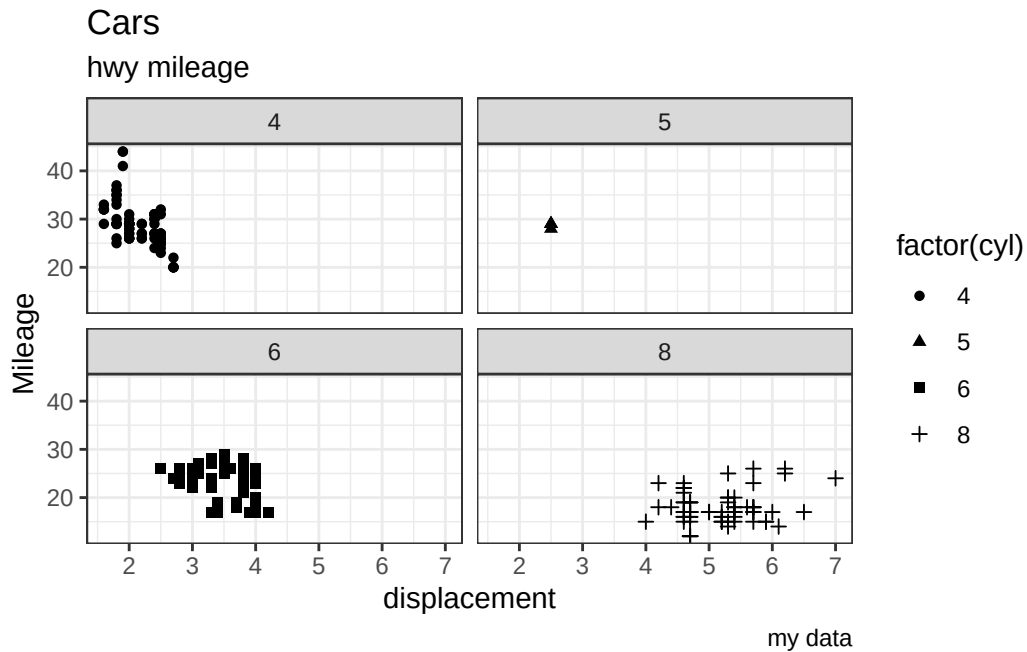
```
#Generating numbers from 1-5 and adding 1 to each number
x<-1:5
x+1
```

```
[1] 2 3 4 5 6
```

Plot from class:

```
#Generating scatter plot using the mpg dataset
library(ggplot2)
ggplot(data=mpg) +
  #Setting variables for axes and grouping
  aes(x=displ, y=hwy, shape=factor(cyl)) +
  #Adding points to the plot
  geom_point() +
  #Split plot into panels by cylinder number
  facet_wrap(~cyl) +
  #Generating simple theme
```

```
theme_bw() +
#Generating labels and titles
labs(title="Cars", subtitle= "hwy mileage",
      caption="my data", x="displacement", y="Mileage")
```



Q1. For which phases is data visualization important for our scientific workflows?

All of the above

Q2. TRUE or FALSE? The ggplot2 package comes already installed with R?

FALSE

Part 5

Q. Which plot types are typically NOT used to compare distributions of numeric variables?

Network graphs

Q. Which statement about data visualization with ggplot2 is incorrect?

ggplot2 is the only way to create plots in R

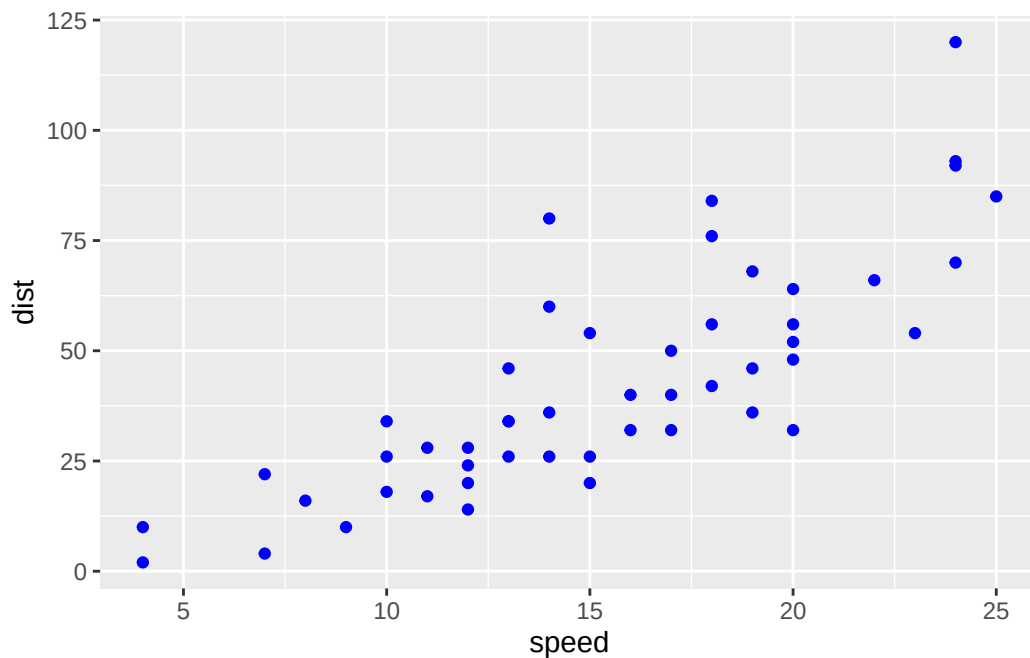
Part 6. Creating Scatter Plots

Q. Which geometric layer should be used to create scatter plots in ggplot2?

geom_point()

Generating scatter plot for part 6

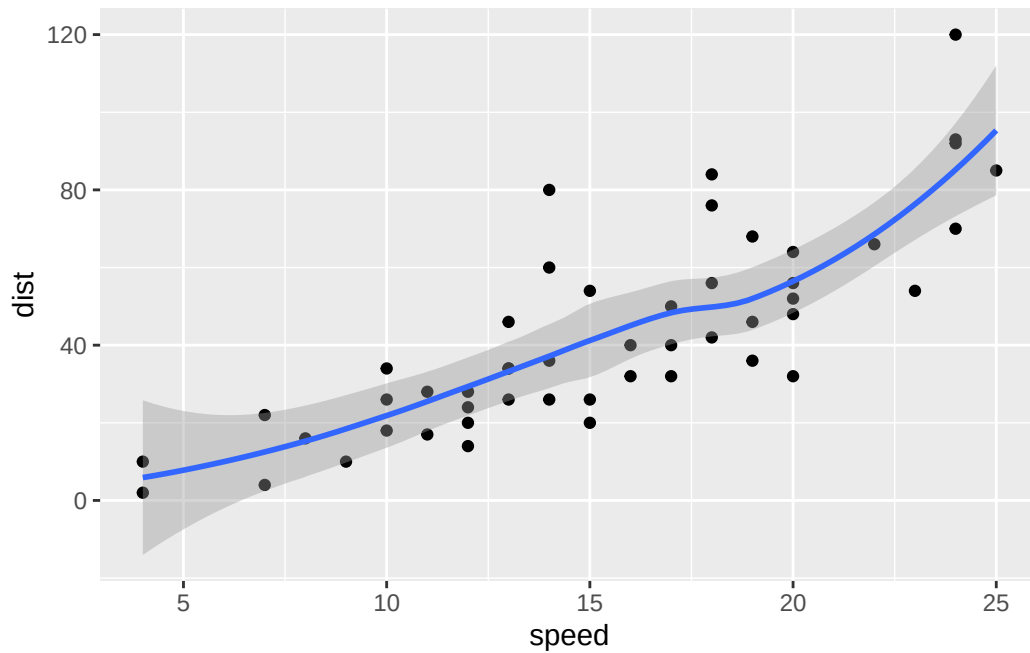
```
#Loading ggplot2 for creating plots
library(ggplot2)
#Create a scatter plot of the cars dataset
ggplot(cars)+
  #Define which variables used for x and y axes and making points blue
  aes(x=speed,y=dist)+geom_point(col="Blue")
```



Q. In your own RStudio can you add a trend line layer to help show the relationship between the plot variables with the `geom_smooth()` function?

```
#Generating a trend line layer to show relationship between the plot variables
library(ggplot2)
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_smooth()
```

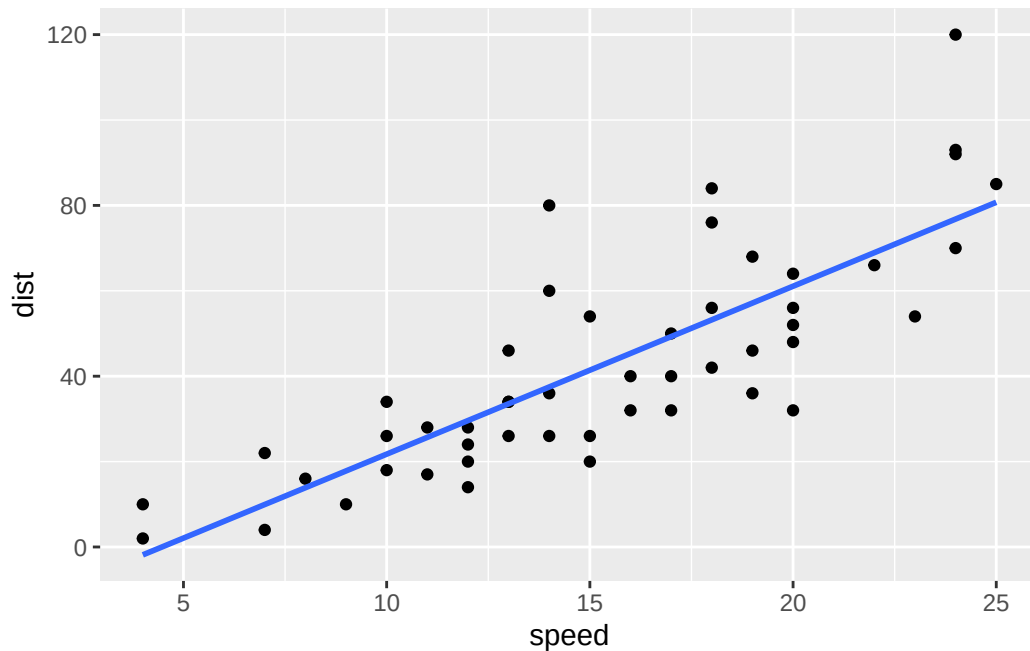
``geom_smooth()`` using `method = 'loess'` and `formula = 'y ~ x'`



Q. Argue with `geom_smooth()` to add a straight line from a linear model without the shaded standard error region?

```
#Generating scatter plot of cars data with linear trend line
library(ggplot2)
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

`geom_smooth()` using formula = 'y ~ x'



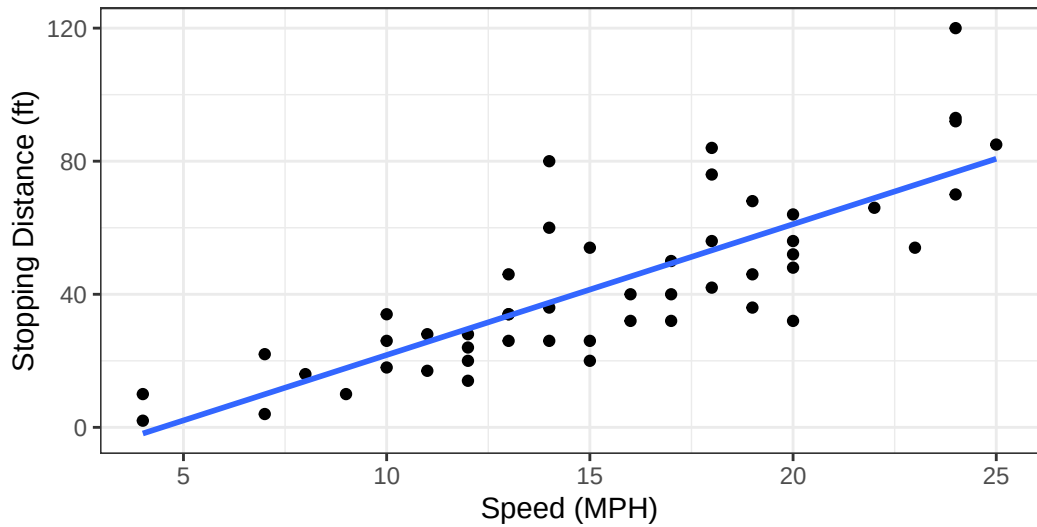
Q. Can you finish this plot by adding various label annotations with the `labs()` function and changing the plot look to a more conservative “black & white” theme by adding the `theme_bw()` function:

```
#Adding various label annotations and changing the plot to black and white theme
library(ggplot2)
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  labs(title="Speed and Stopping Distances of Cars",
       x="Speed (MPH)",
       y="Stopping Distance (ft)",
       subtitle = "Your informative subtitle text here",
       caption="Dataset: 'cars'") +
  geom_smooth(method="lm", se=FALSE) +
  theme_bw()
```

``geom_smooth()`` using formula = 'y ~ x'

Speed and Stopping Distances of Cars

Your informative subtitle text here



Dataset: 'cars'

Adding more plot aesthetics through aes()

```
#Loading in gene expression dataset from online file
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
#Read the data into R and preview it
genes <- read.delim(url)
head(genes)
```

	Gene	Condition1	Condition2	State
1	A4GNT	-3.6808610	-3.4401355	unchanging
2	AAAS	4.5479580	4.3864126	unchanging
3	AASDH	3.7190695	3.4787276	unchanging
4	AATF	5.0784720	5.0151916	unchanging
5	AATK	0.4711421	0.5598642	unchanging
6	AB015752.4	-3.6808610	-3.5921390	unchanging

Q. Use the nrow() function to find out how many genes are in this dataset. What is your answer?

```
#Counting total number of genes in the dataset
nrow(genes)
```

```
[1] 5196
```

Q. Use the `colnames()` function and the `ncol()` function on the `genes` data frame to find out what the column names are (we will need these later) and how many columns there are. How many columns did you find?

```
#Displaying column names in the dataset
colnames(genes)
```

```
[1] "Gene"          "Condition1" "Condition2" "State"
```

```
#Counting how many columns are present
ncol(genes)
```

```
[1] 4
```

Q. Use the `table()` function on the `State` column of this data.frame to find out how many ‘up’ regulated genes there are. What is your answer?

```
#Counting how many genes are in each category
table(genes$State)
```

down	unchanging	up
72	4997	127

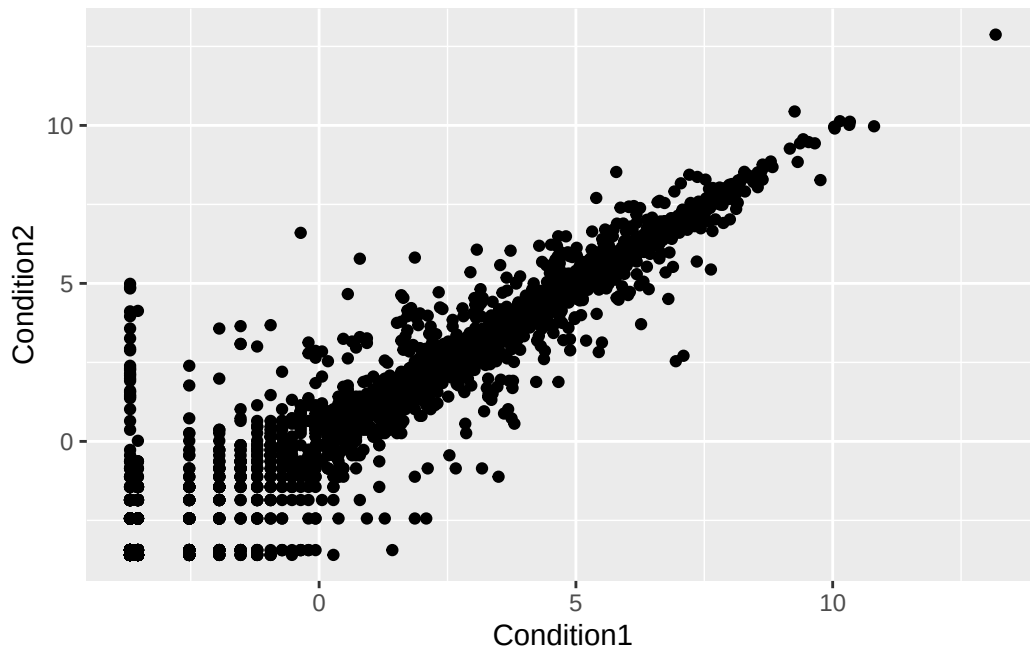
Q. Using your values above and 2 significant figures. What fraction of total genes is up-regulated in this dataset?

```
#Calculating the percentage of each gene category
round( table(genes$State)/nrow(genes) * 100, 2)
```

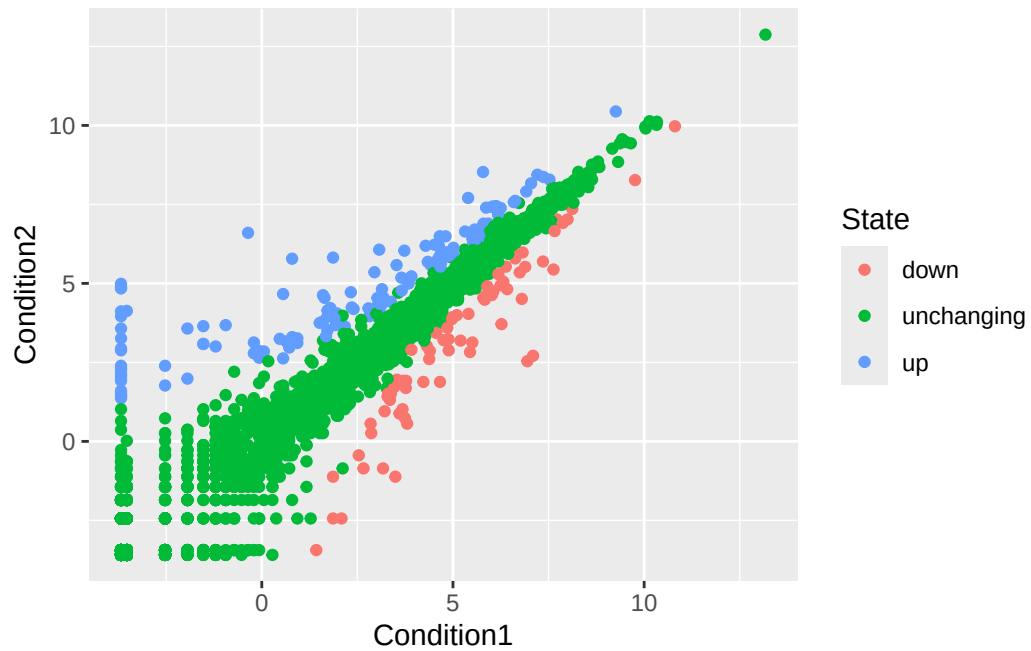
down	unchanging	up
1.39	96.17	2.44

Q. Complete the code below to produce the following plot

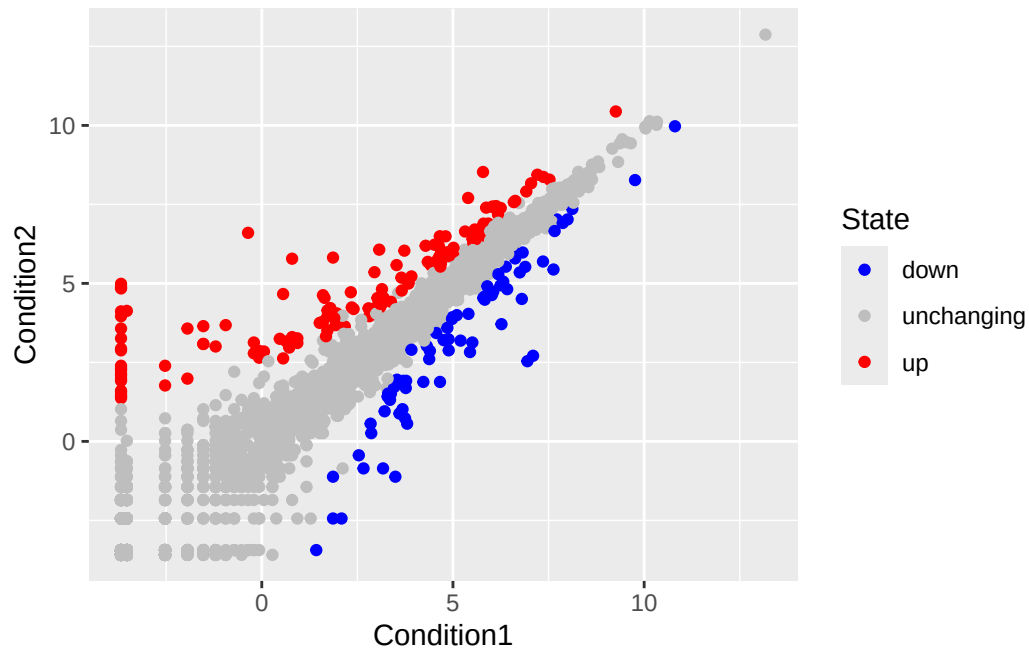
```
#Generating scatter plot of genes data set comparing Condition 1 and Condition 2
ggplot(genes) +
  aes(x=Condition1, y=Condition2) +
  geom_point()
```



```
#Adding State column to see the difference in expression values between conditions
ggplot(genes) +
  aes(x=Condition1, y=Condition2, col=State) +
  geom_point()
```



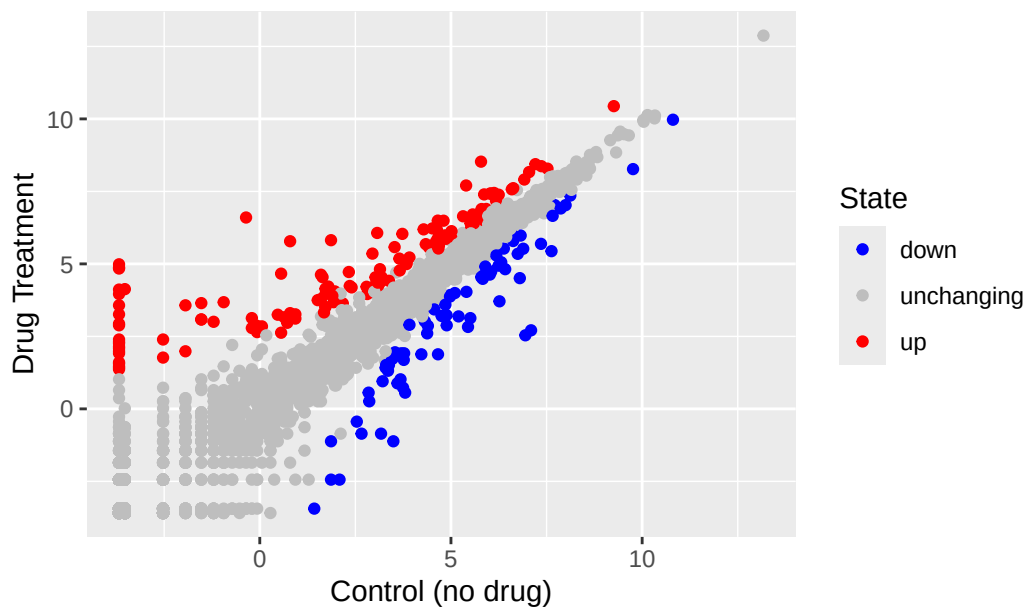
```
#Adding another layer to explicitly specify color scale
ggplot(genes) +
  aes(x=Condition1, y=Condition2, col=State) +
  geom_point() +
  scale_colour_manual( values=c("blue", "gray", "red"))
```



Q. Nice, now add some plot annotations to the p object with the labs() function

```
#Adding plot annotations using labs()
ggplot(genes) +
  aes(x=Condition1, y=Condition2, col=State) +
  geom_point() +
  scale_colour_manual( values=c("blue", "gray", "red")) +
  labs(title="Gene Expression Changes Upon Drug Treatment",
       x="Control (no drug)",
       y="Drug Treatment")
```

Gene Expression Changes Upon Drug Treatment



Part 7. Going Further

```
# File location online
url <- "https://raw.githubusercontent.com/jennybc/gapminder/master/inst/extdata/gapminder.tsv"

gapminder <- read.delim(url)
```

```
# install.packages("dplyr")
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

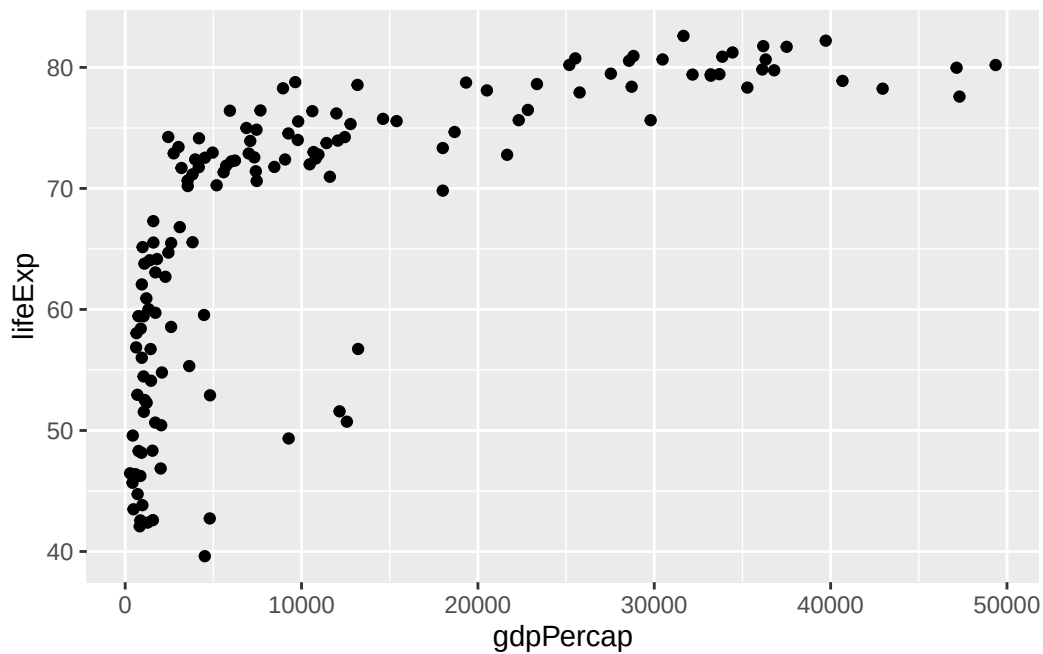
The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

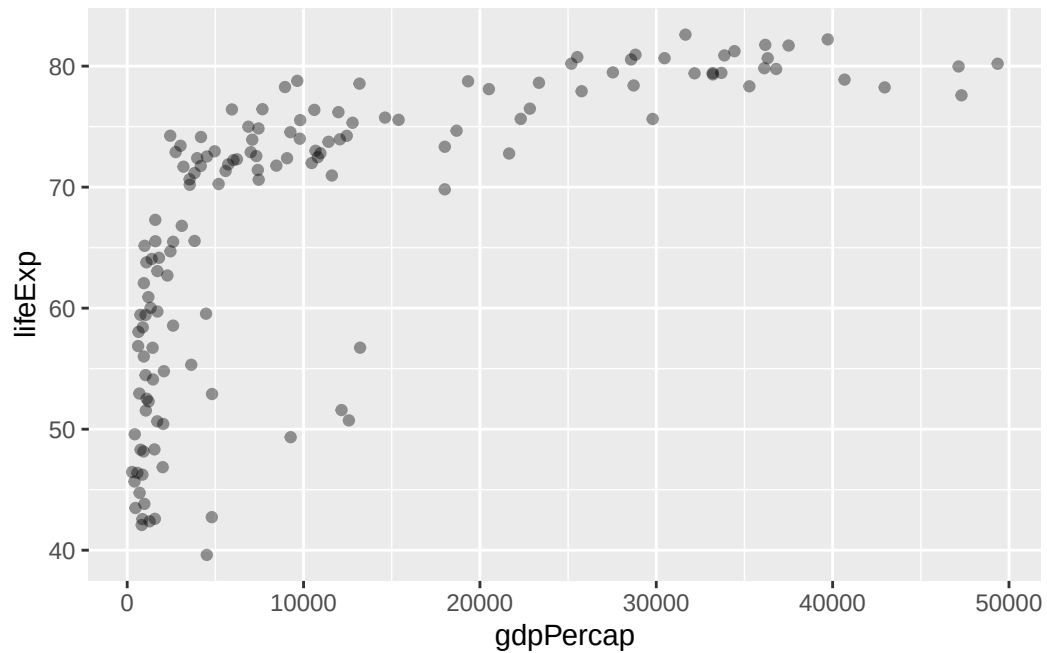
```
gapminder_2007 <- gapminder %>% filter(year==2007)
```

Q. Complete the code below to produce a first basic scatter plot of this gapminder_2007 dataset:

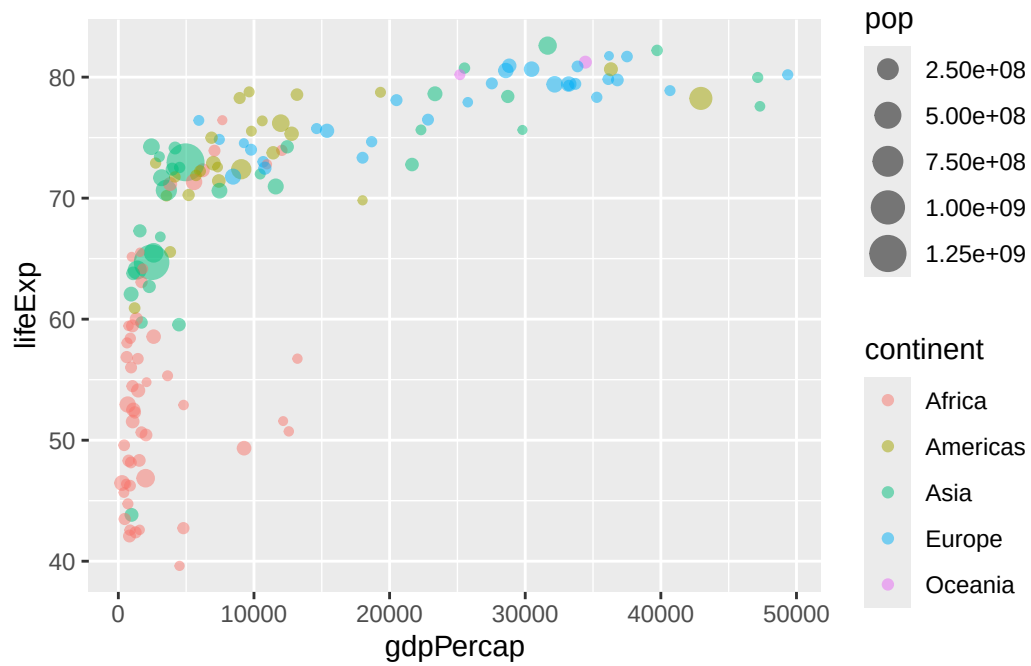
```
#Generating scatter plot of gapminder_2007 dataset
ggplot(gapminder_2007) +
  aes(x=gdpPercap, y=lifeExp) +
  geom_point()
```



```
#Making the points slightly more transparent with alpha=0.4
ggplot(gapminder_2007) +
  aes(x=gdpPercap, y=lifeExp) +
  geom_point(alpha=0.4)
```

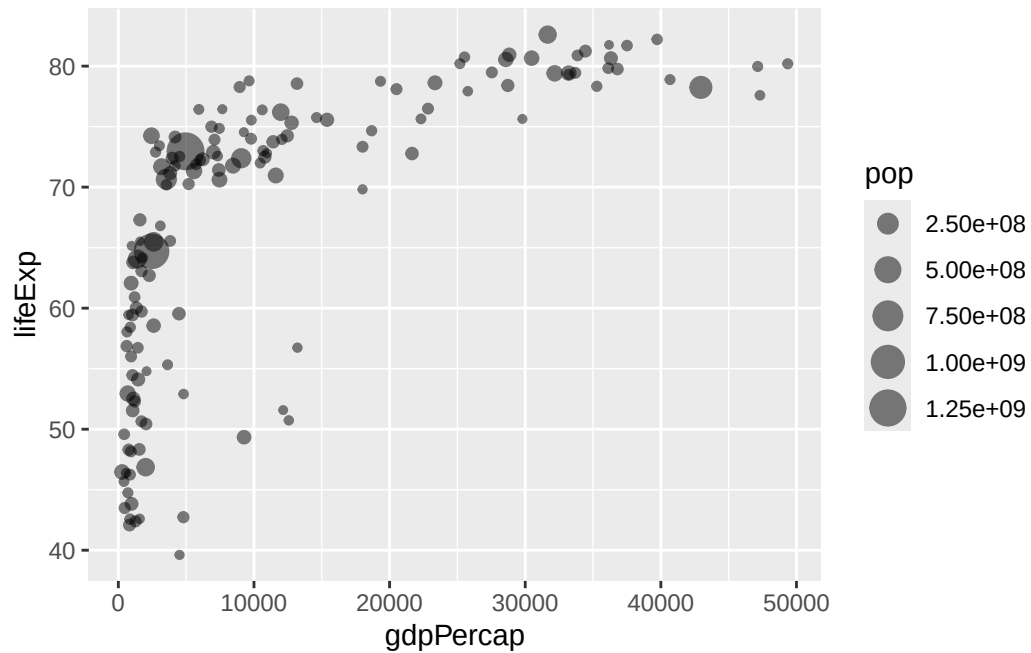
```
#Mapping the continent variable to the point color aesthetic and the population pop
#through the point size argument to aes() to obtain plot with 4 different variables
ggplot(gapminder_2007) +
  aes(x=gdpPercap, y=lifeExp, color=continent, size=pop) +
  geom_point(alpha=0.5)
```



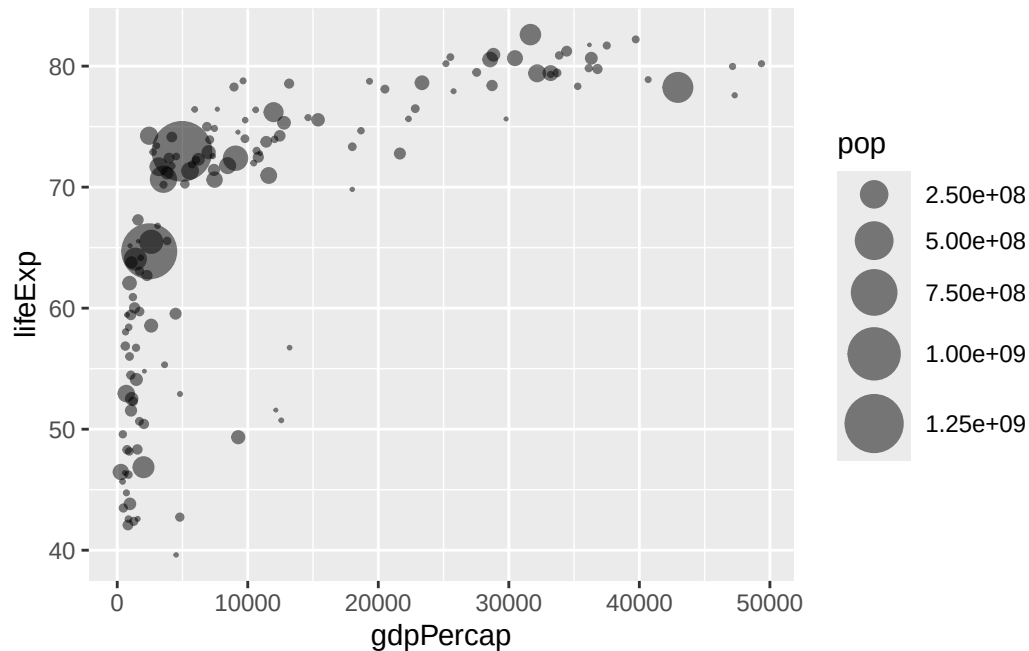
```
#Generating plot and color the point by the numeric variable population pop
ggplot(gapminder_2007) +
  aes(x = gdpPerCap, y = lifeExp, color = pop) +
  geom_point(alpha=0.8)
```



```
#Plotting GDP per capita vs. the life expectancy and set the point size
#based on the population of each country
ggplot(gapminder_2007) +
  aes(x = gdpPerCap, y = lifeExp, size = pop) +
  geom_point(alpha=0.5)
```



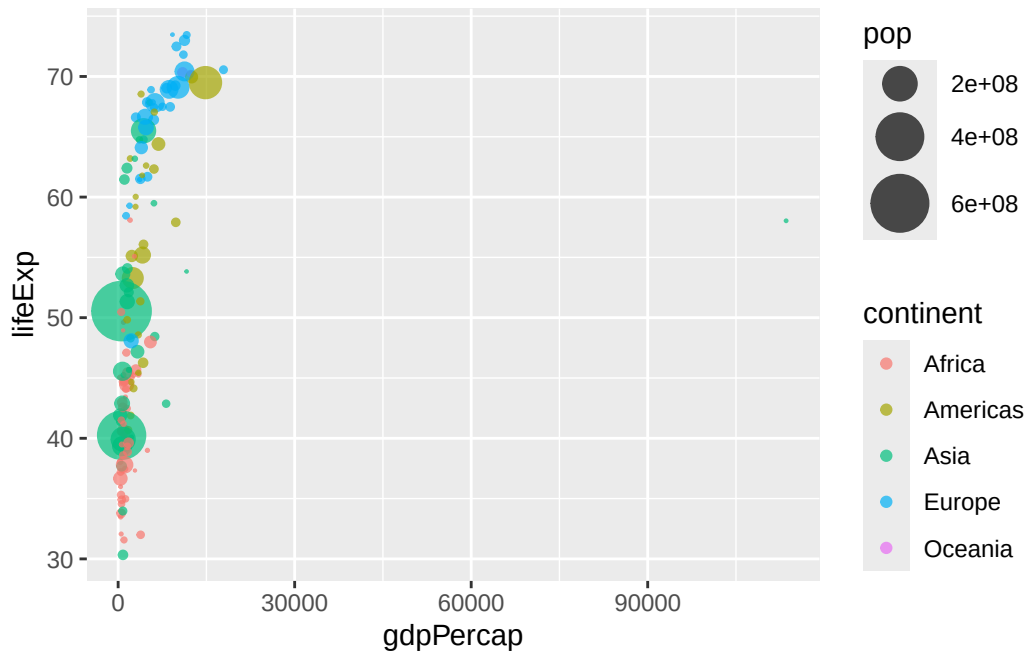
```
#Reflect the actual population differences by the point size and
#using the scale_size_area() function
ggplot(gapminder_2007) +
  geom_point(aes(x = gdpPerCap, y = lifeExp,
                 size = pop), alpha=0.5) +
  scale_size_area(max_size = 10)
```



Q. Can you adapt the code you have learned thus far to reproduce our gapminder scatter plot for the year 1957? What do you notice about this plot is it easy to compare with the one for 2007?

```
#Generating gapminder scatter plot for year 1957
gapminder_1957 <- gapminder %>% filter(year==1957)

ggplot(gapminder_1957) +
  aes(x = gdpPercap, y = lifeExp, color=continent,
      size = pop) +
  geom_point(alpha=0.7) +
  scale_size_area(max_size = 10)
```

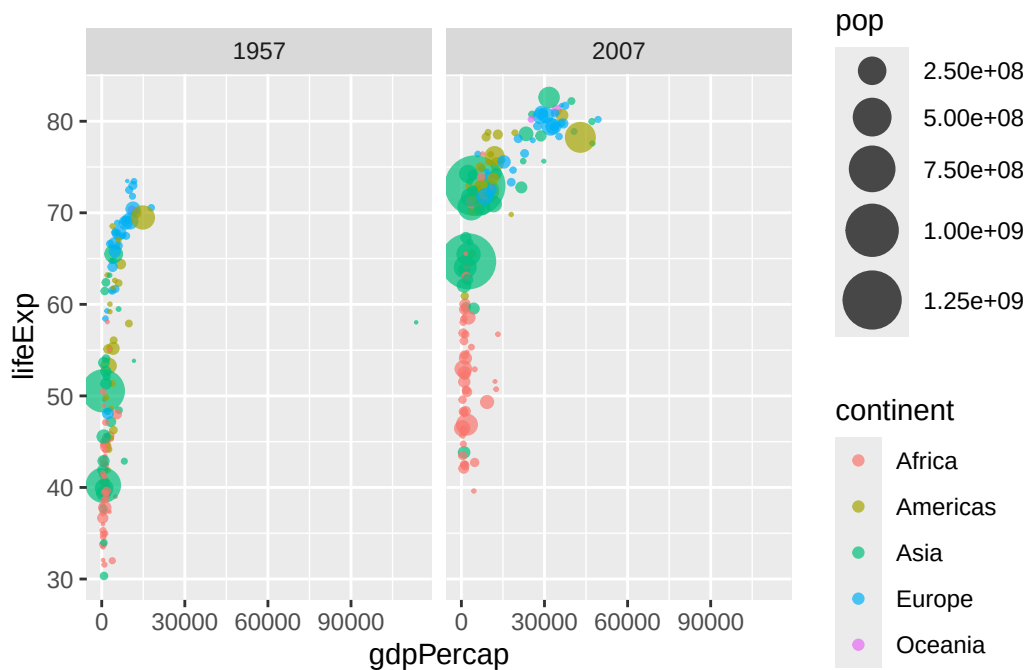


Q. Do the same steps above but include 1957 and 2007 in your input dataset for `ggplot()`. You should now include the layer `facet_wrap(~year)` to produce the following plot:

```
#Generating scatter plot to include both 1957 and 2007

gapminder_1957 <- gapminder %>% filter(year==1957 | year==2007)

ggplot(gapminder_1957) +
  geom_point(aes(x = gdpPercap, y = lifeExp, color=continent,
                 size = pop), alpha=0.7) +
  scale_size_area(max_size = 10) +
  facet_wrap(~year)
```



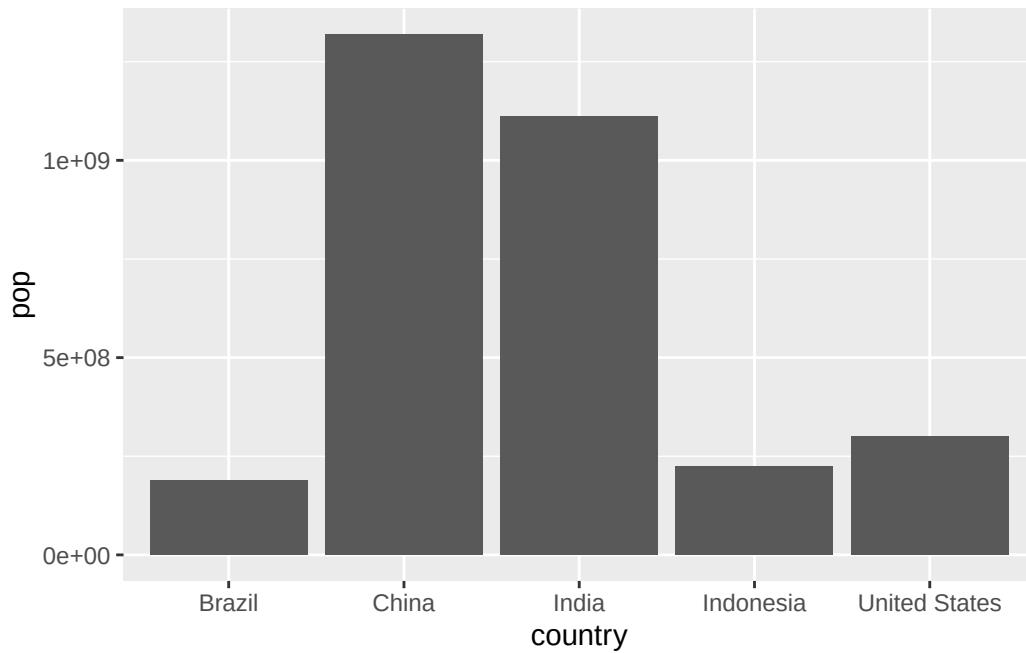
Part 8: Bar Charts

```
#Generating the number of people in the five biggest countries by population in 2007
gapminder_top5 <- gapminder %>%
  filter(year==2007) %>%
  arrange(desc(pop)) %>%
  top_n(5, pop)

gapminder_top5
```

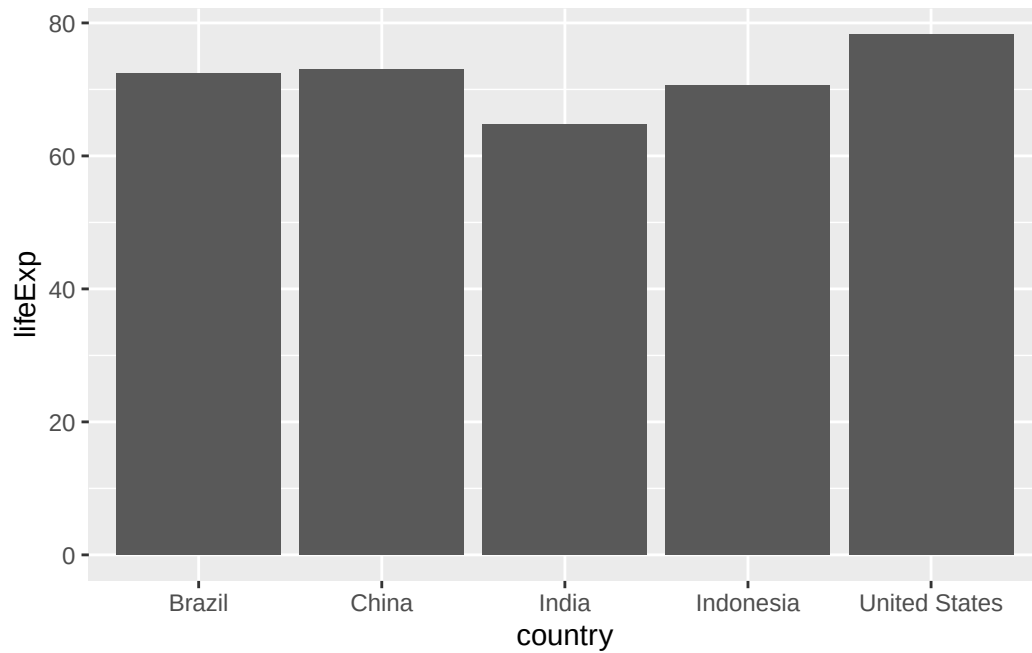
	country	continent	year	lifeExp	pop	gdpPercap
1	China	Asia	2007	72.961	1318683096	4959.115
2	India	Asia	2007	64.698	1110396331	2452.210
3	United States	Americas	2007	78.242	301139947	42951.653
4	Indonesia	Asia	2007	70.650	223547000	3540.652
5	Brazil	Americas	2007	72.390	190010647	9065.801

```
#Generating bar chart with gapminder_top5 dataset
ggplot(gapminder_top5) +
  geom_col(aes(x = country, y = pop))
```

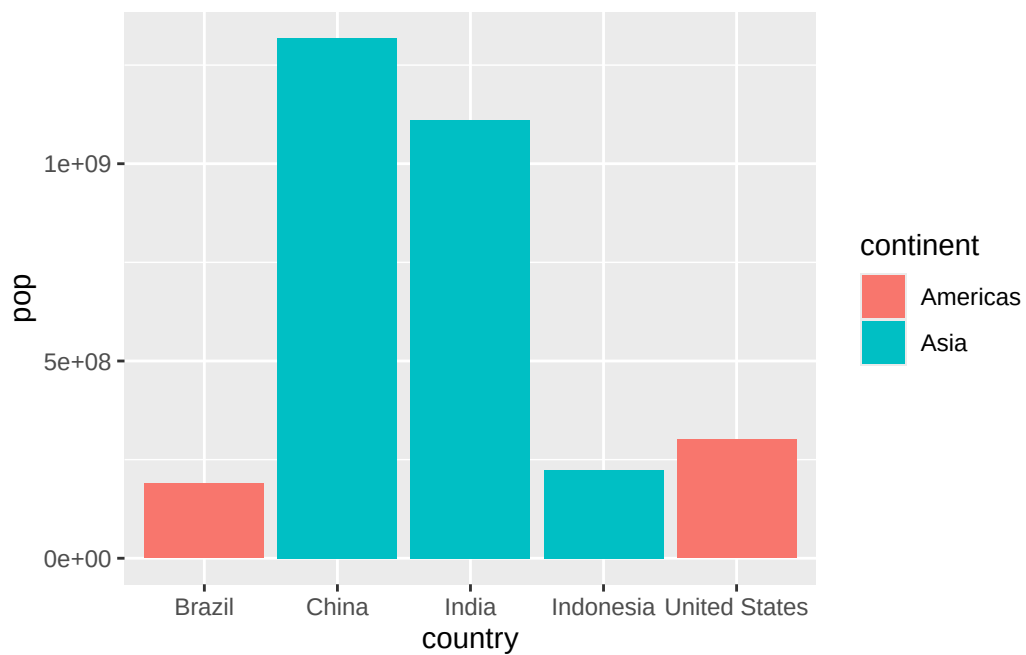


Q Create a bar chart showing the life expectancy of the five biggest countries by population in 2007.

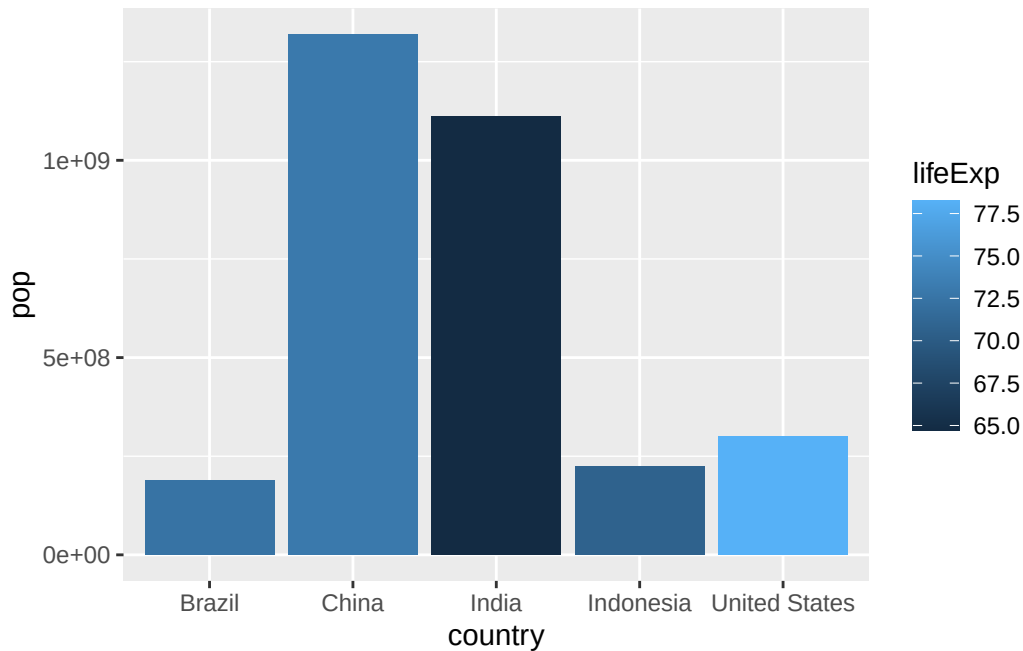
```
# Generating bar chart with gapminder_top5 dataset
ggplot(gapminder_top5) +
  geom_col(aes(x = country, y = lifeExp))
```

```
#Filling the bars with color
ggplot(gapminder_top5) +
  geom_col(aes(x = country, y = pop, fill = continent))
```

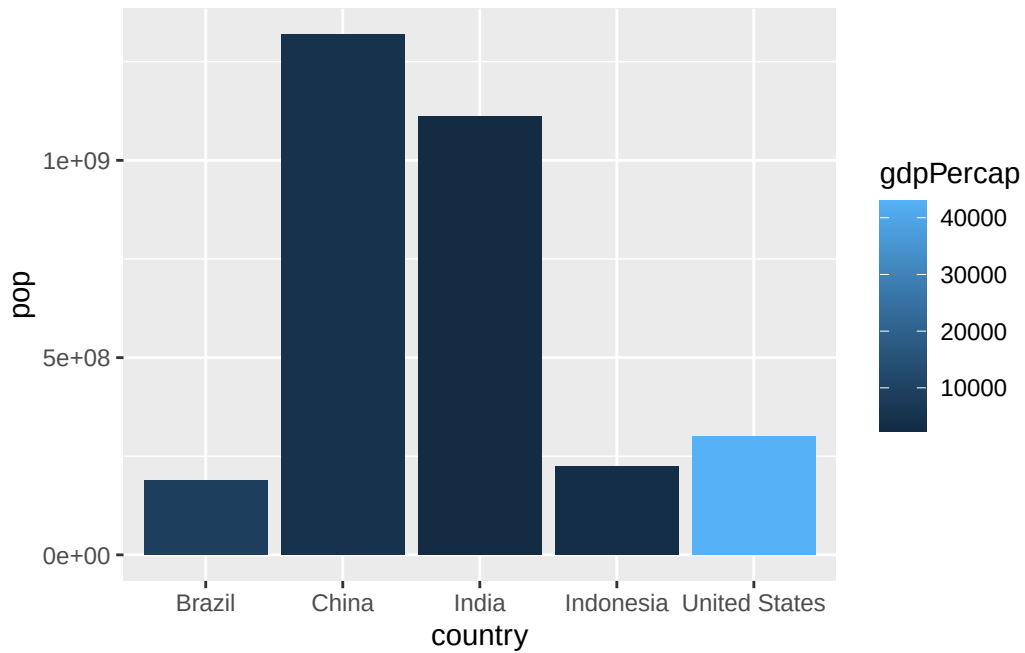


```
#Using numeric variable (life expectancy) to fill color
ggplot(gapminder_top5) +
  geom_col(aes(x = country, y = pop, fill = lifeExp))
```

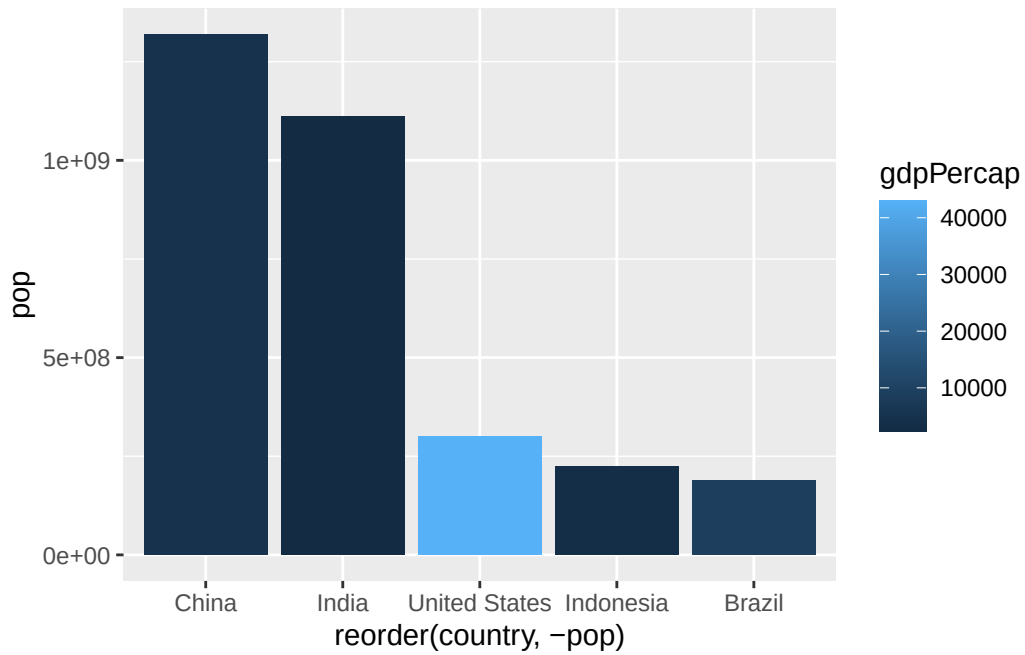


Q. Plot population size by country. Create a bar chart showing the population (in millions) of the five biggest countries by population in 2007.

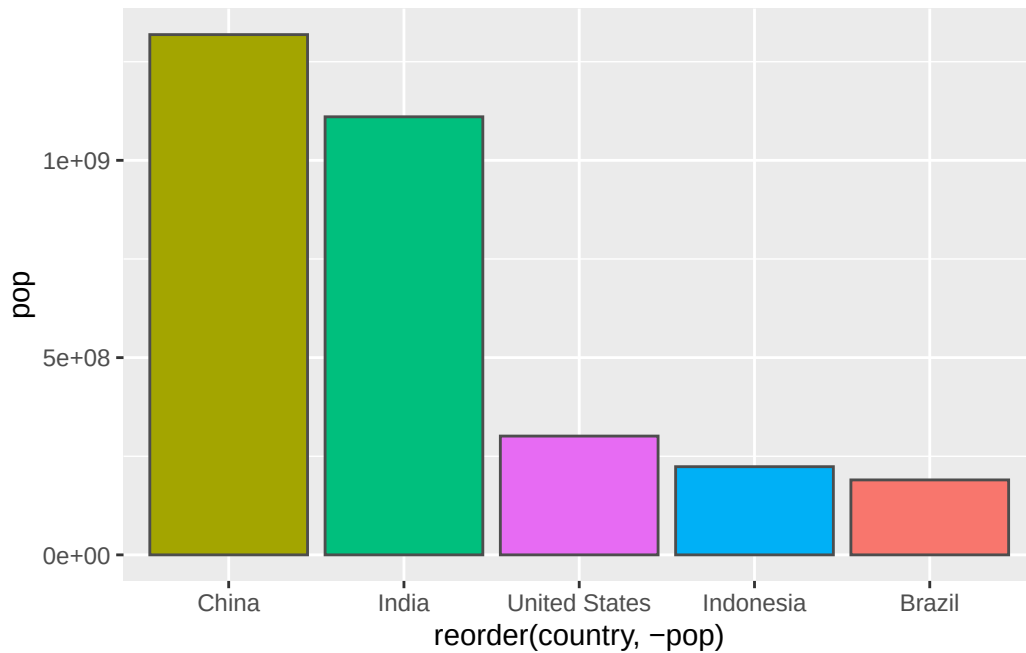
```
#Generating bar chart showing population (in millions) of the five biggest countries by popul
ggplot(gapminder_top5) +
  aes(x=country, y=pop, fill=gdpPercap) +
  geom_col()
```



```
#Changing the order of the bars
ggplot(gapminder_top5) +
  aes(x=reorder(country, -pop), y=pop, fill=gdpPercap) +
  geom_col()
```



```
#Filling just by country
ggplot(gapminder_top5) +
  aes(x=reorder(country, -pop), y=pop, fill=country) +
  geom_col(col="gray30") +
  guides(fill="none")
```

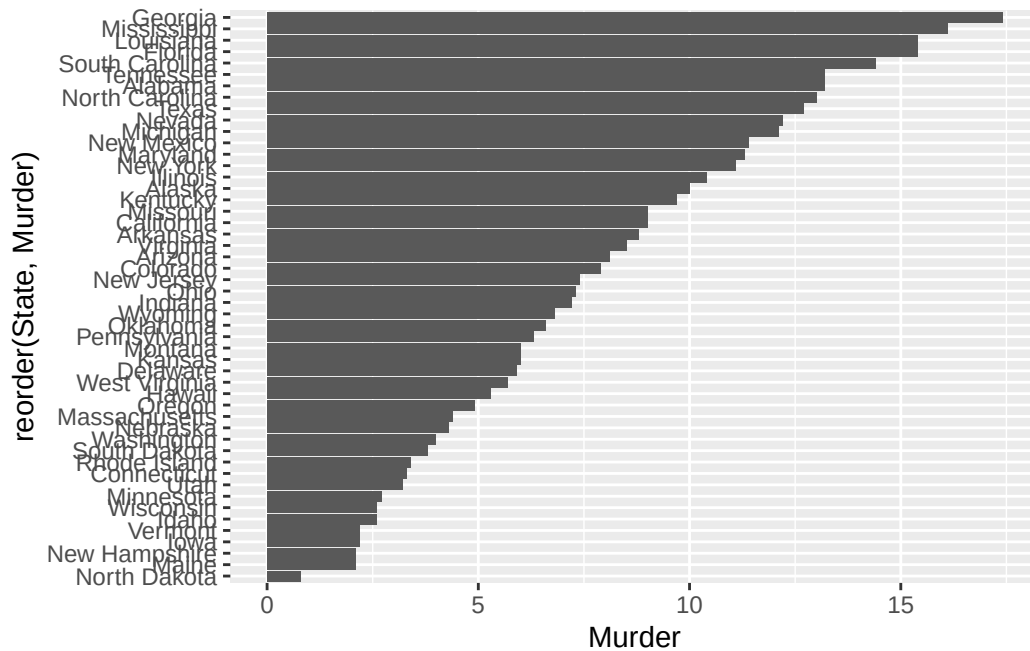


Flipping bar charts

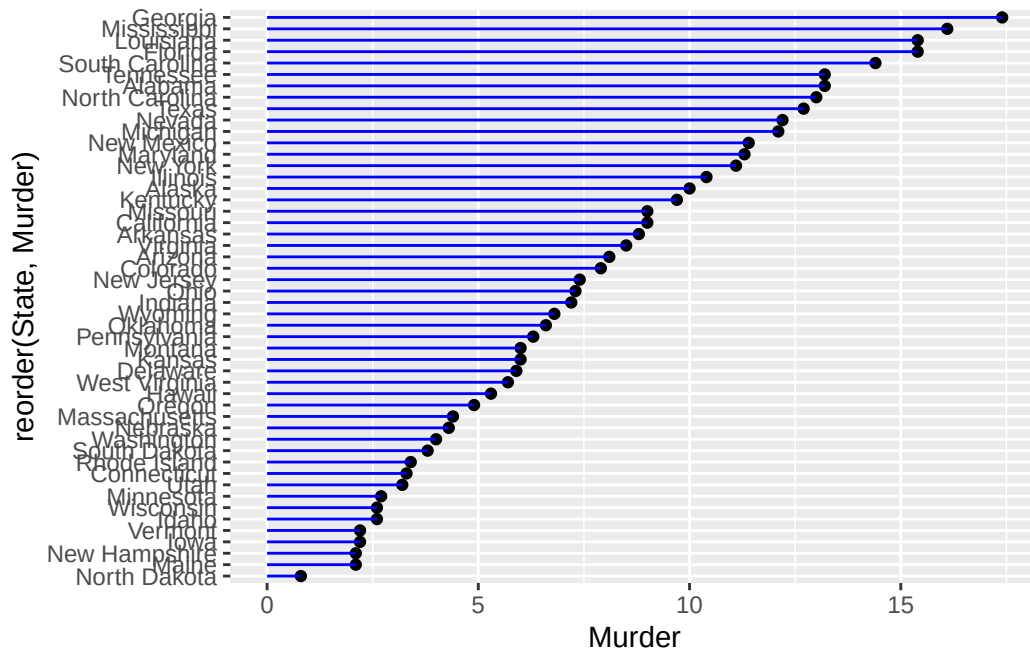
```
#Loading dataset USArrests
head(USArrests)
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0
Arkansas	8.8	190	50	19.5
California	9.0	276	91	40.6
Colorado	7.9	204	78	38.7

```
#Generating bar chart of USArrests
USArrests$State <- rownames(USArrests)
ggplot(USArrests) +
  aes(x=reorder(State,Murder), y=Murder) +
  geom_col() +
  coord_flip()
```



```
#Plotting USArrests by using points and line segments
ggplot(USArrests) +
  #Mapping variables
  aes(x=reorder(State,Murder), y=Murder) +
  geom_point() +
  #Adding verticle line segments from 0 up to each state's murder rate
  geom_segment(aes(x=State,
                   xend=State,
                   y=0,
                   yend=Murder), color="blue") +
  #Flip the axes so states are on the y=axis
  coord_flip()
```



Part 9. Extensions: Animation

```
library(gapminder)
```

Attaching package: 'gapminder'

The following object is masked _by_ '.GlobalEnv':

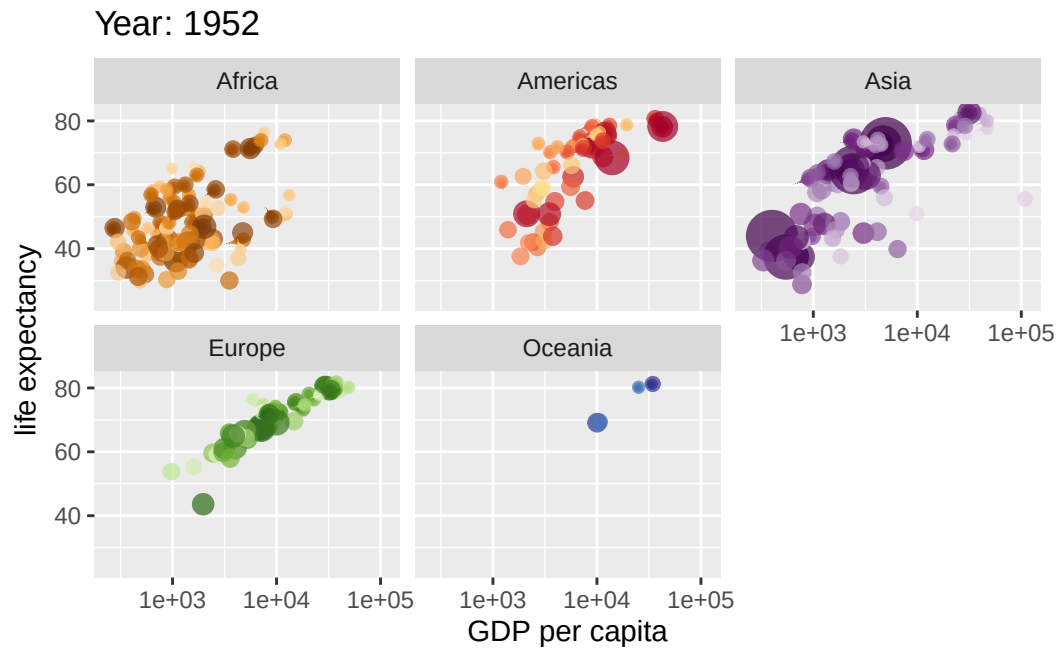
gapminder

```
library(gganimate)

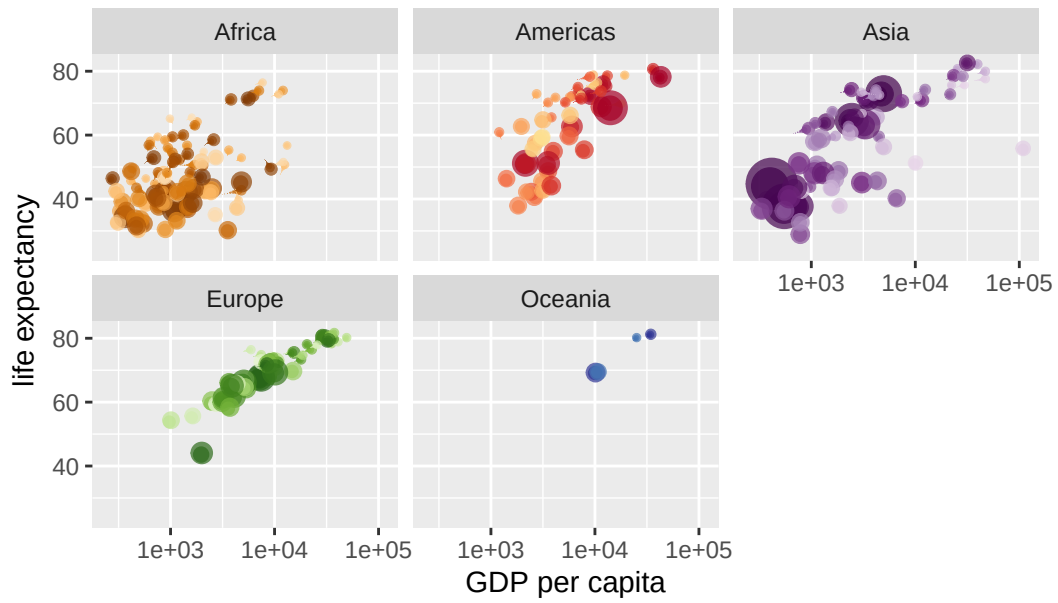
# Setup nice regular ggplot of the gapminder data
ggplot(gapminder, aes(gdpPercap, lifeExp, size = pop, colour = country)) +
  geom_point(alpha = 0.7, show.legend = FALSE) +
  scale_colour_manual(values = country_colors) +
  scale_size(range = c(2, 12)) +
  scale_x_log10() +
  # Facet by continent
  facet_wrap(~continent) +
  # Here comes the gganimate specific bits
```

```
labs(title = 'Year: {frame_time}', x = 'GDP per capita', y = 'life expectancy') +
transition_time(year) +
shadow_wake(wake_length = 0.1, alpha = FALSE)
```

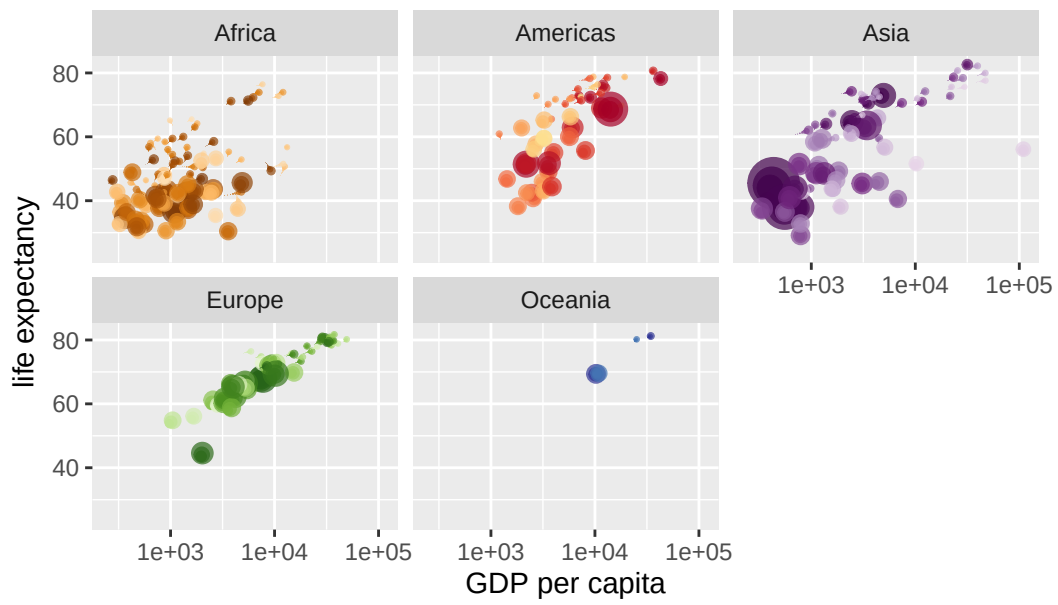
Warning in `formals(fun): argument is not a function`



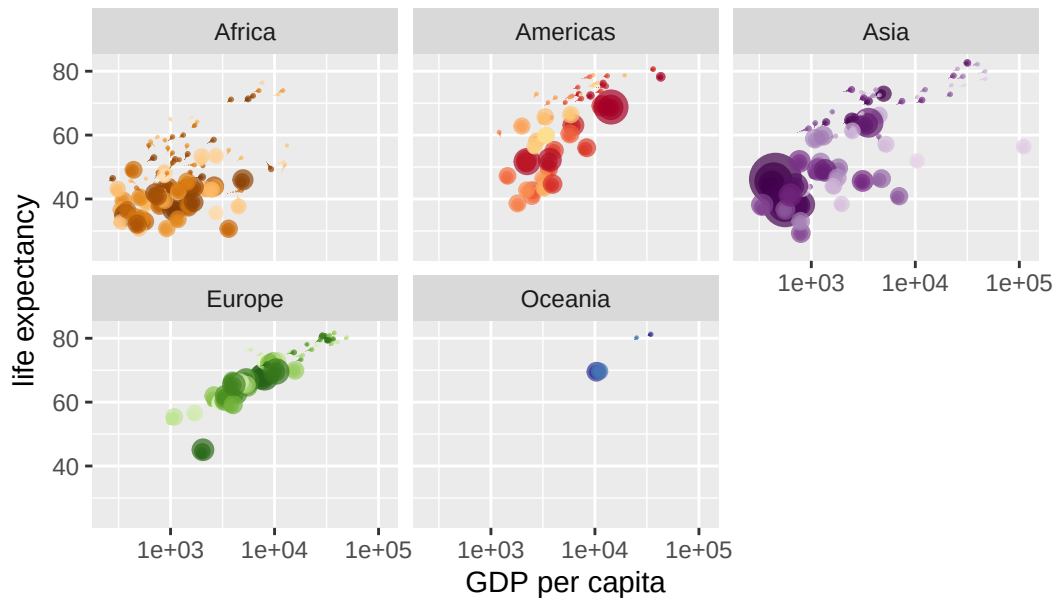
Year: 1953



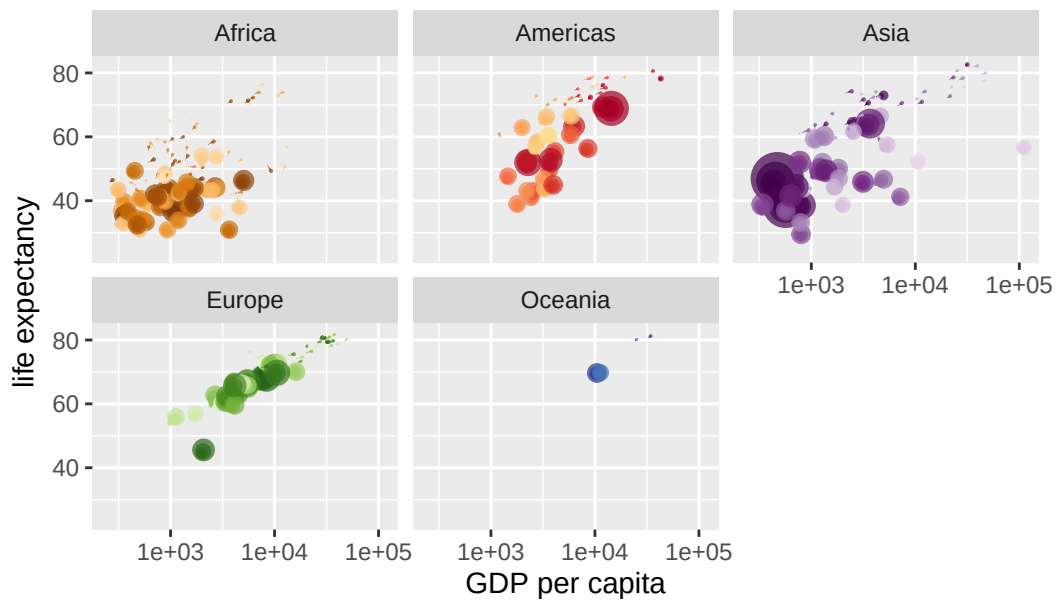
Year: 1953



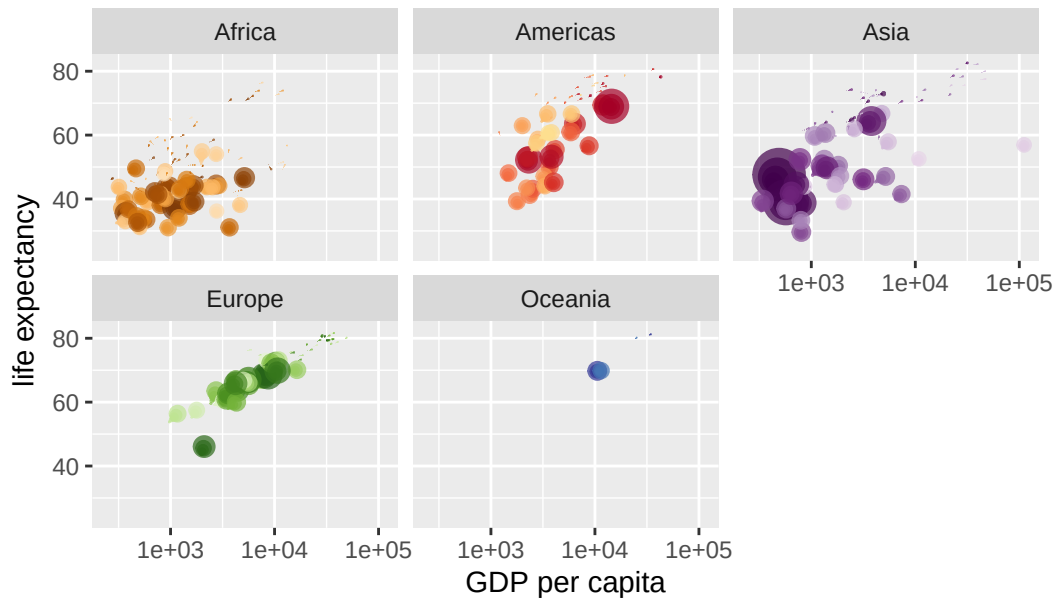
Year: 1954



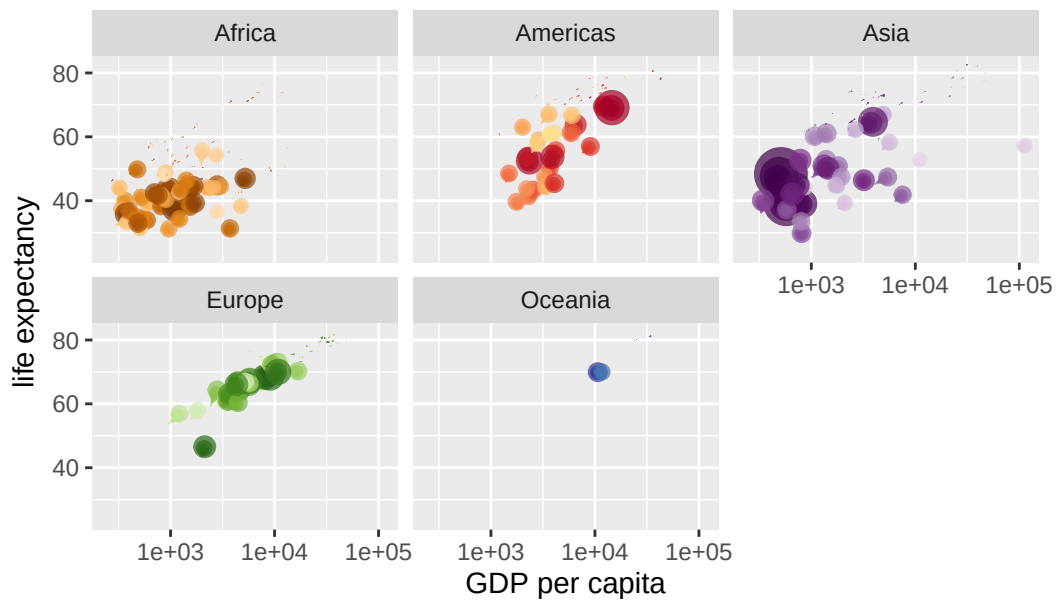
Year: 1954



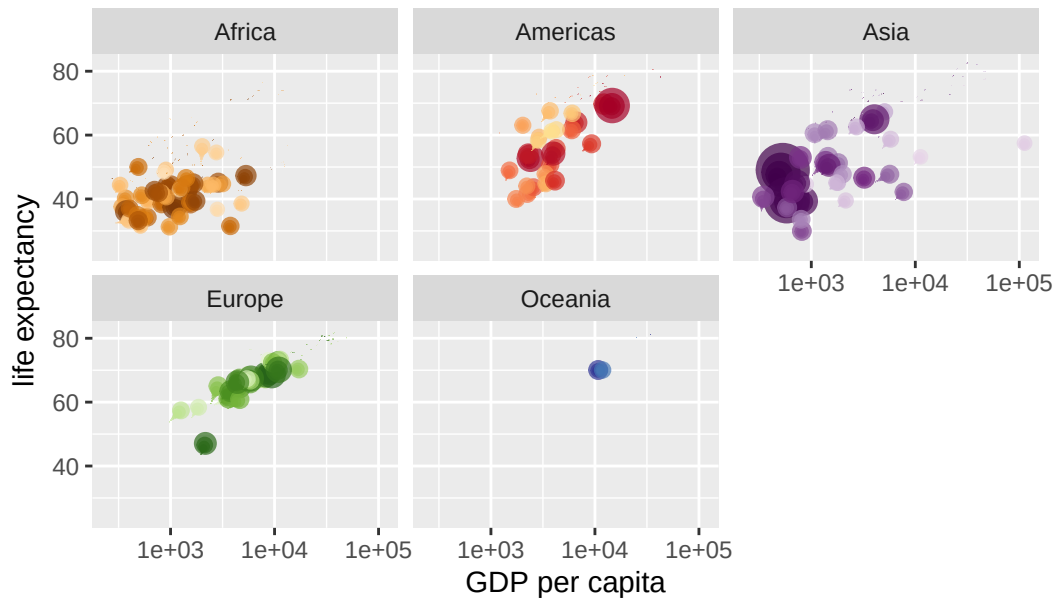
Year: 1955



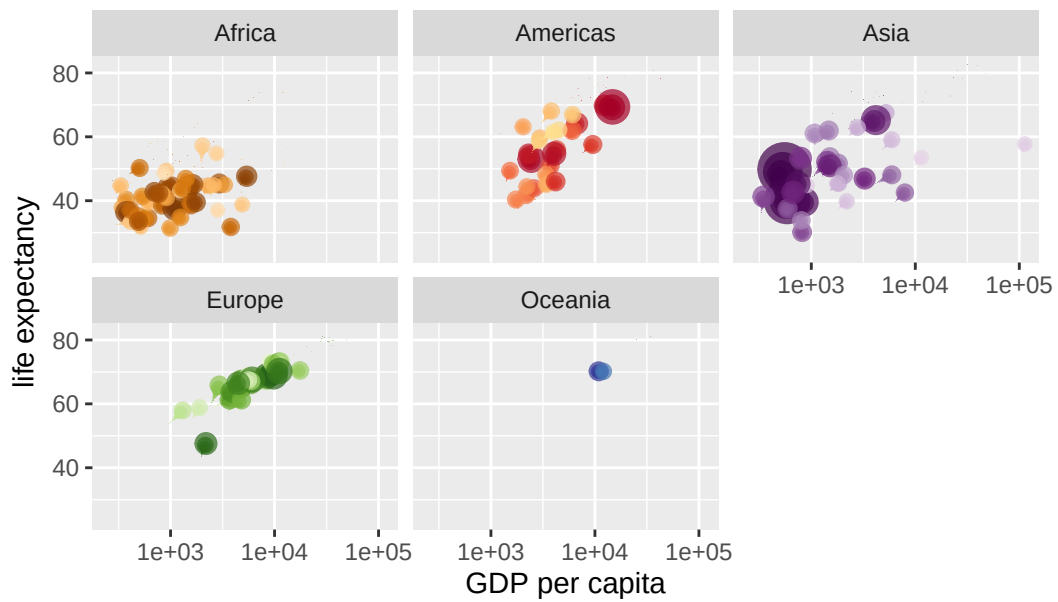
Year: 1955



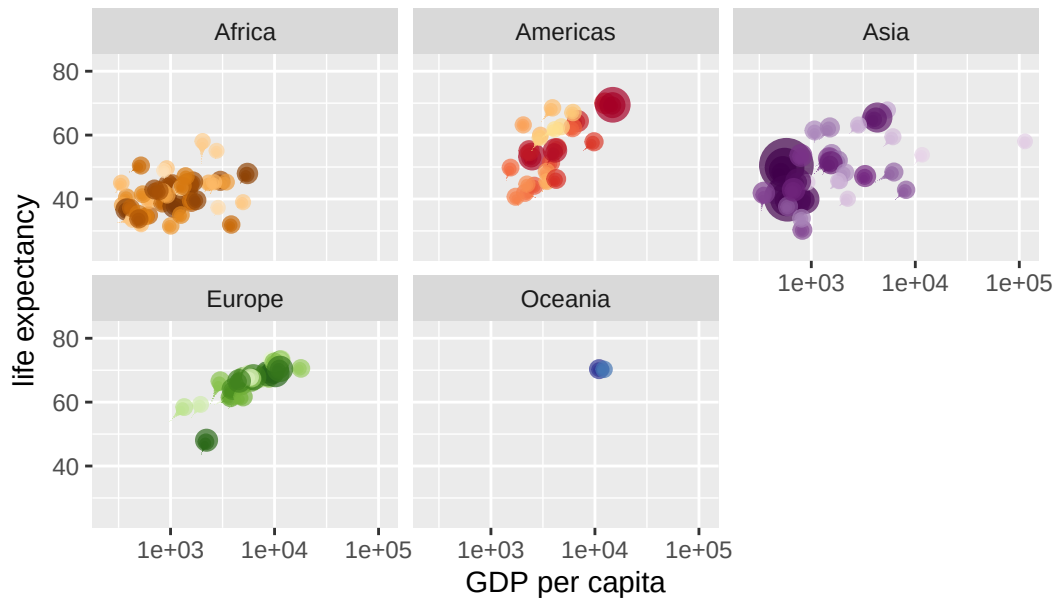
Year: 1956



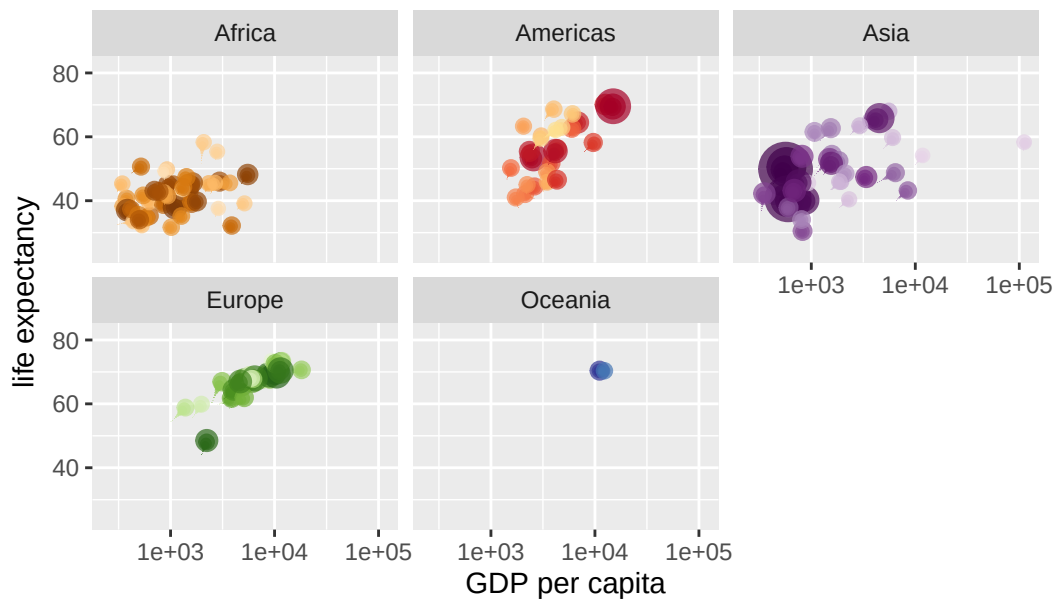
Year: 1956



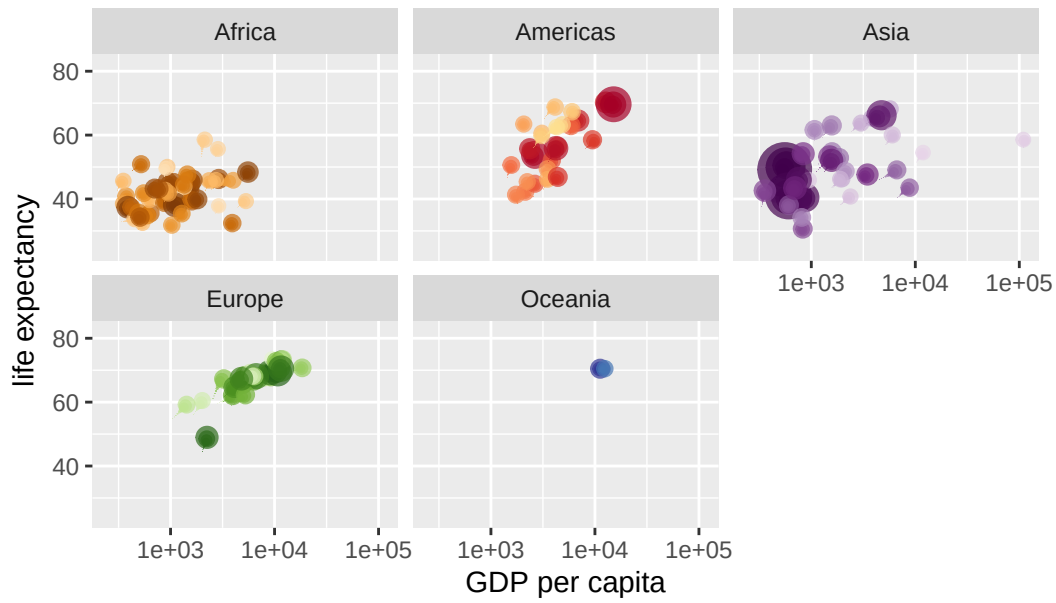
Year: 1957



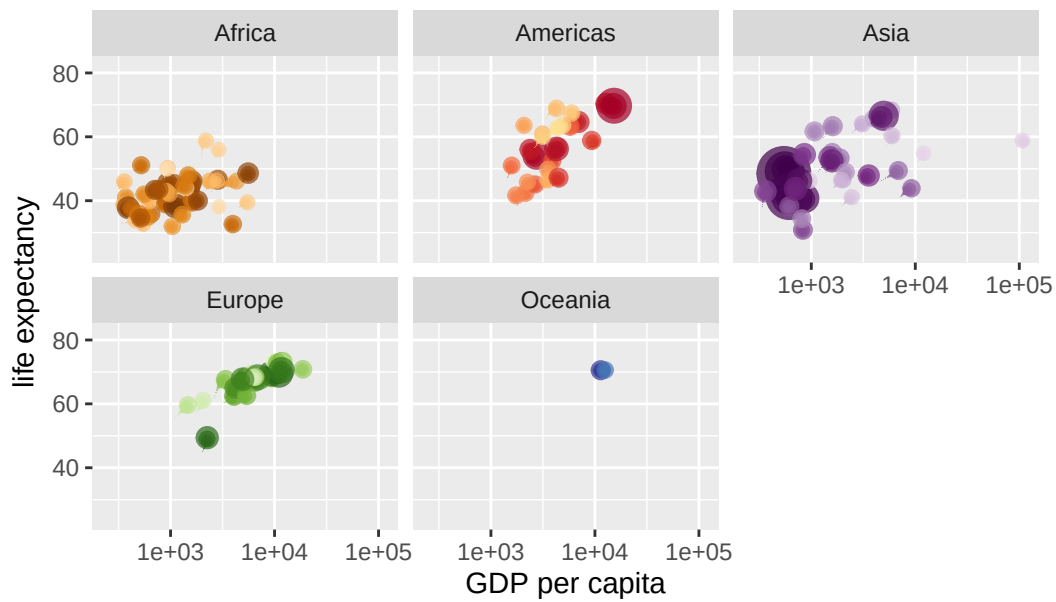
Year: 1958



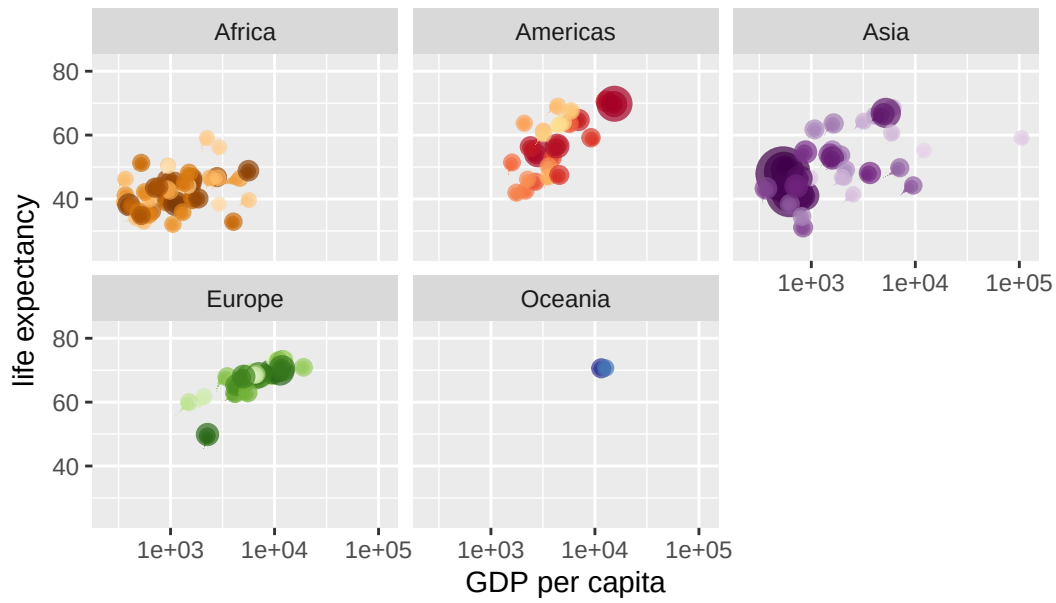
Year: 1958



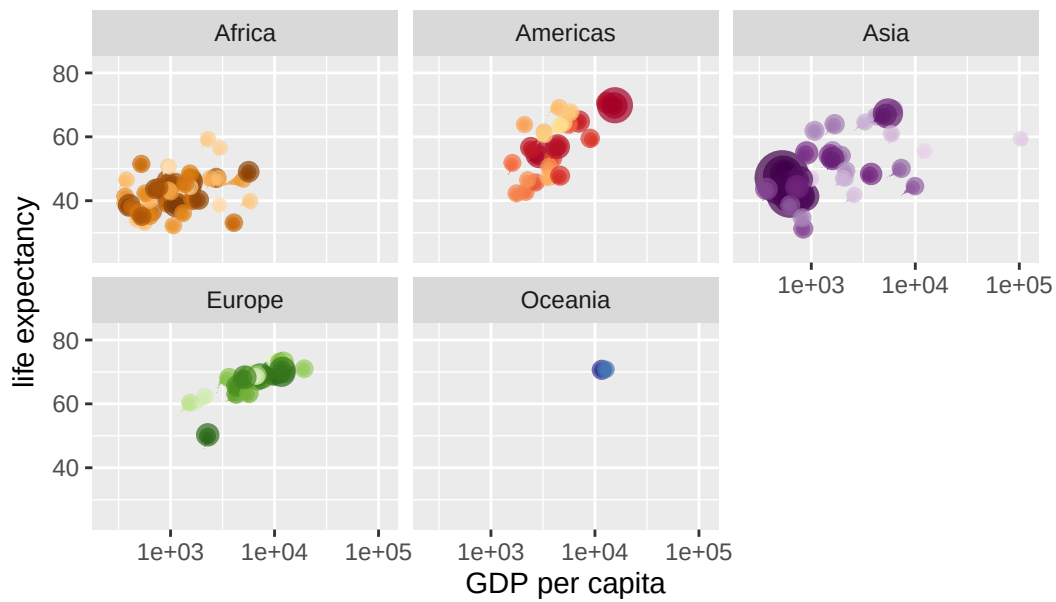
Year: 1959



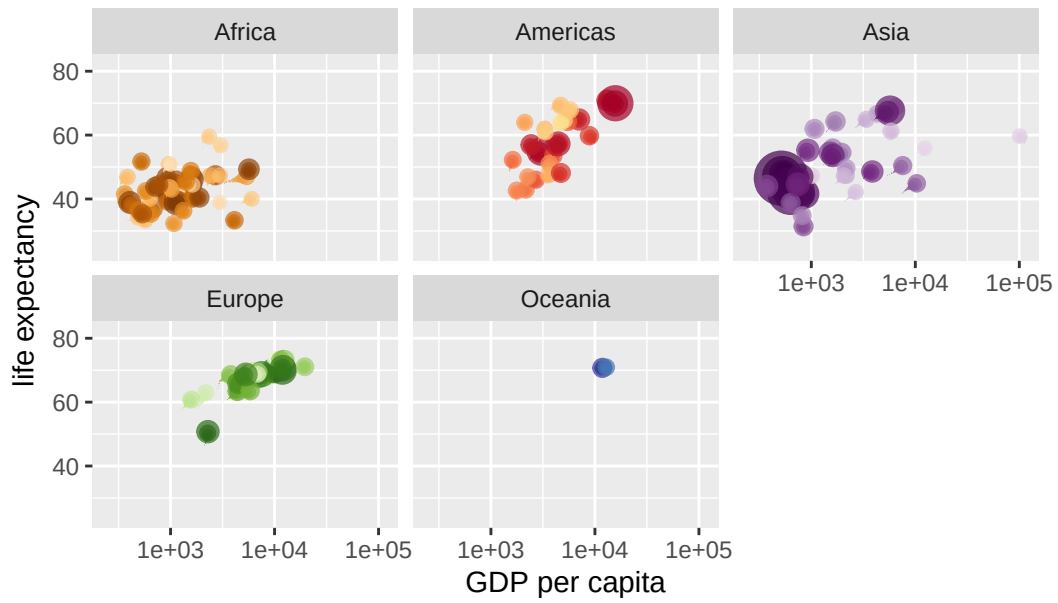
Year: 1959



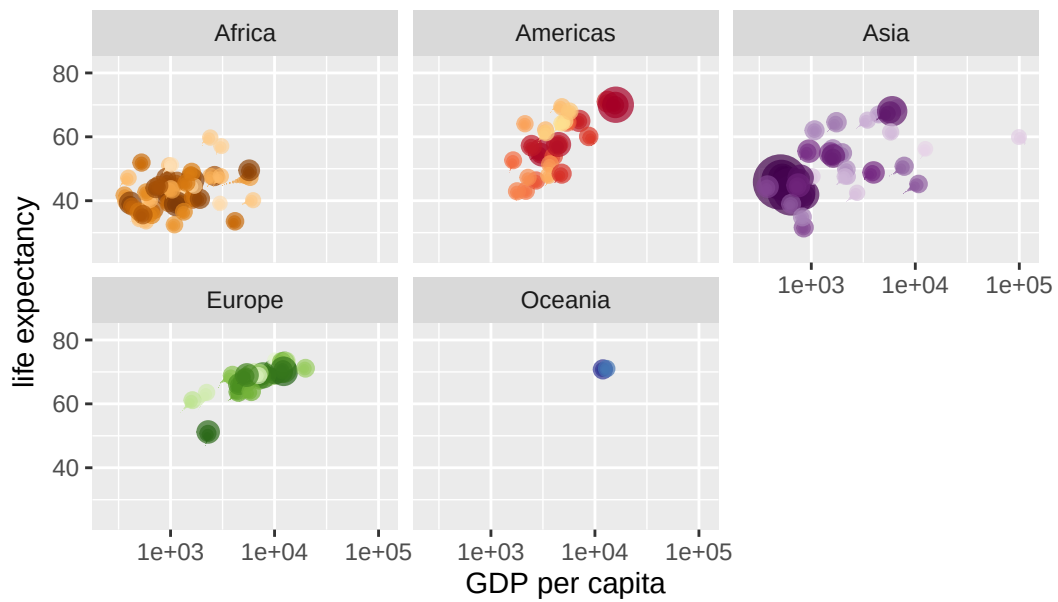
Year: 1960



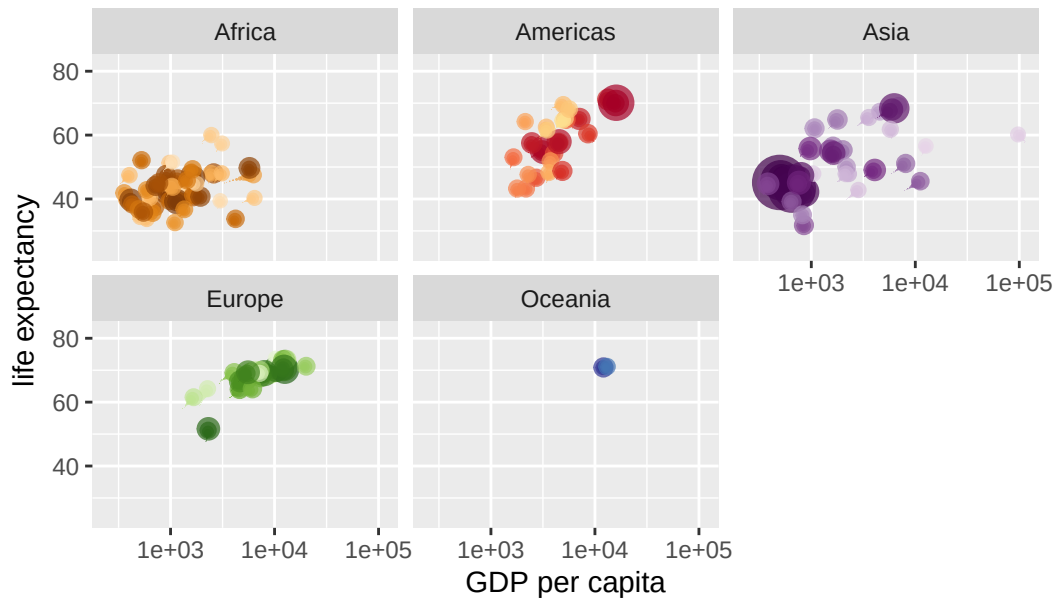
Year: 1960



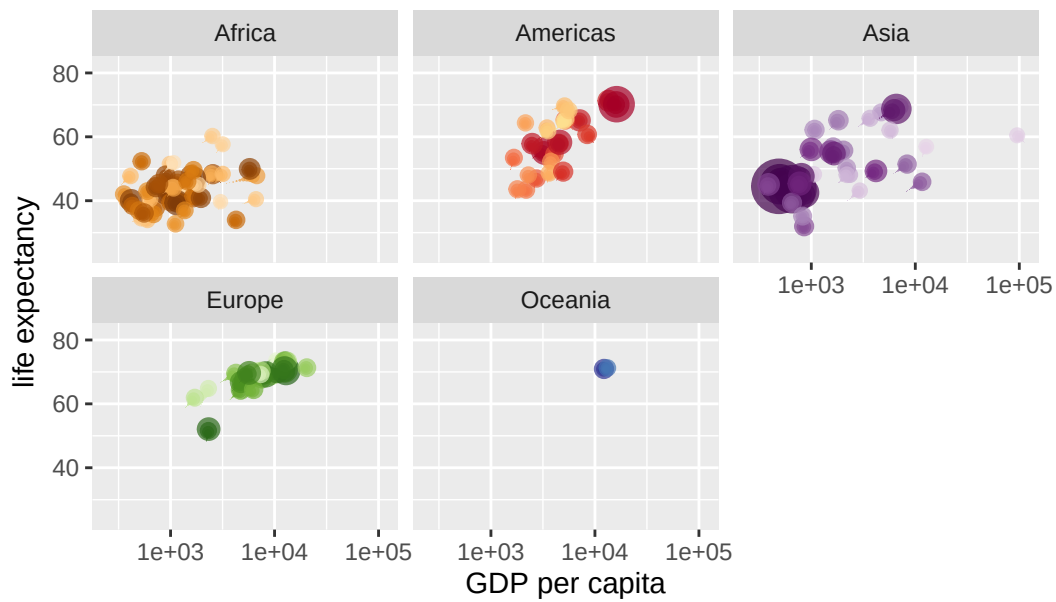
Year: 1961



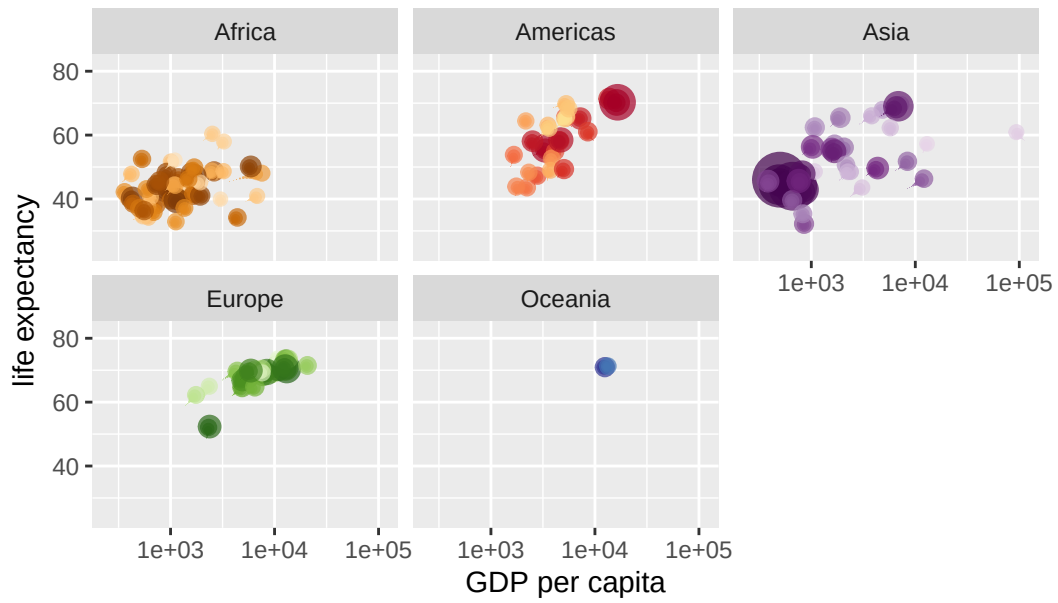
Year: 1961



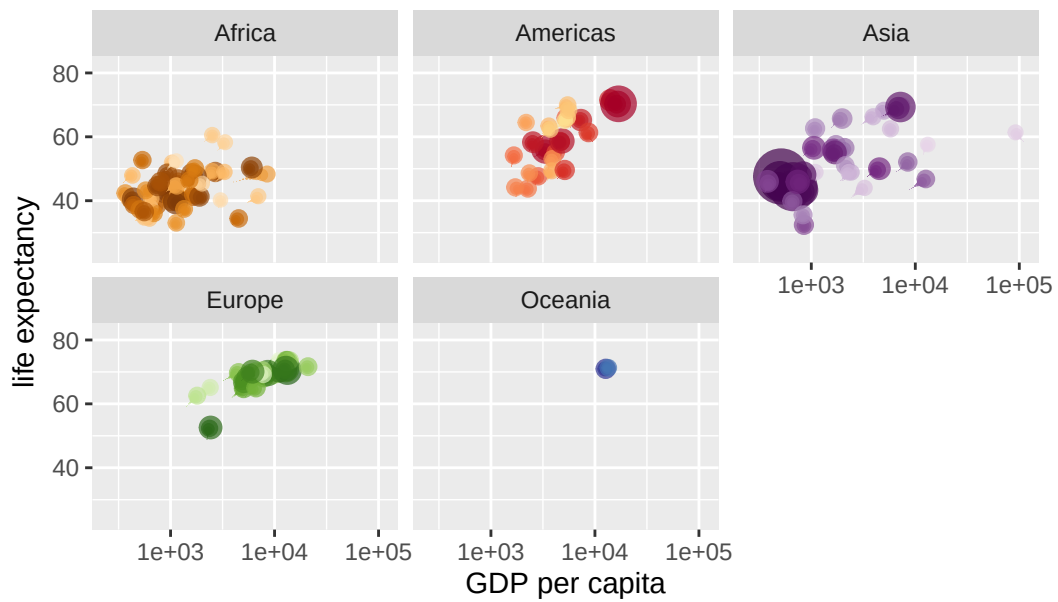
Year: 1962



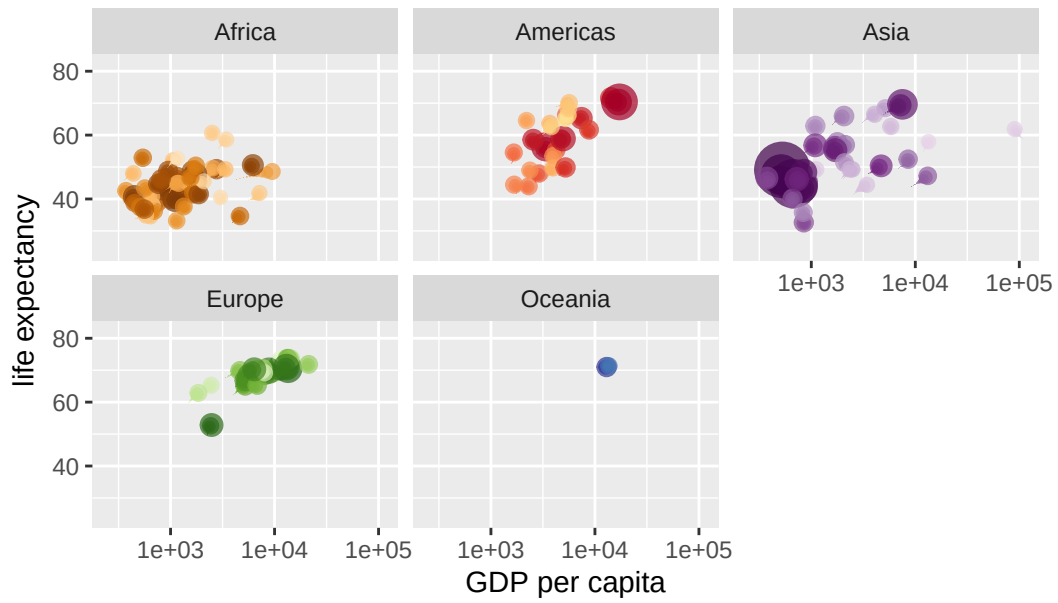
Year: 1963



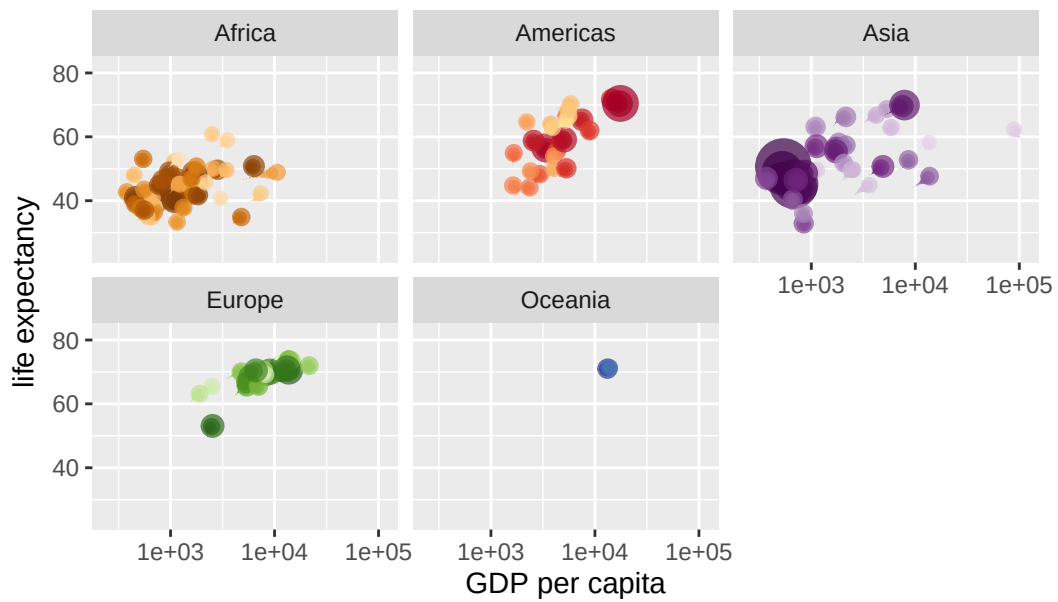
Year: 1963



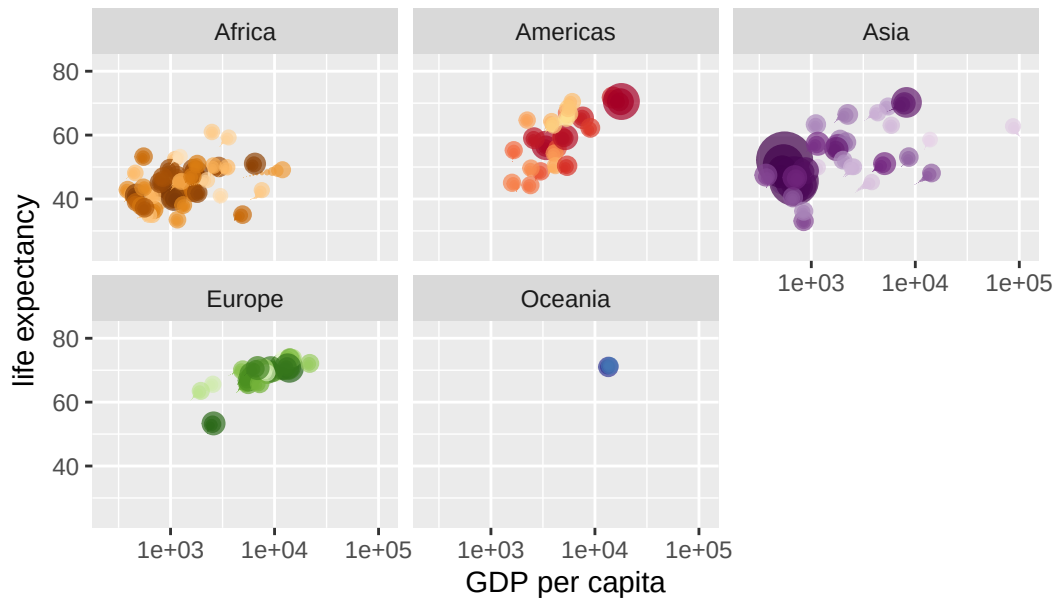
Year: 1964



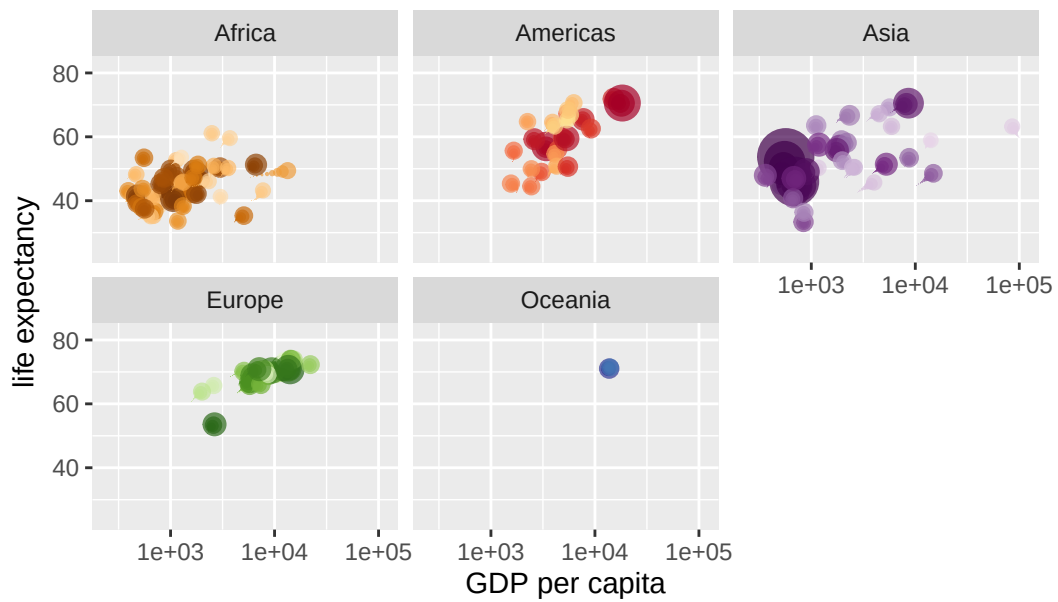
Year: 1964



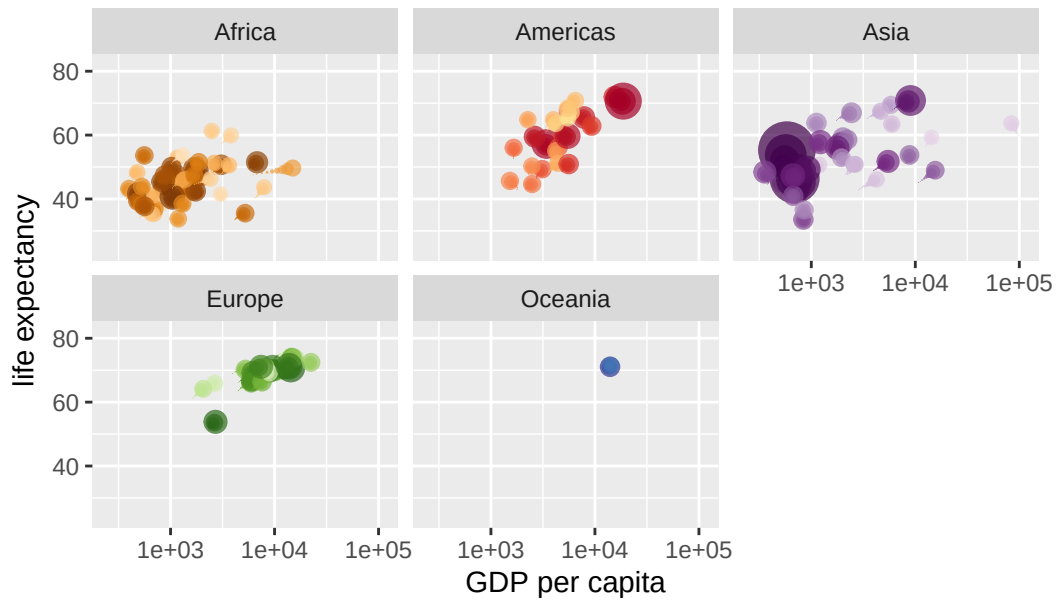
Year: 1965



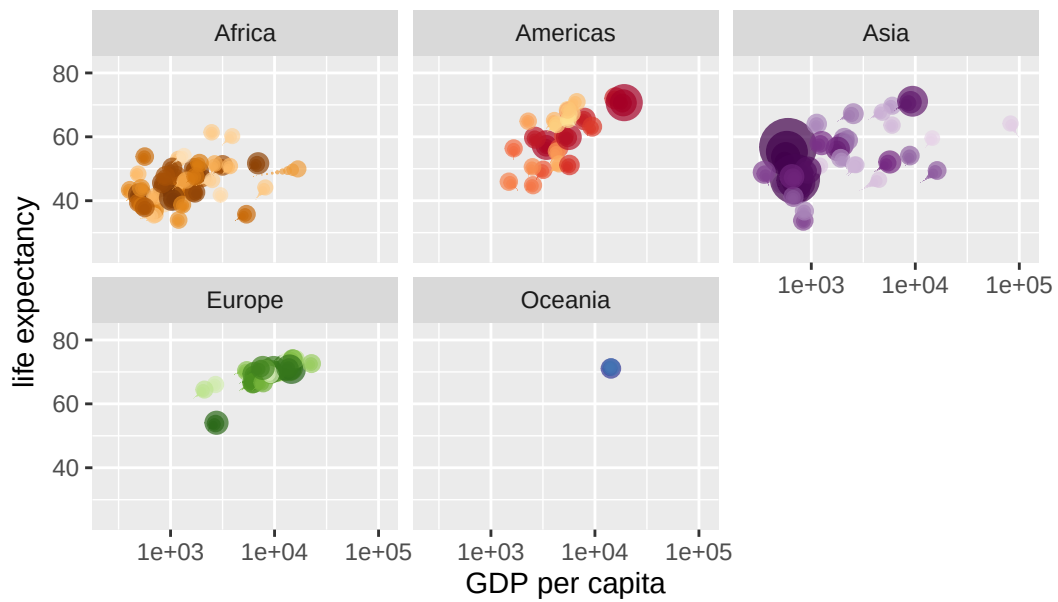
Year: 1965



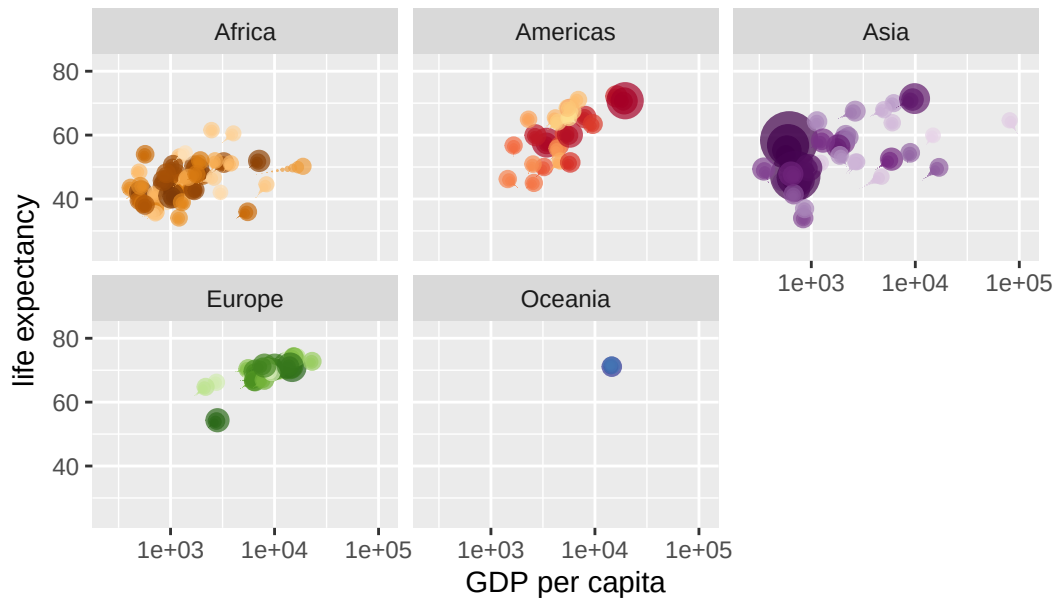
Year: 1966



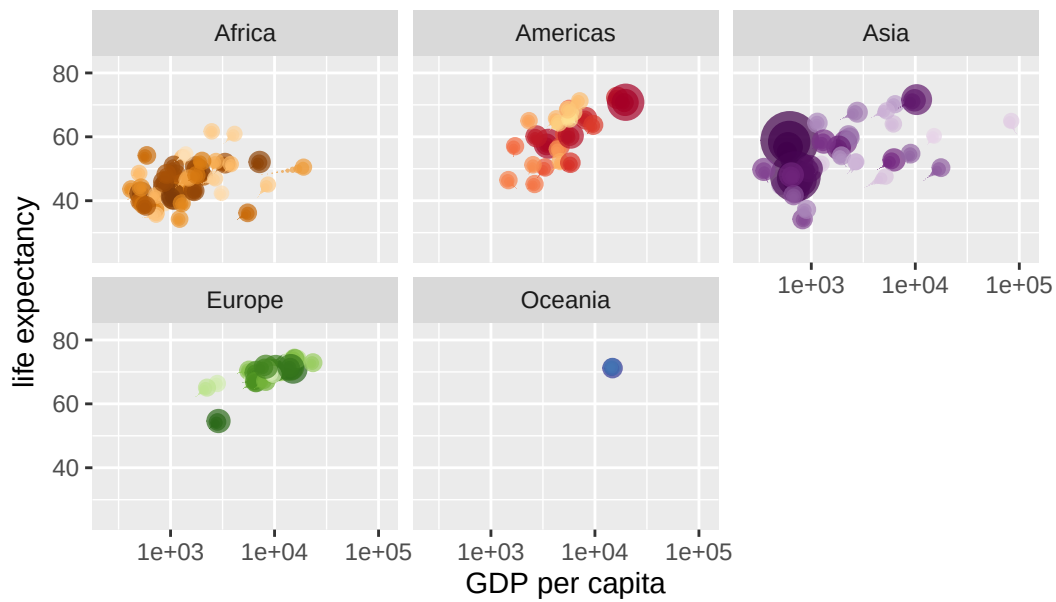
Year: 1966



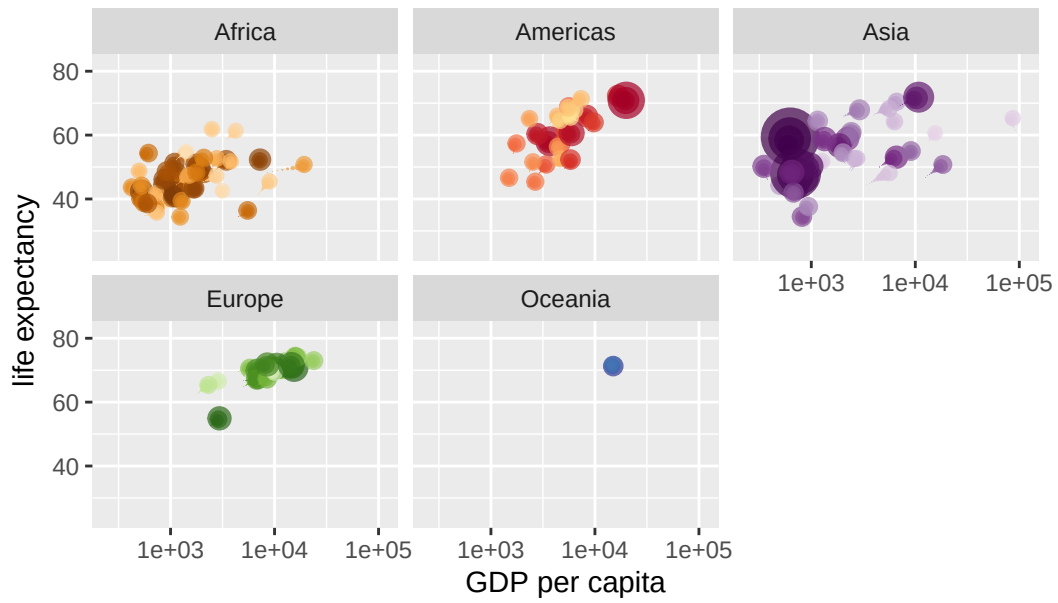
Year: 1967



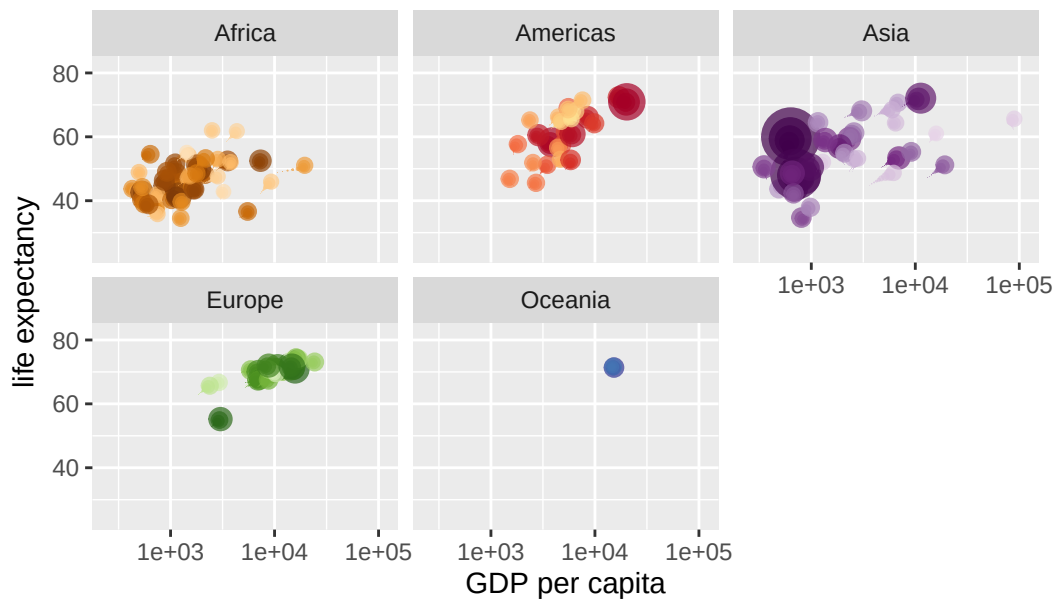
Year: 1968



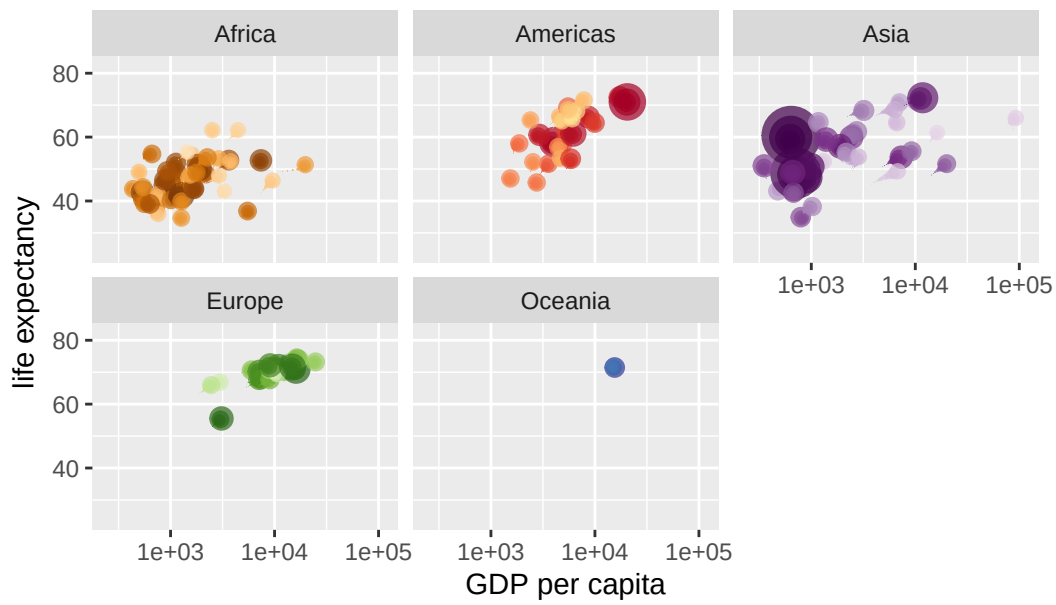
Year: 1968



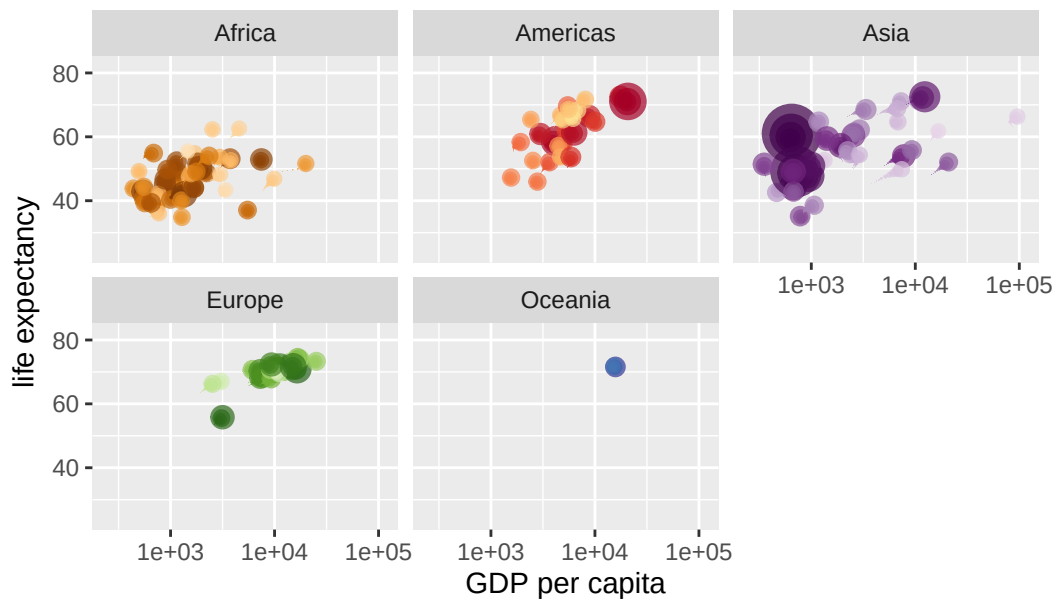
Year: 1969



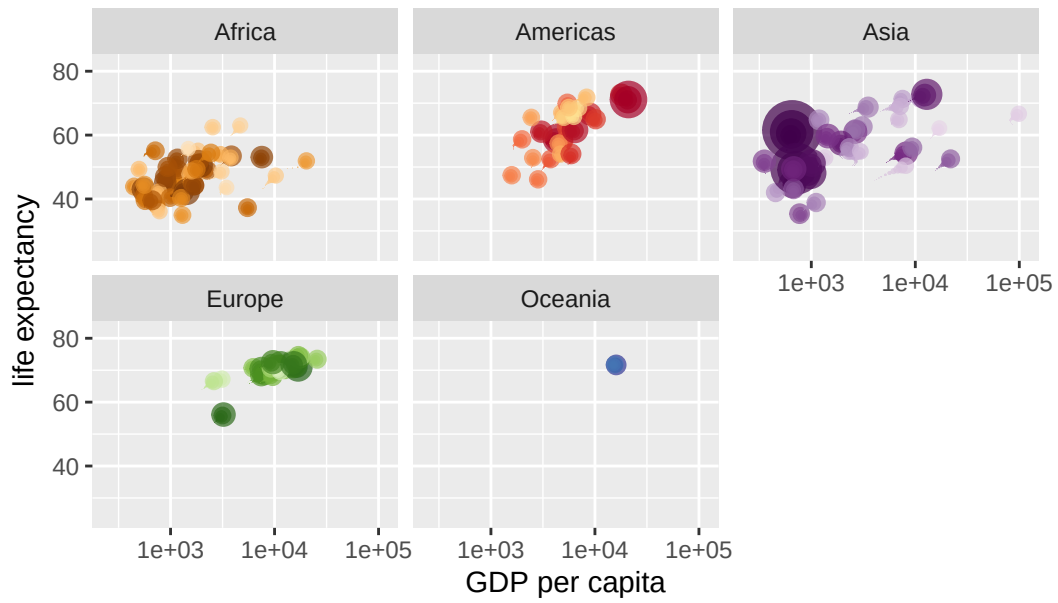
Year: 1969



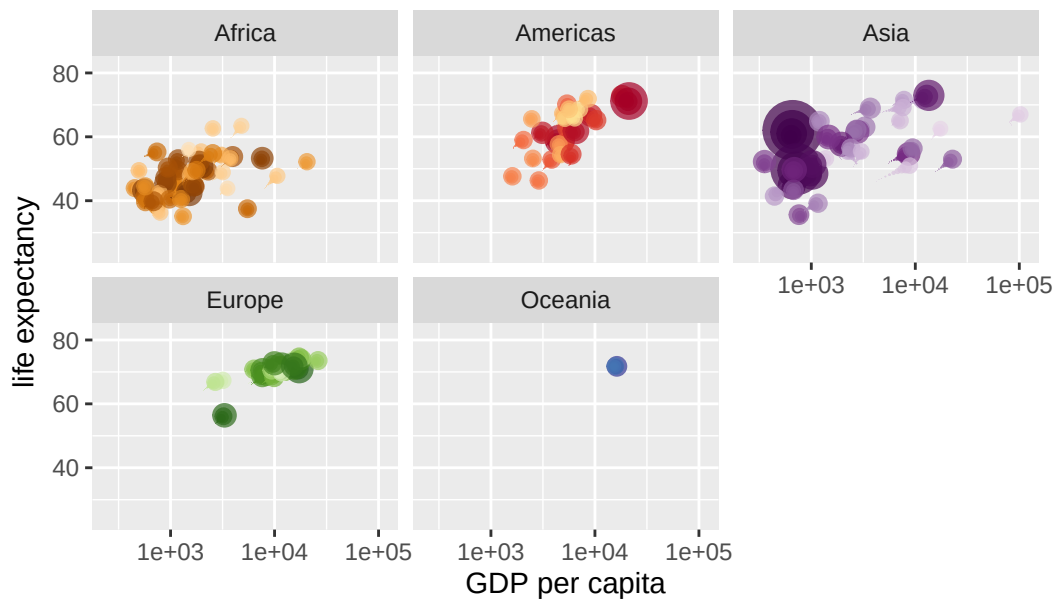
Year: 1970



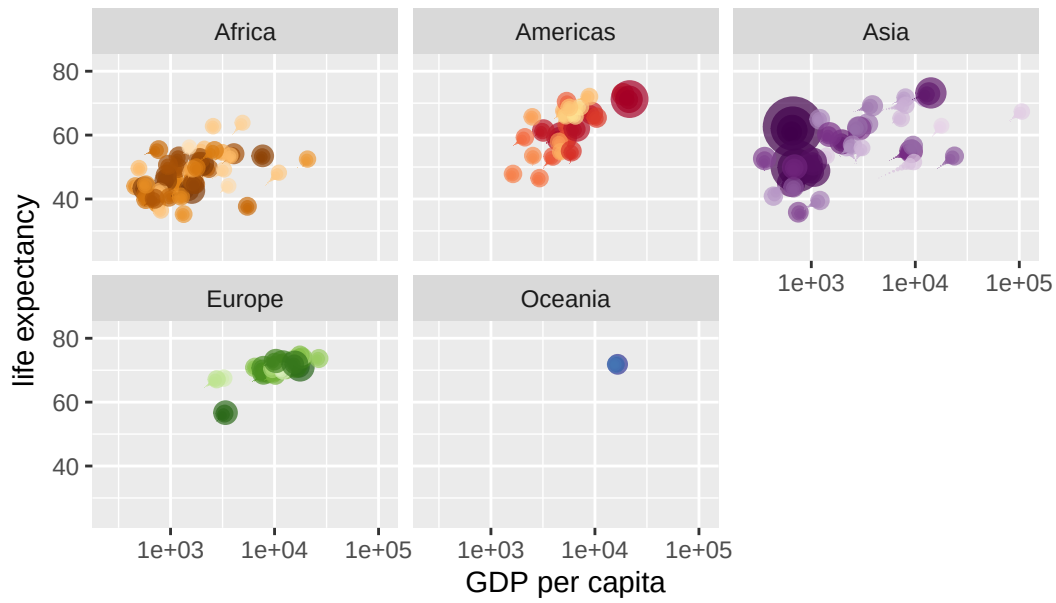
Year: 1970



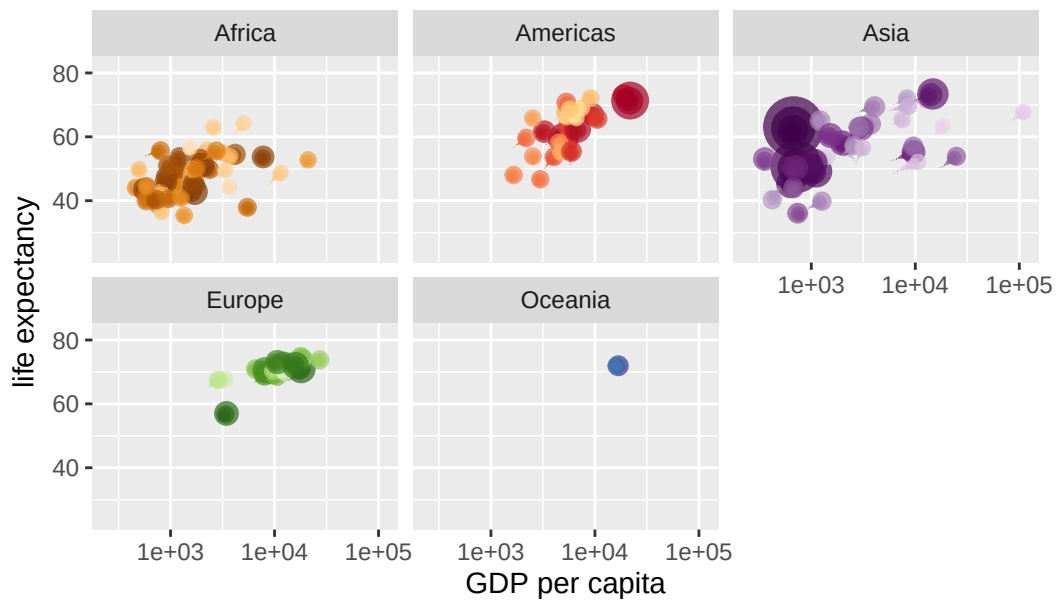
Year: 1971



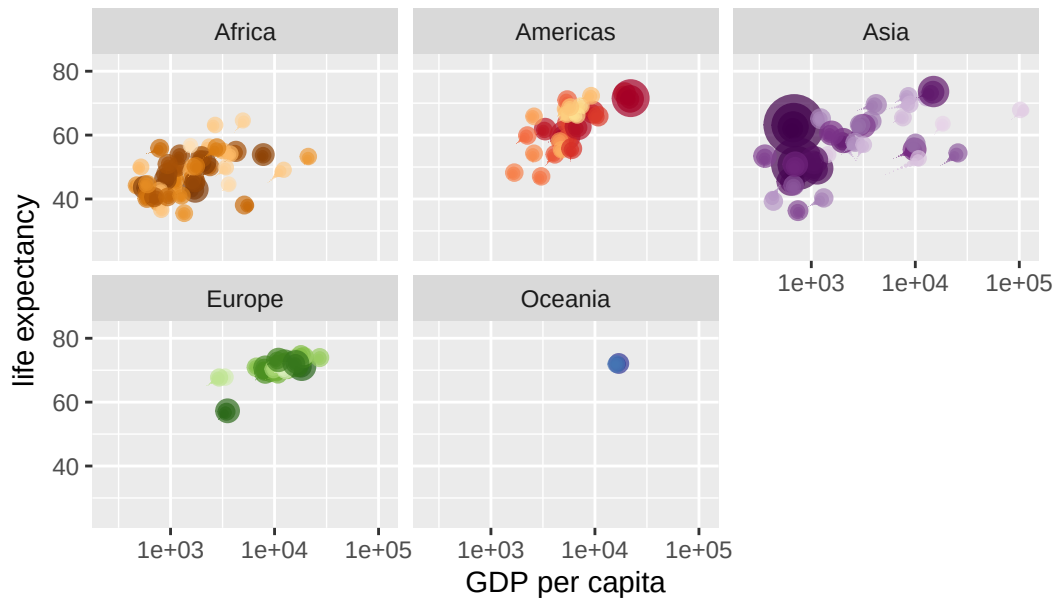
Year: 1971



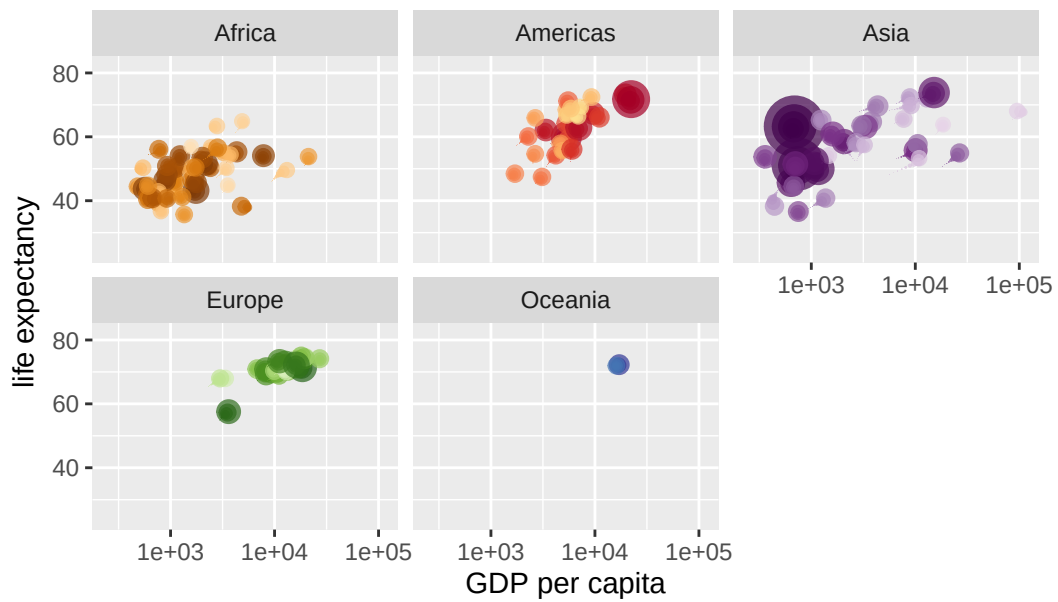
Year: 1972



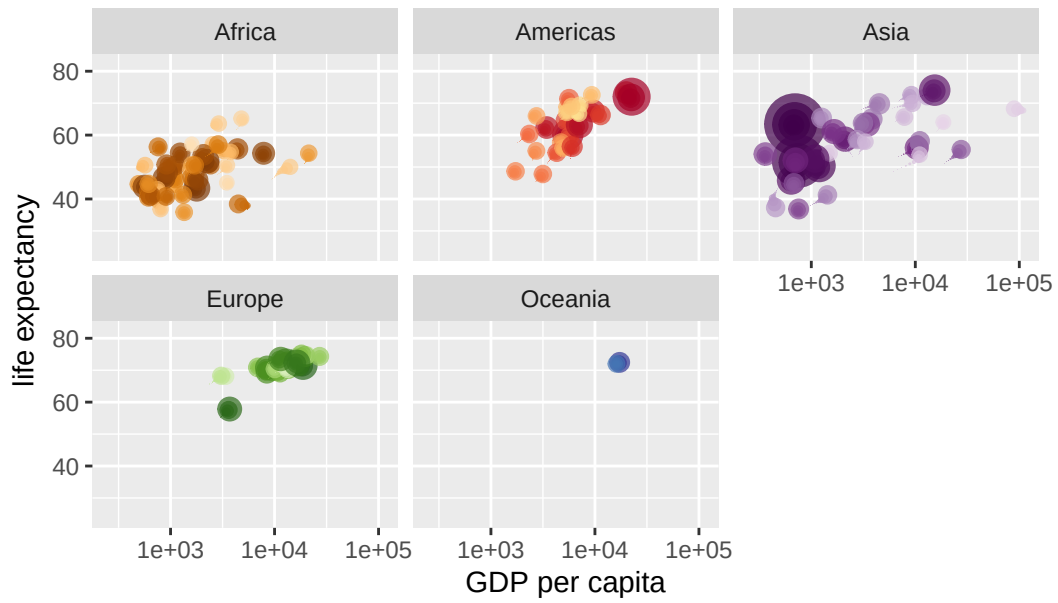
Year: 1973



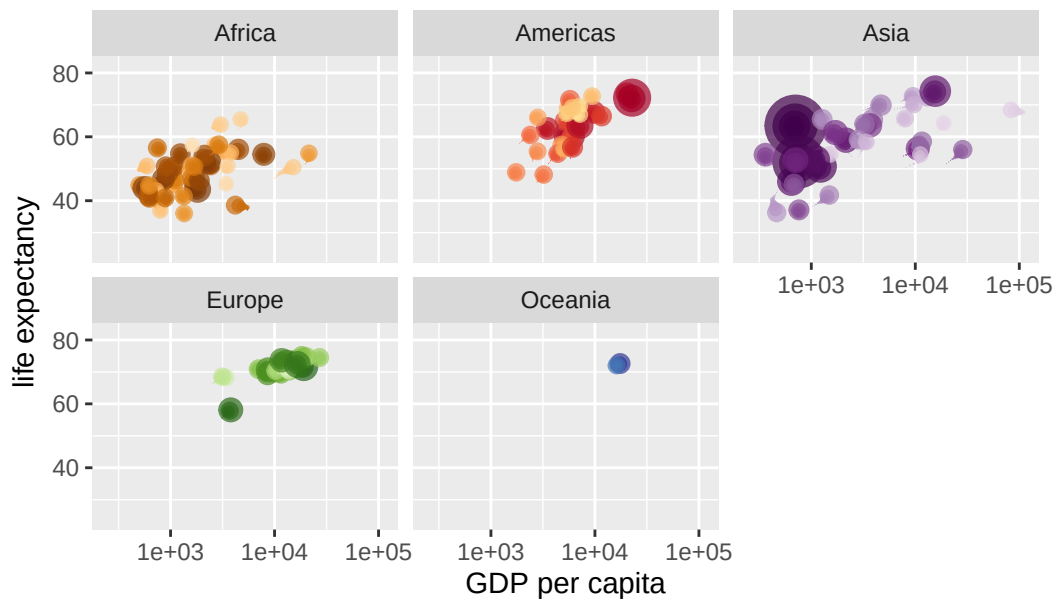
Year: 1973



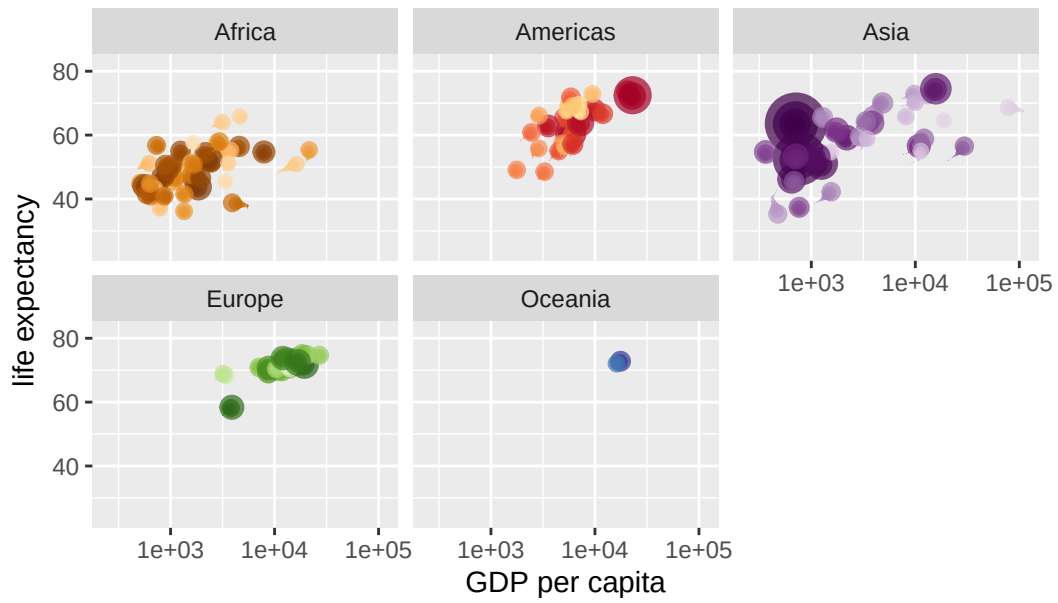
Year: 1974



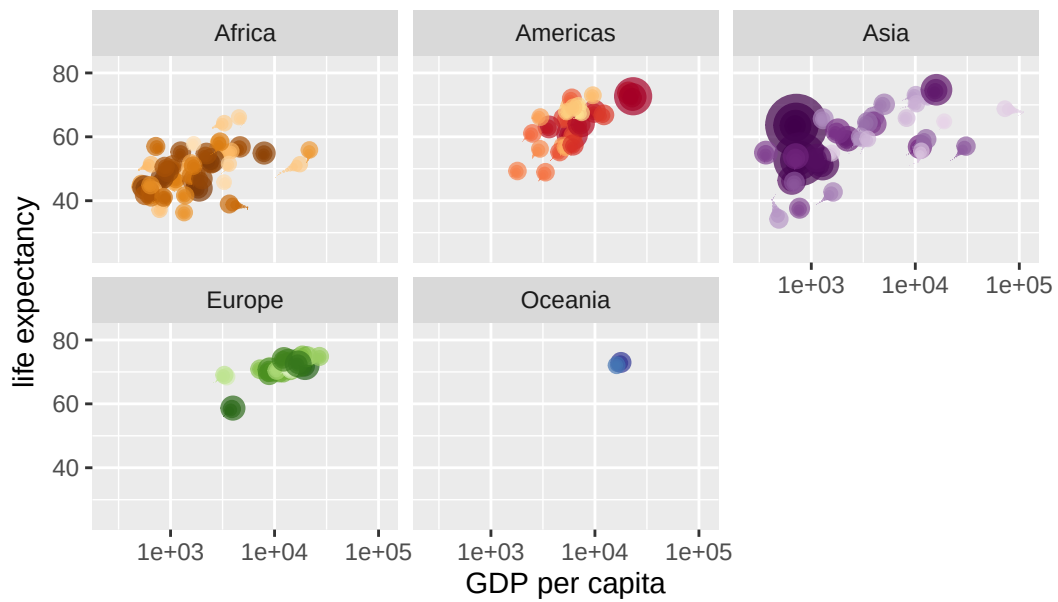
Year: 1974



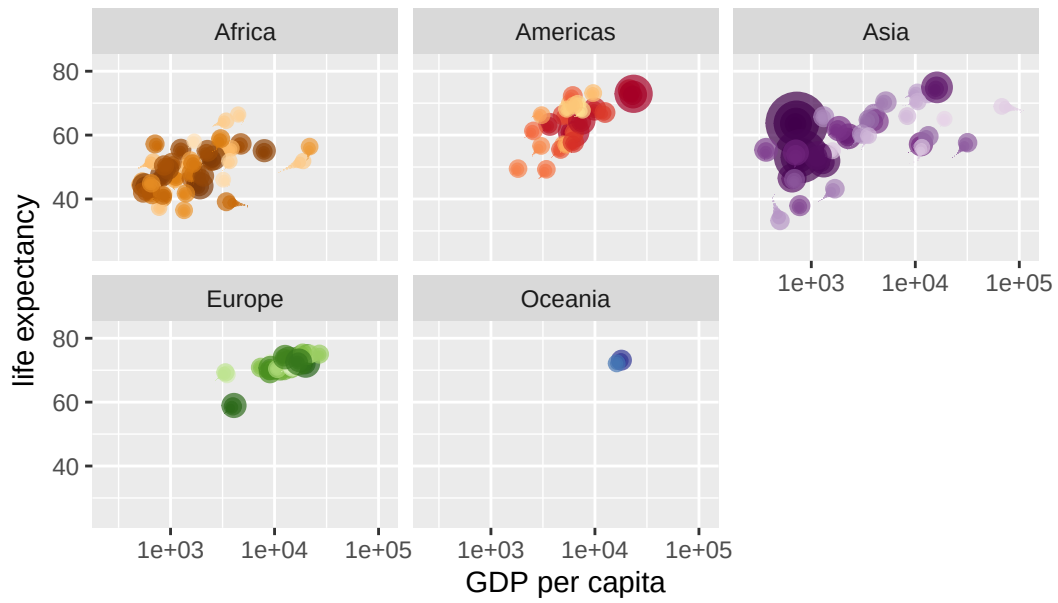
Year: 1975



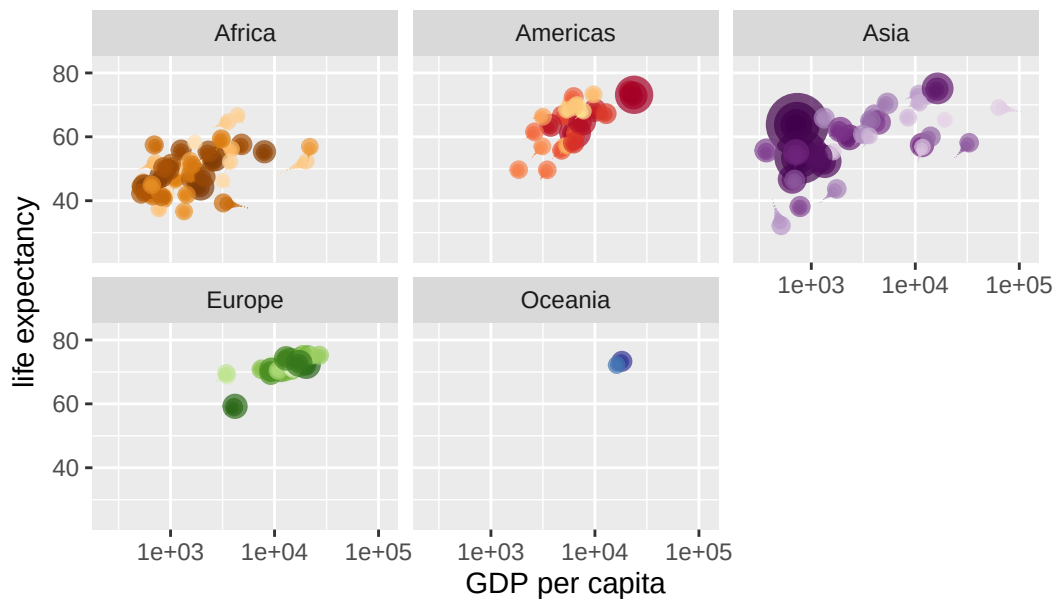
Year: 1975



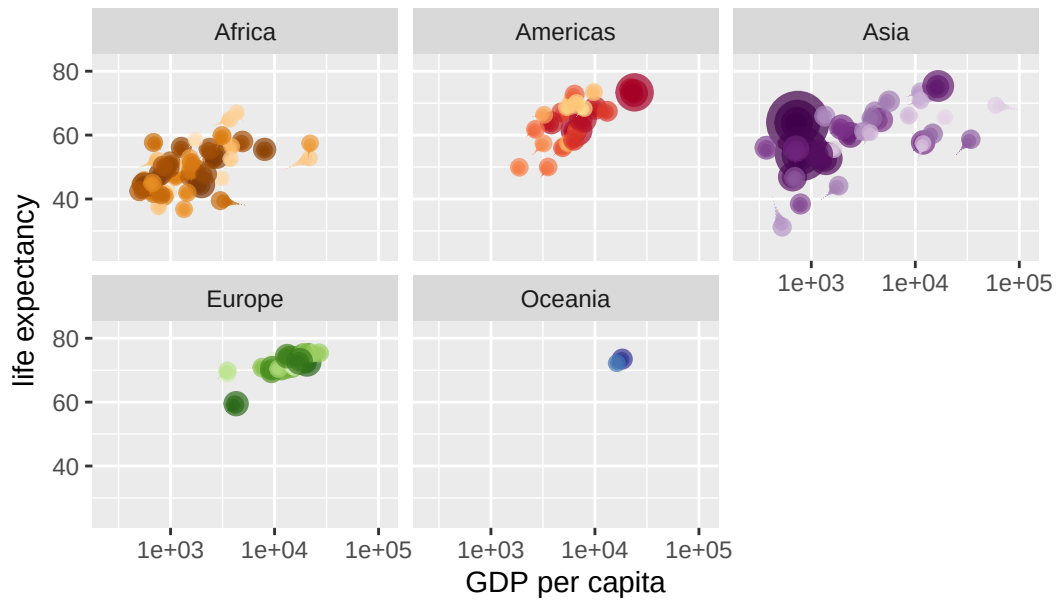
Year: 1976



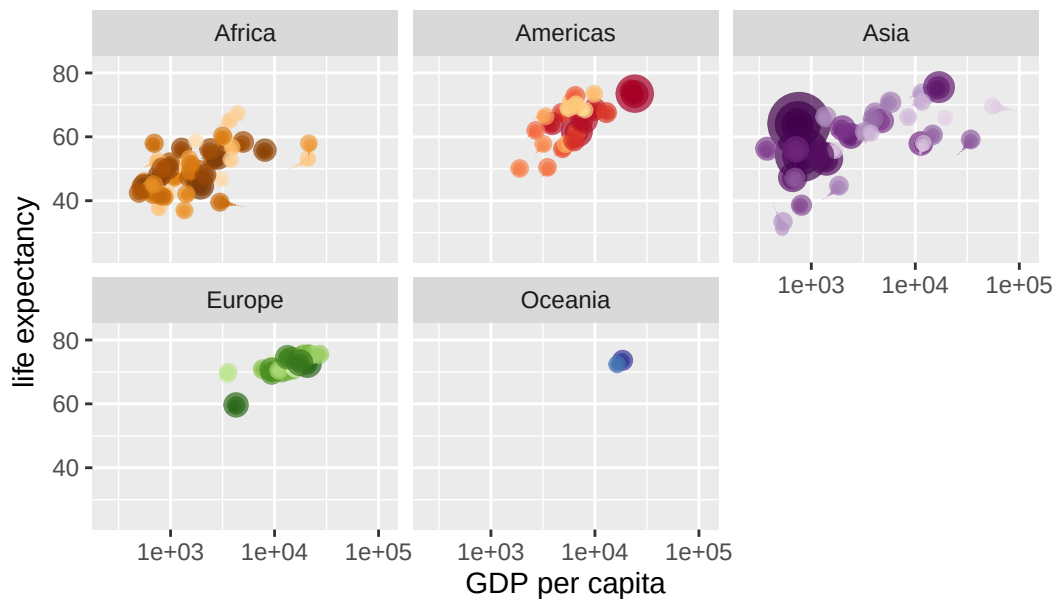
Year: 1976



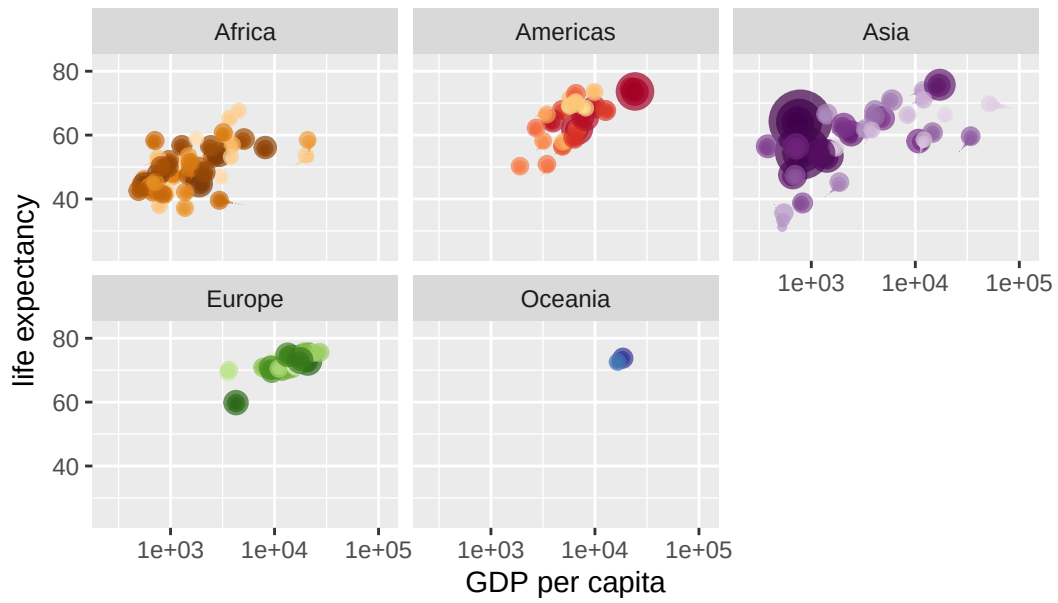
Year: 1977



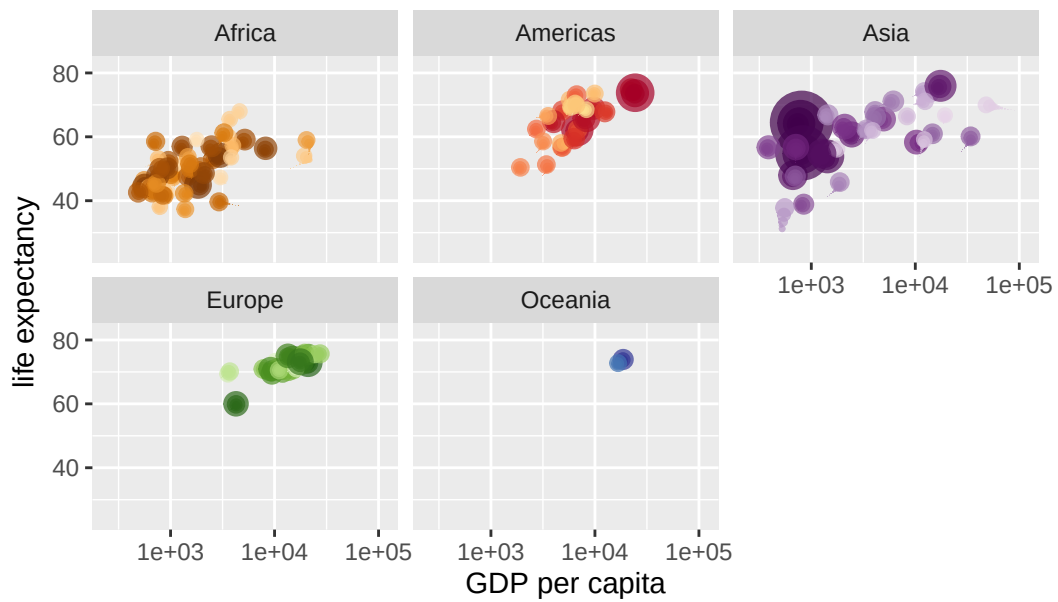
Year: 1978



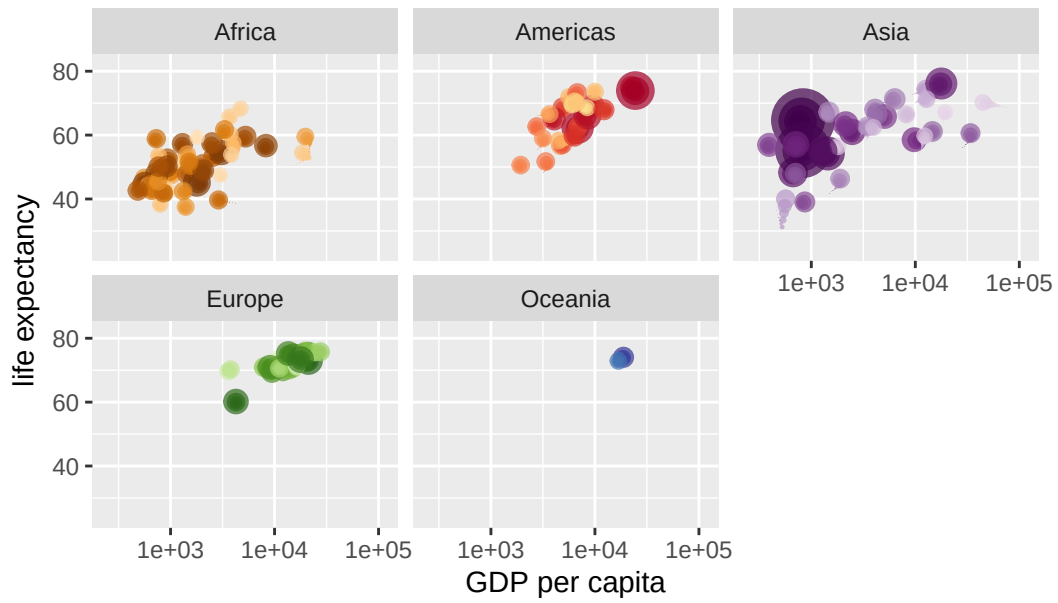
Year: 1978



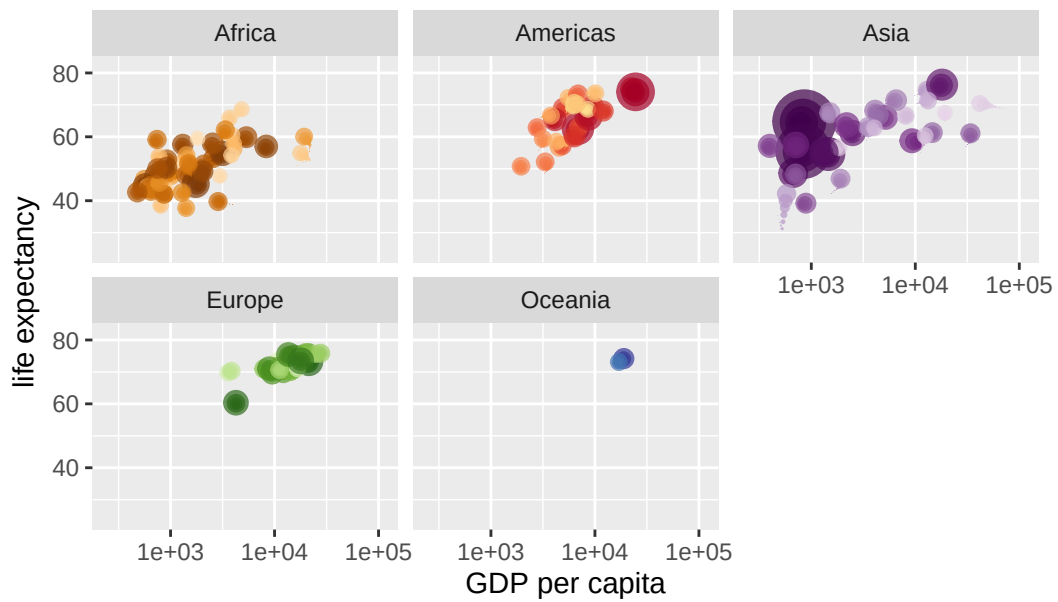
Year: 1979



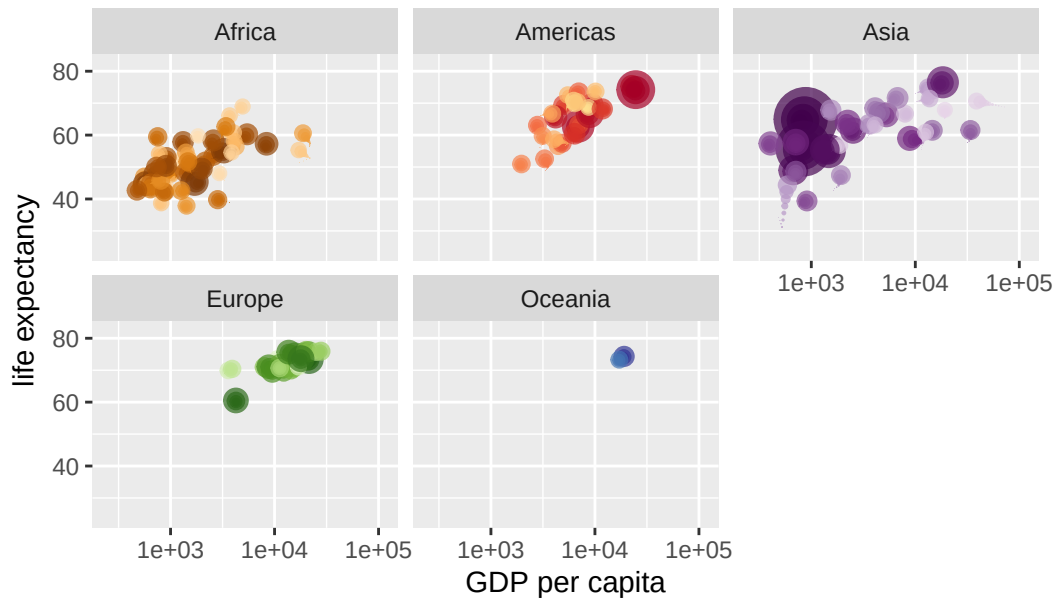
Year: 1979



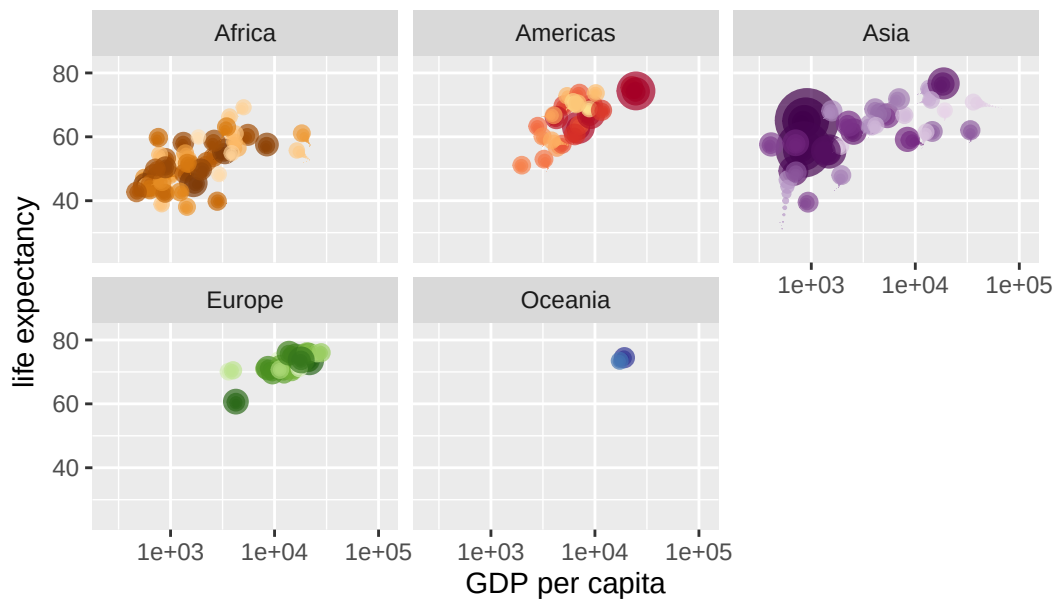
Year: 1980



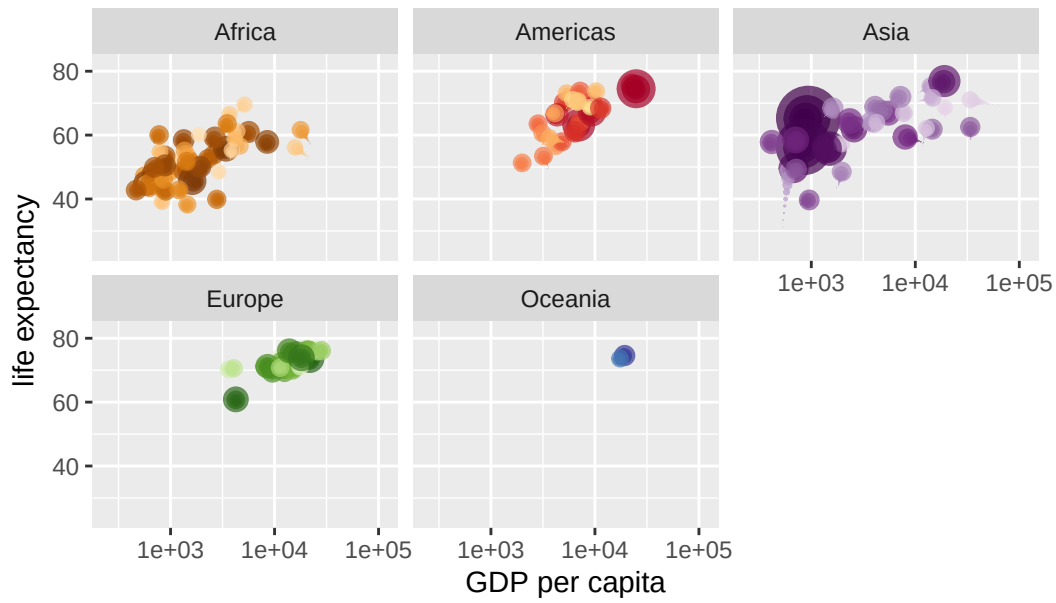
Year: 1980



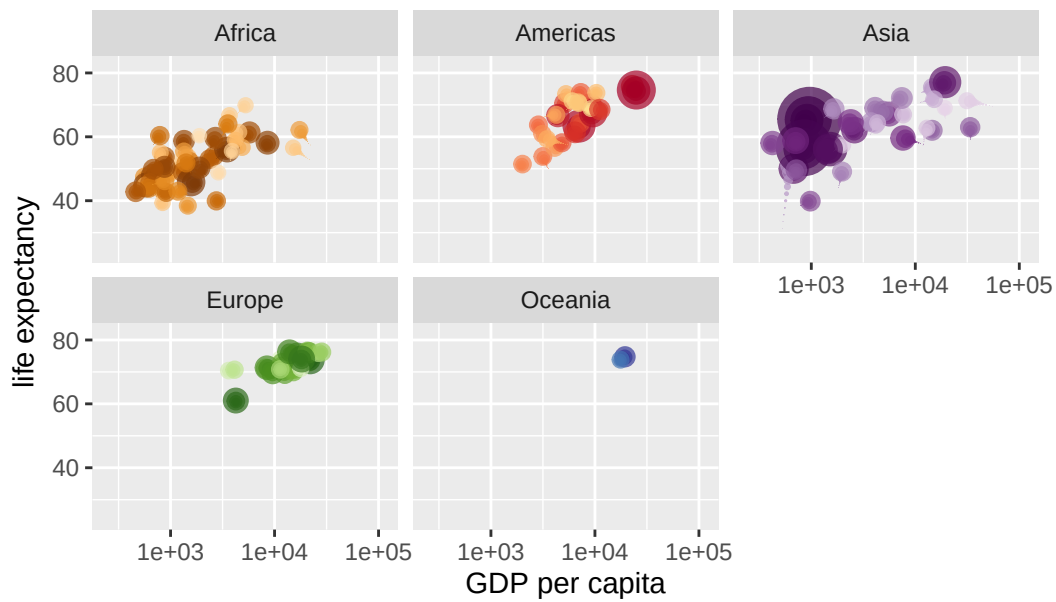
Year: 1981



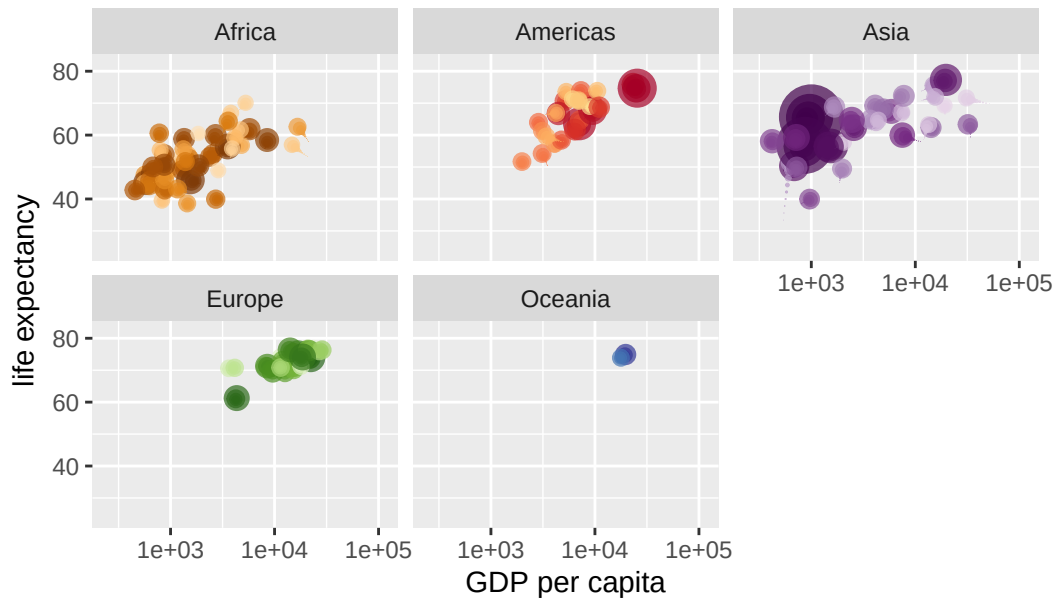
Year: 1981



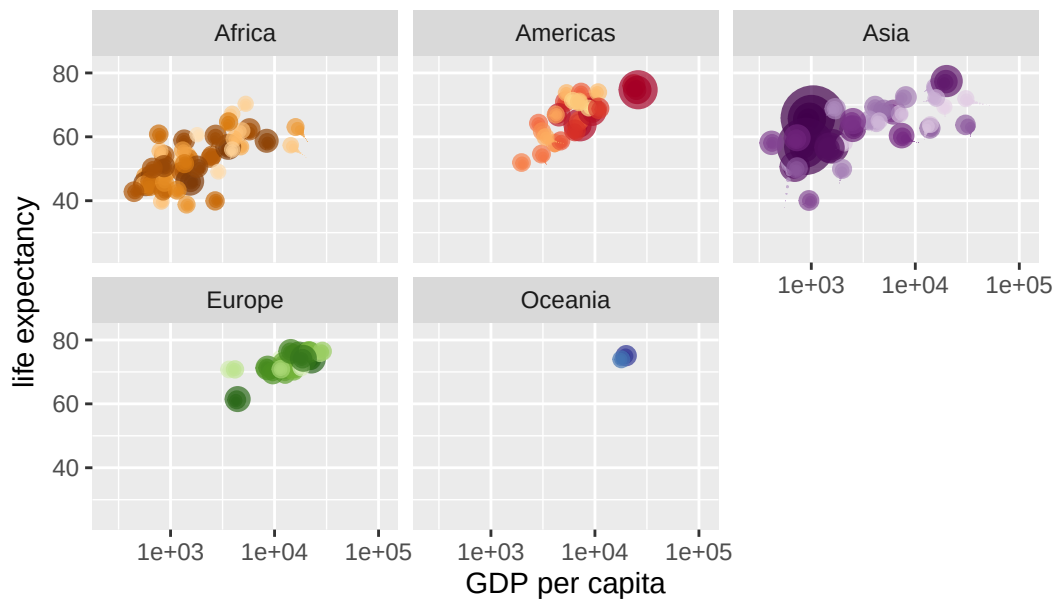
Year: 1982



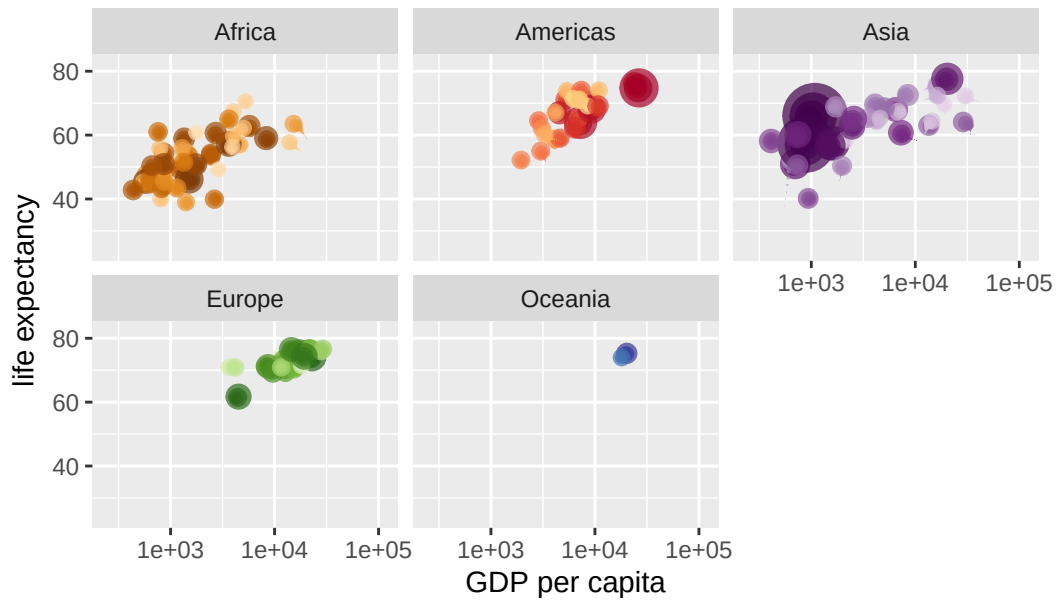
Year: 1983



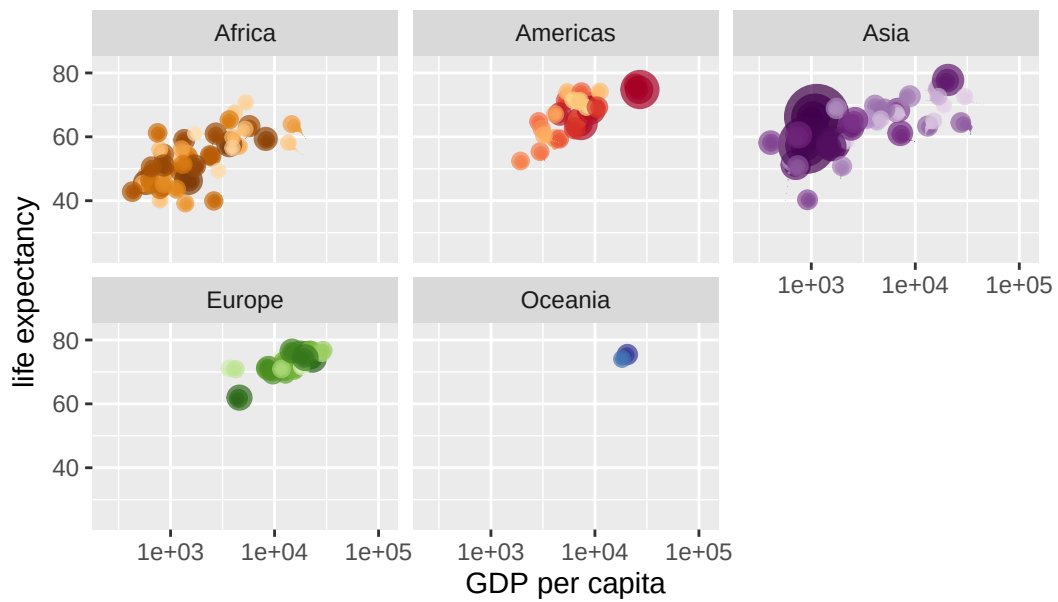
Year: 1983



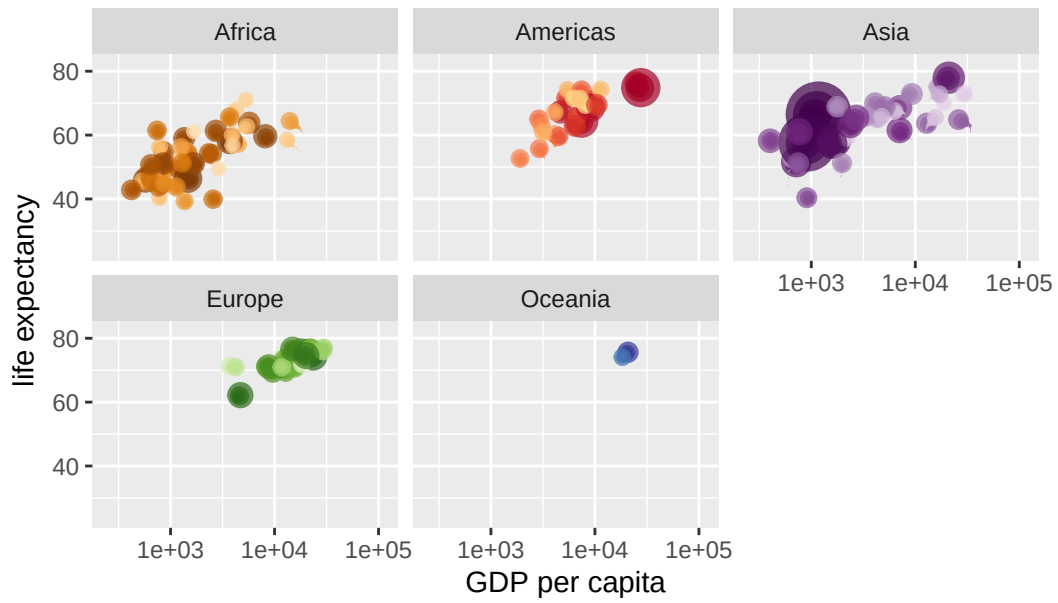
Year: 1984



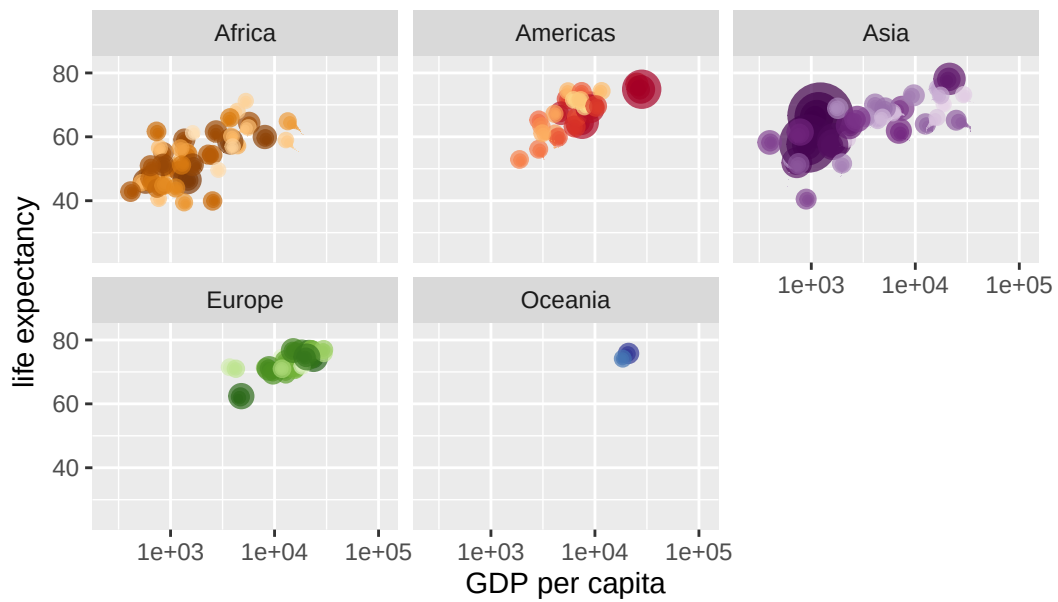
Year: 1984



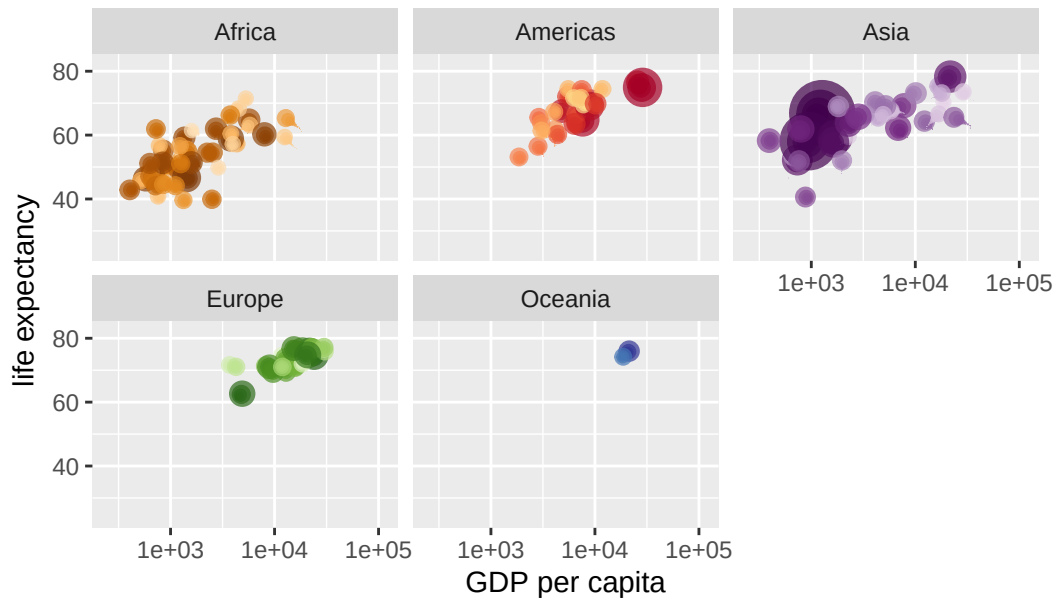
Year: 1985



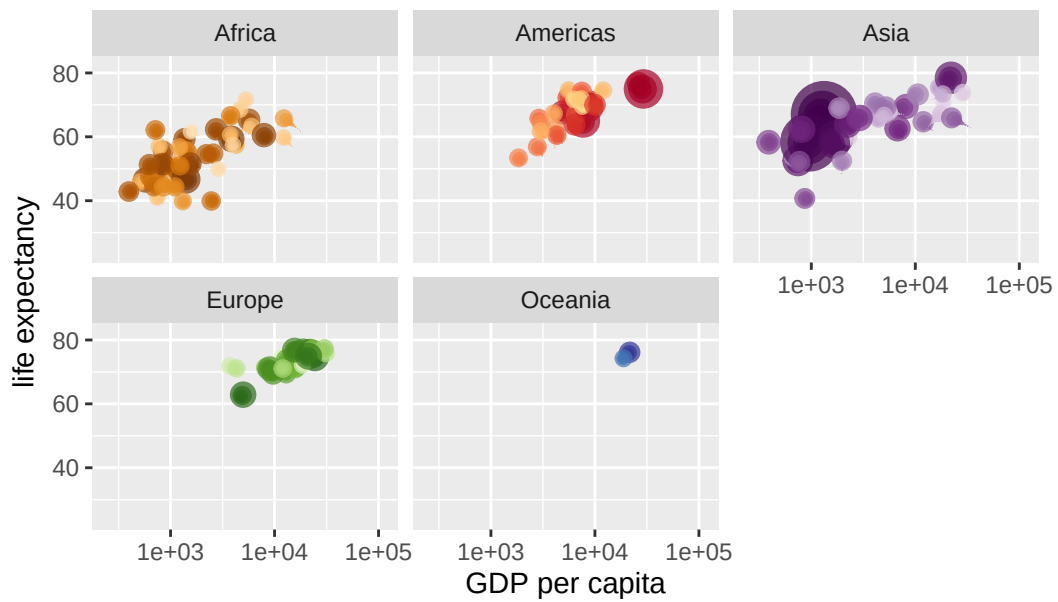
Year: 1985



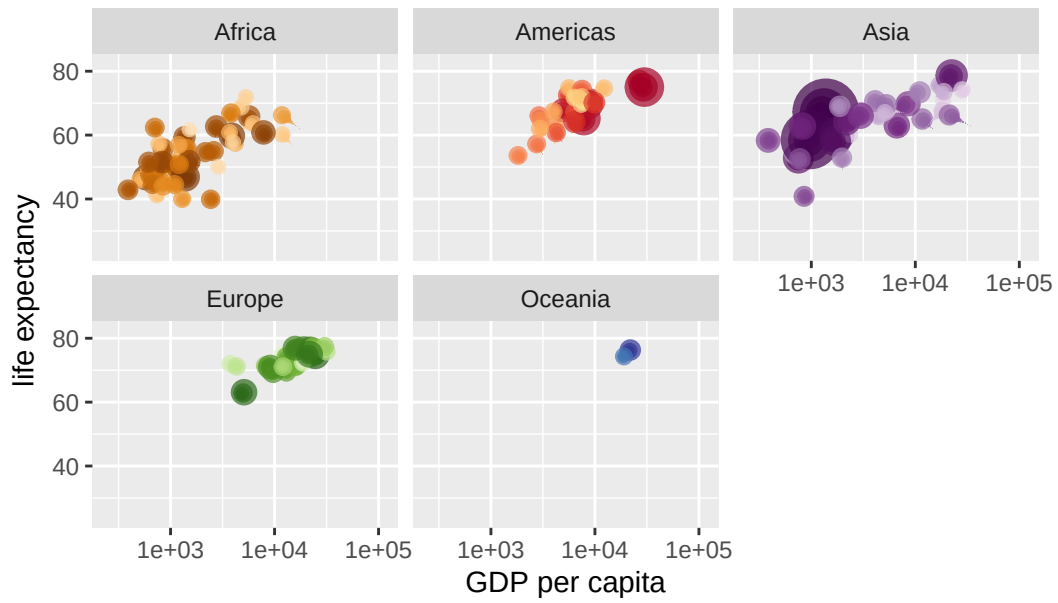
Year: 1986



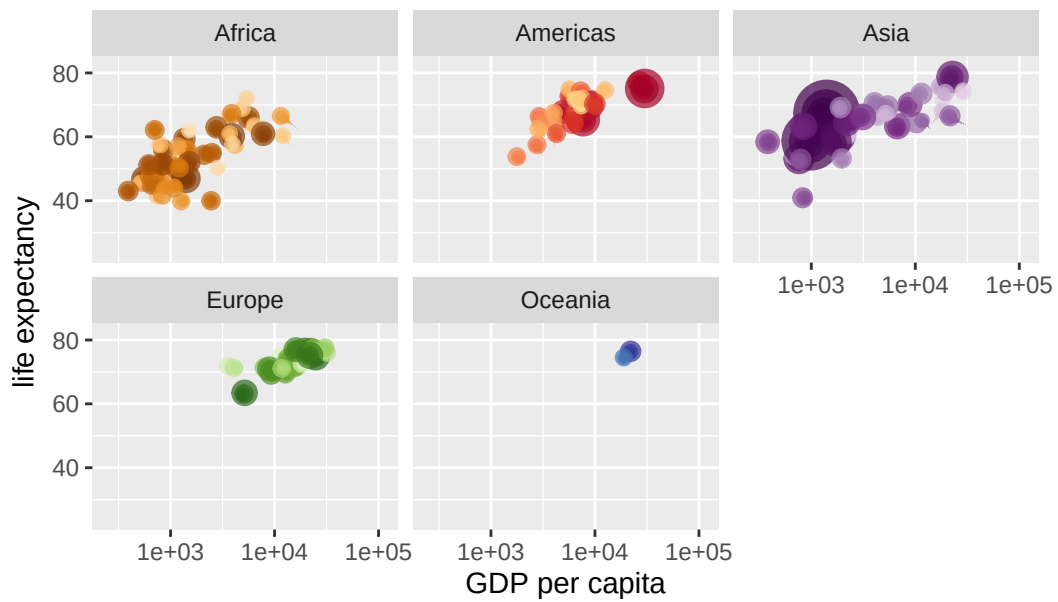
Year: 1986



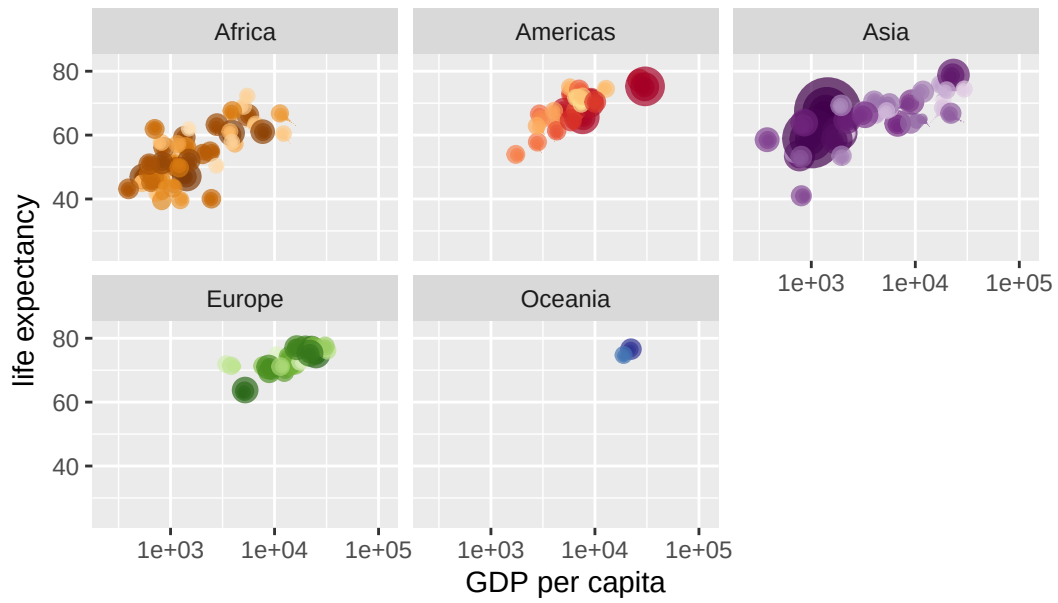
Year: 1987



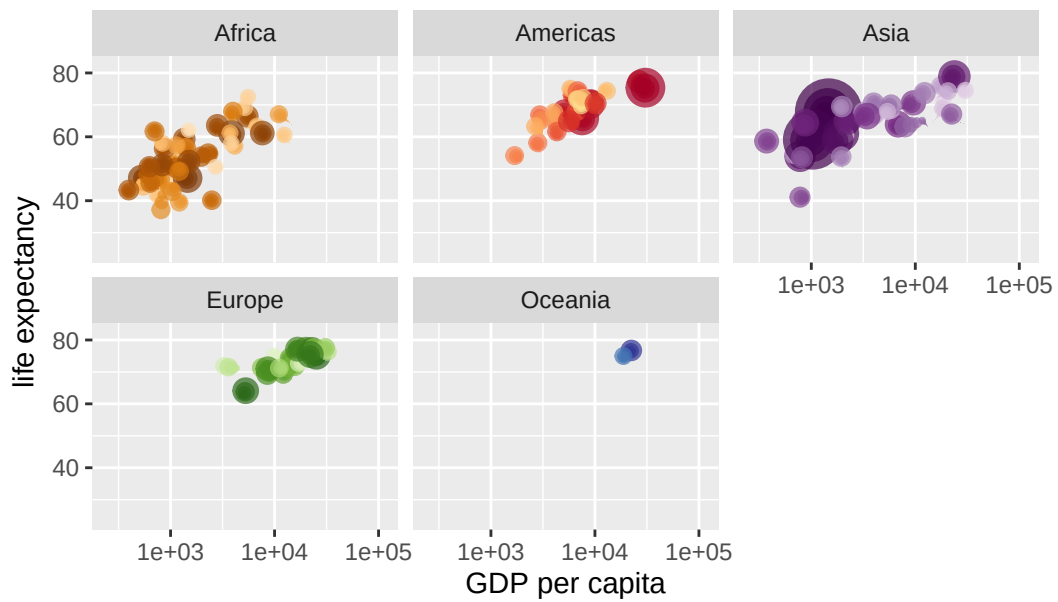
Year: 1988



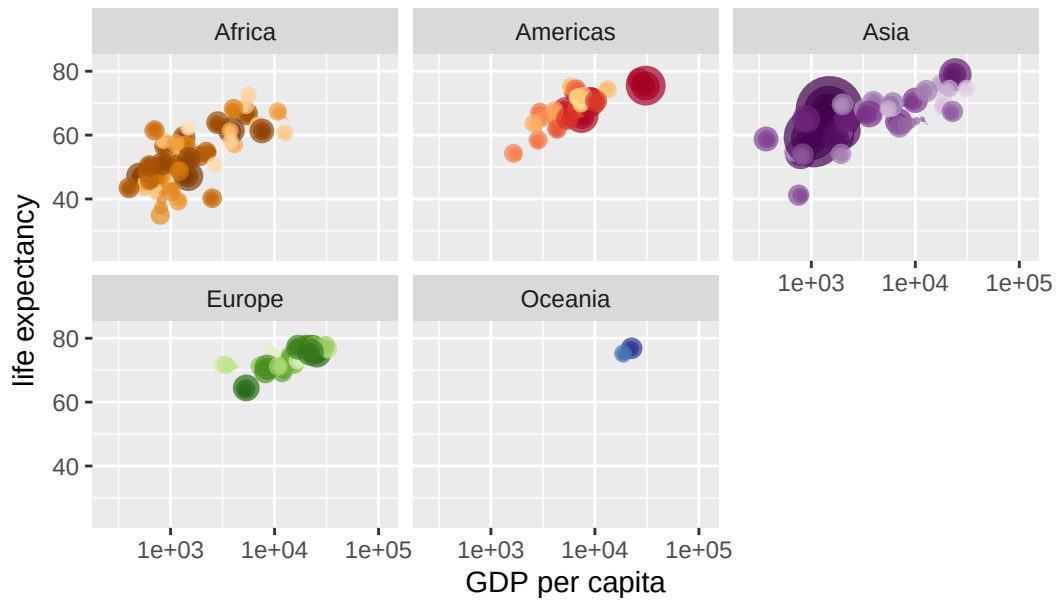
Year: 1988



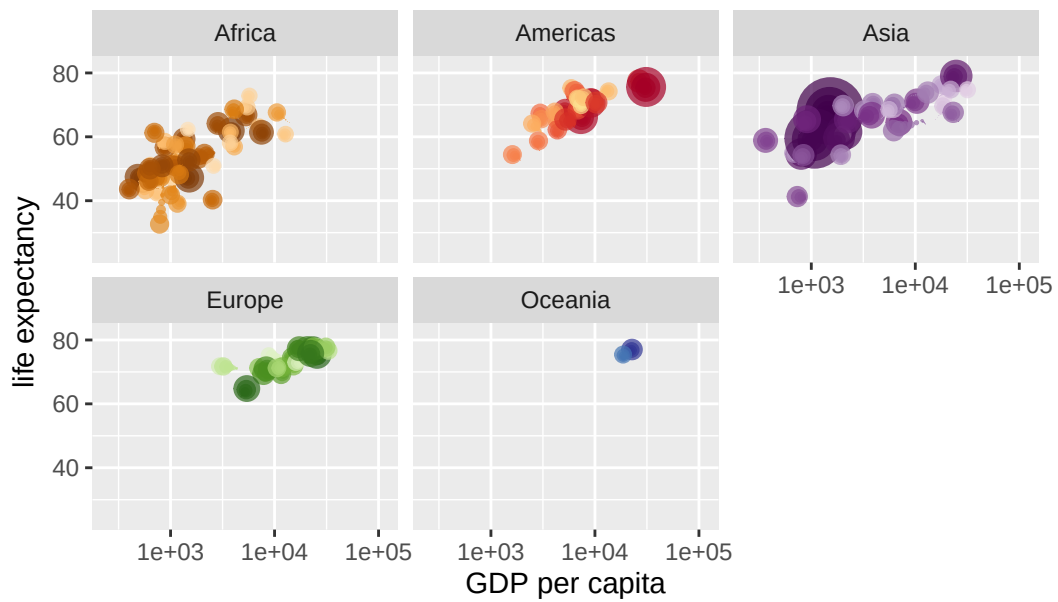
Year: 1989



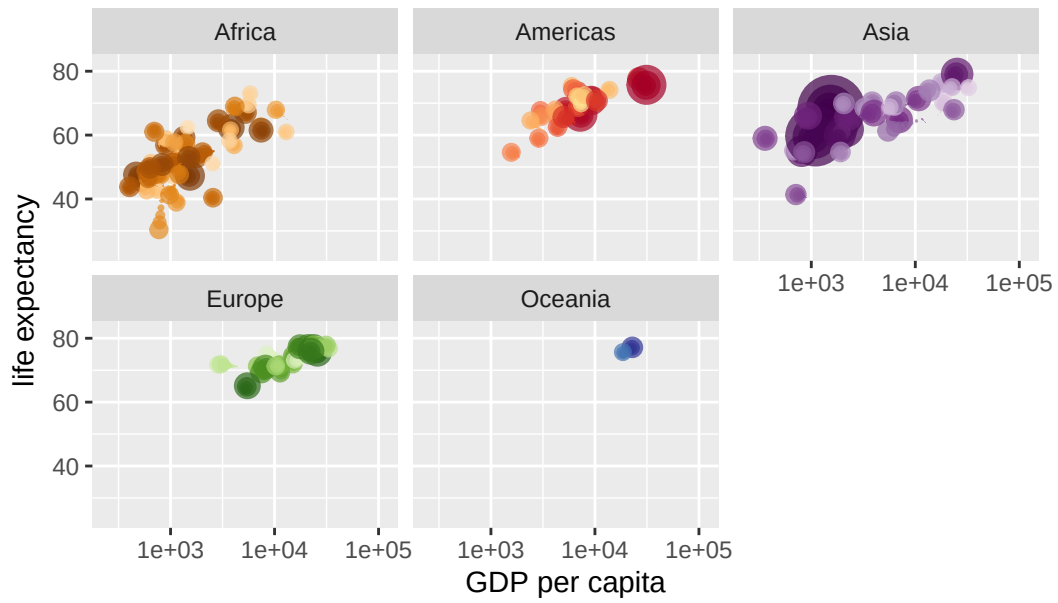
Year: 1989



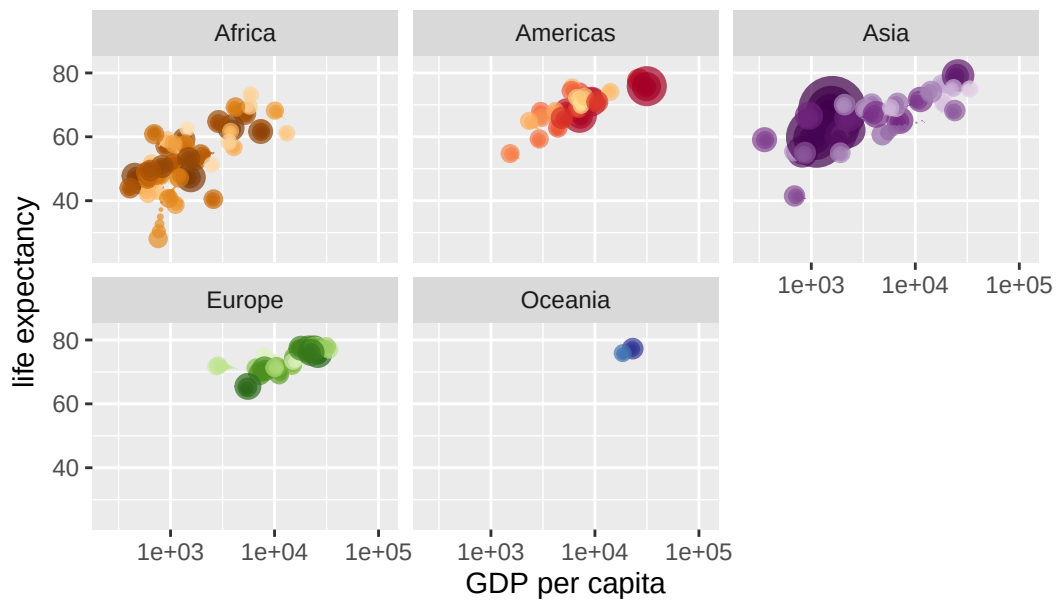
Year: 1990



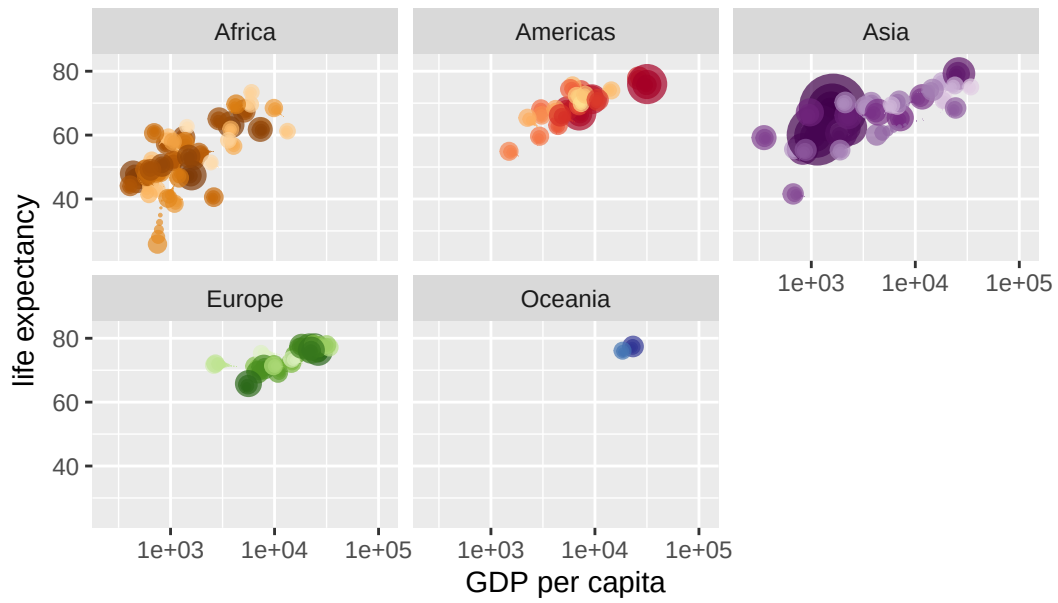
Year: 1990



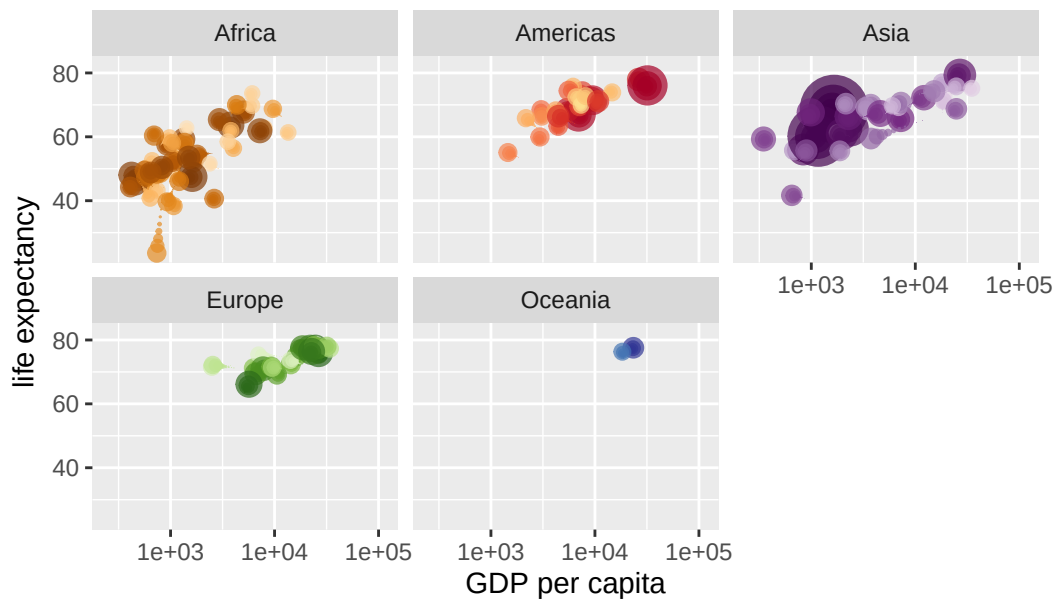
Year: 1991



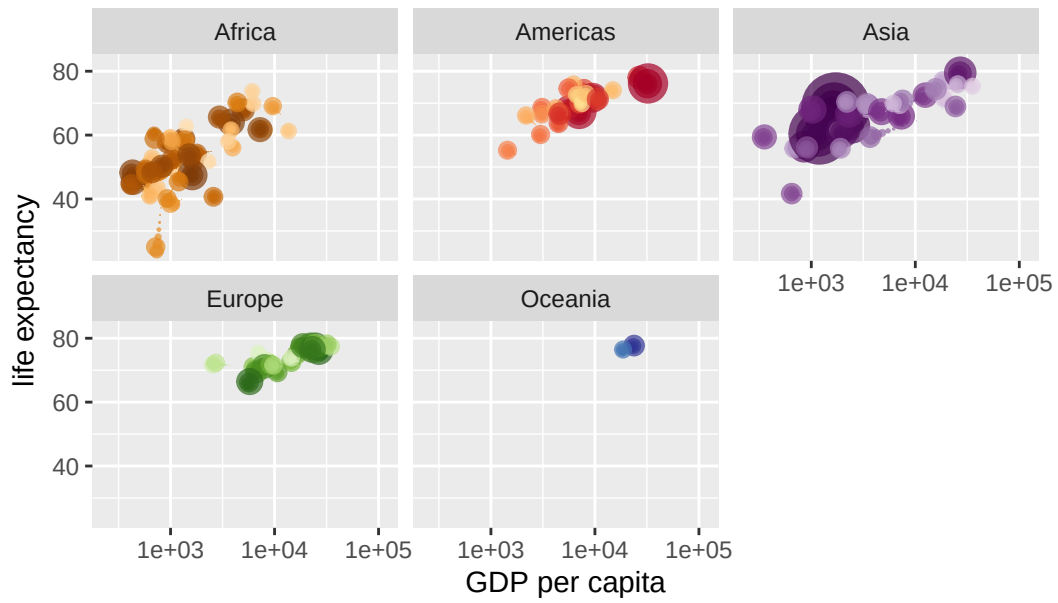
Year: 1991



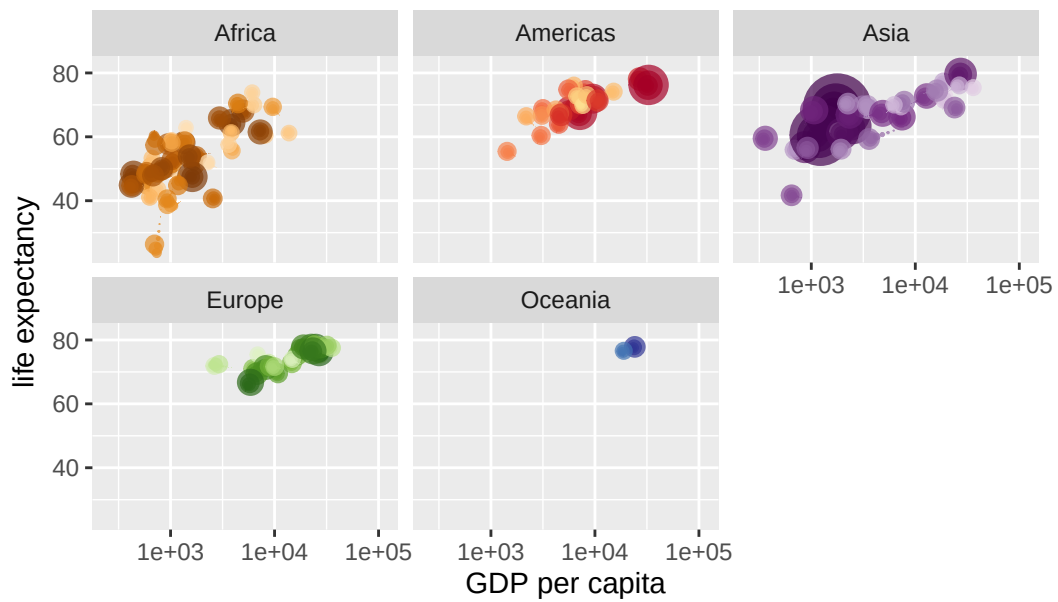
Year: 1992



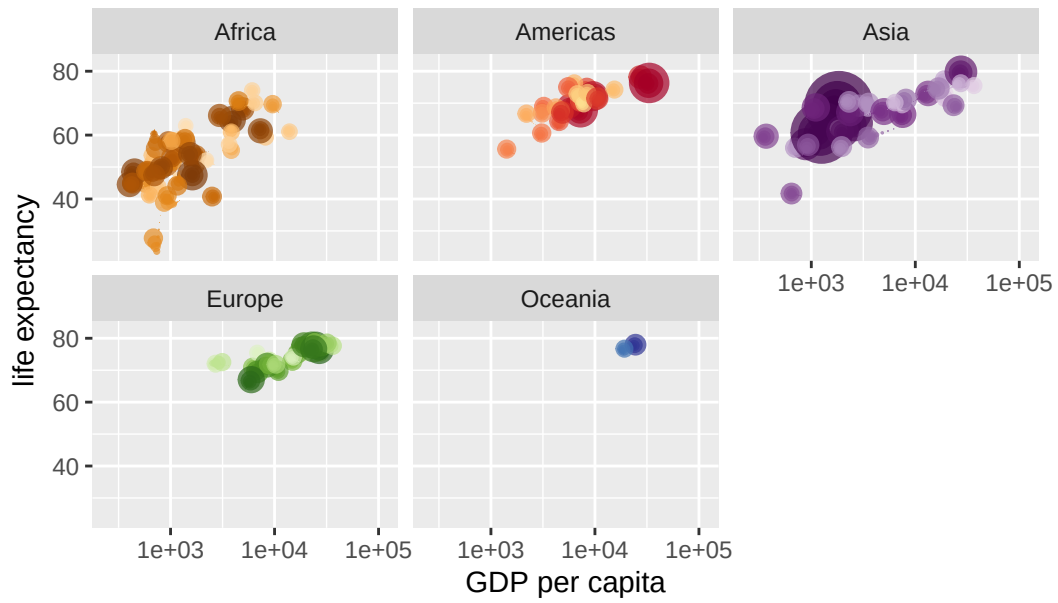
Year: 1993



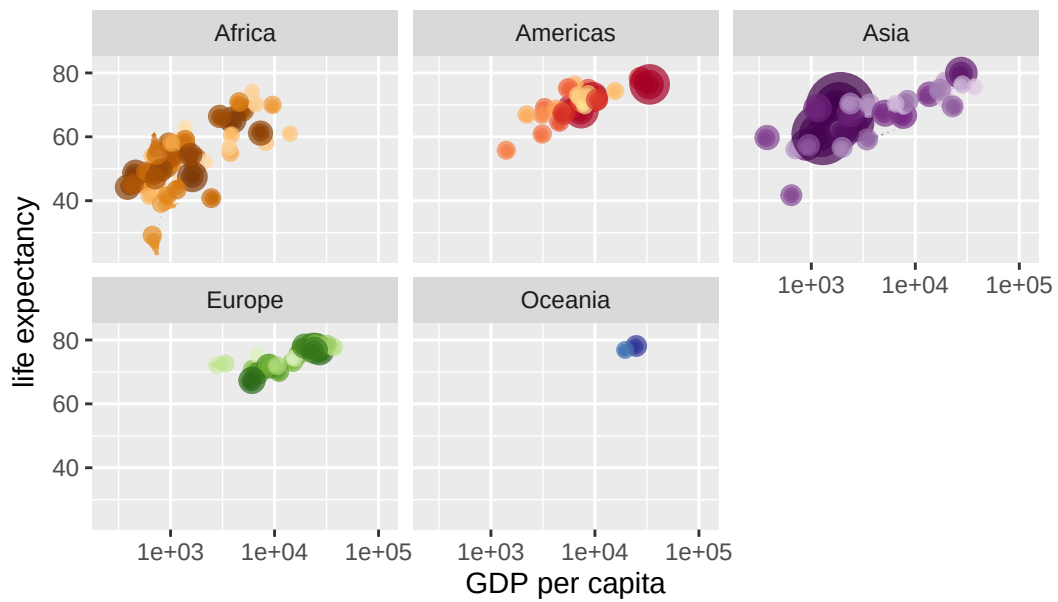
Year: 1993



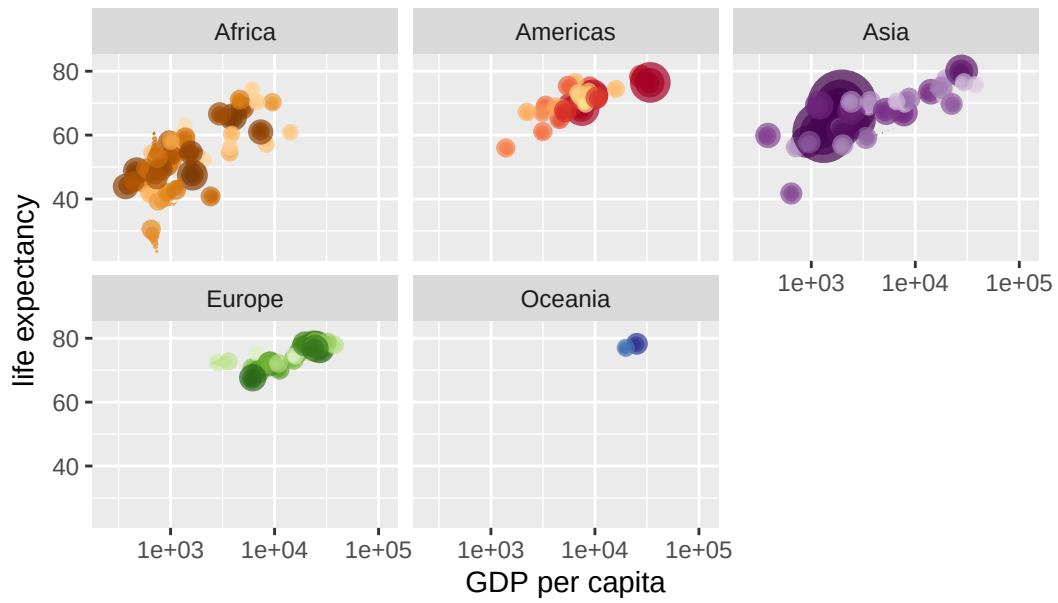
Year: 1994



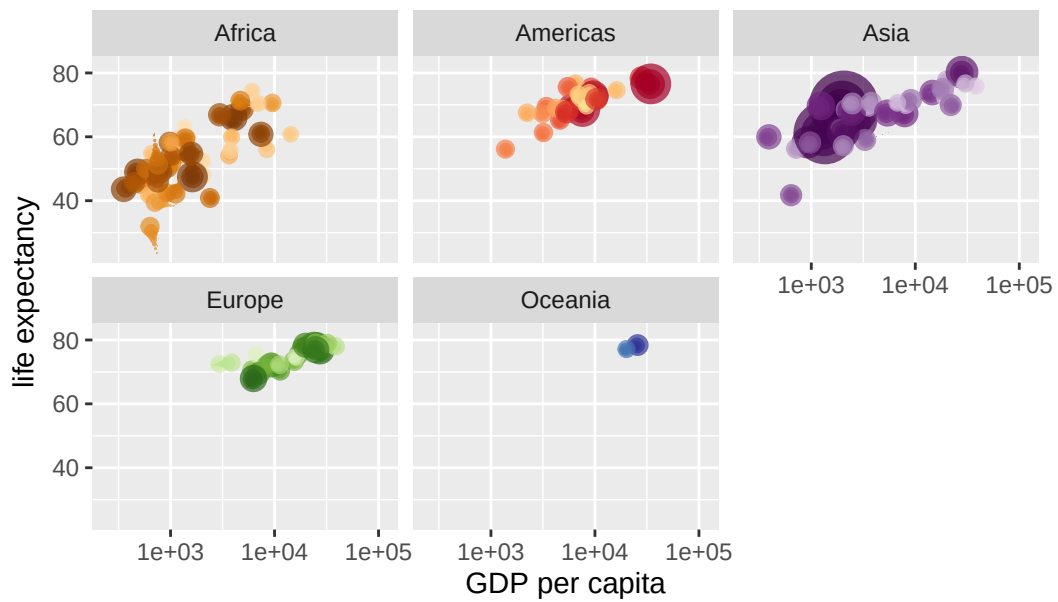
Year: 1994



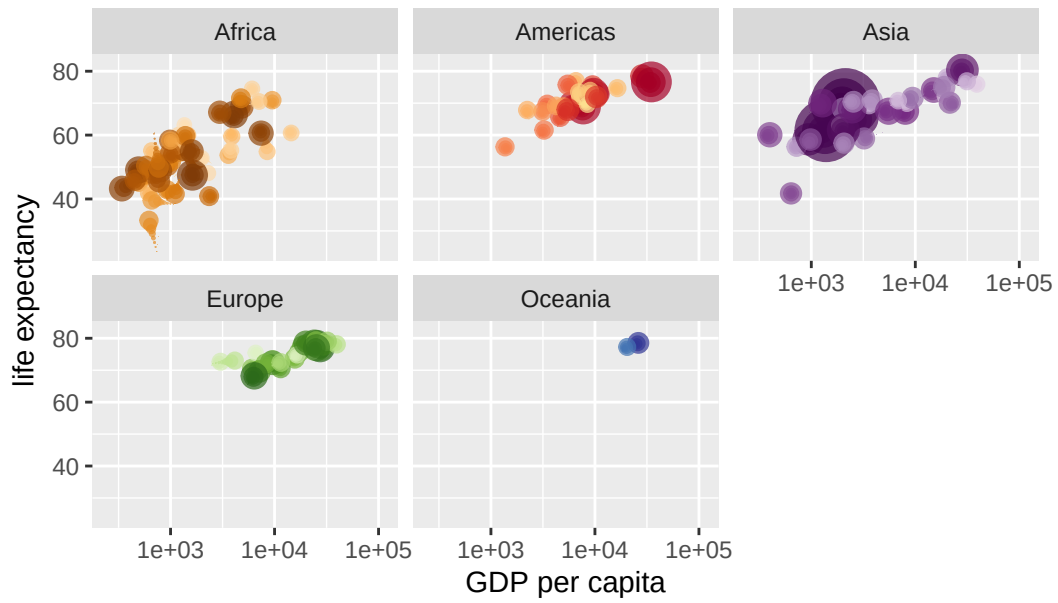
Year: 1995



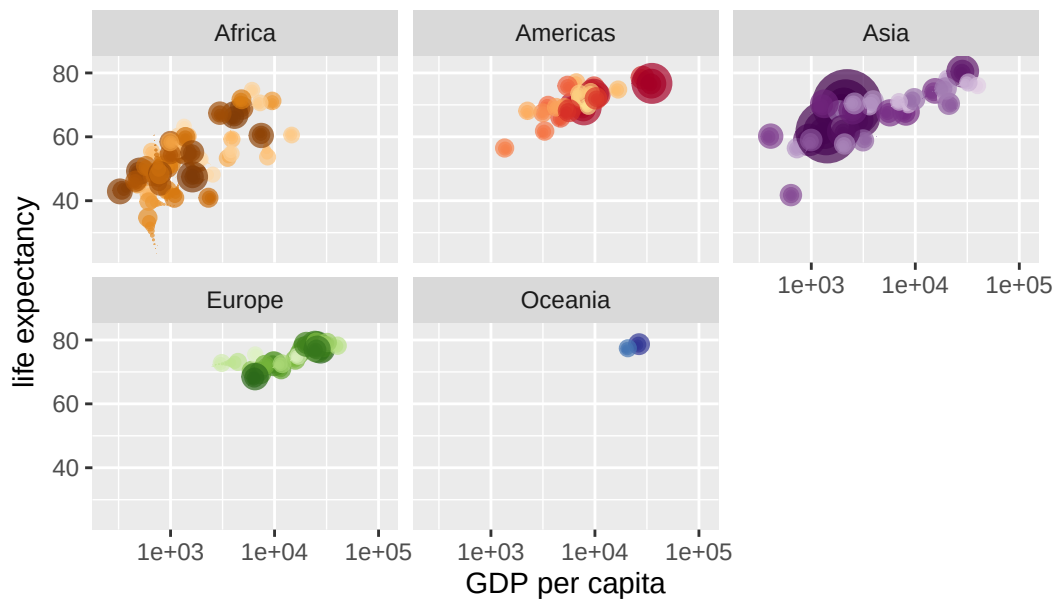
Year: 1995



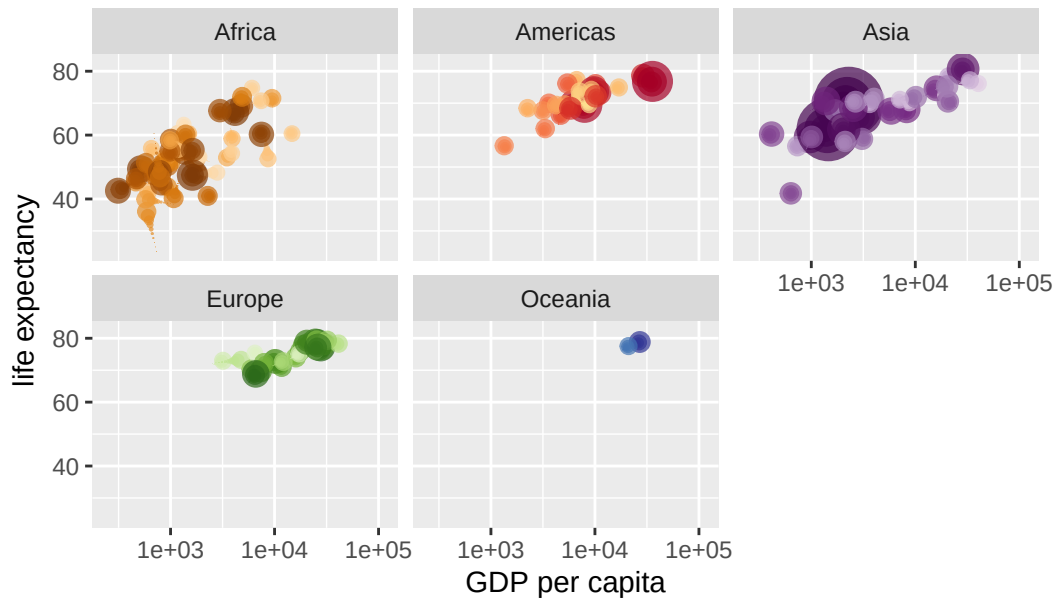
Year: 1996



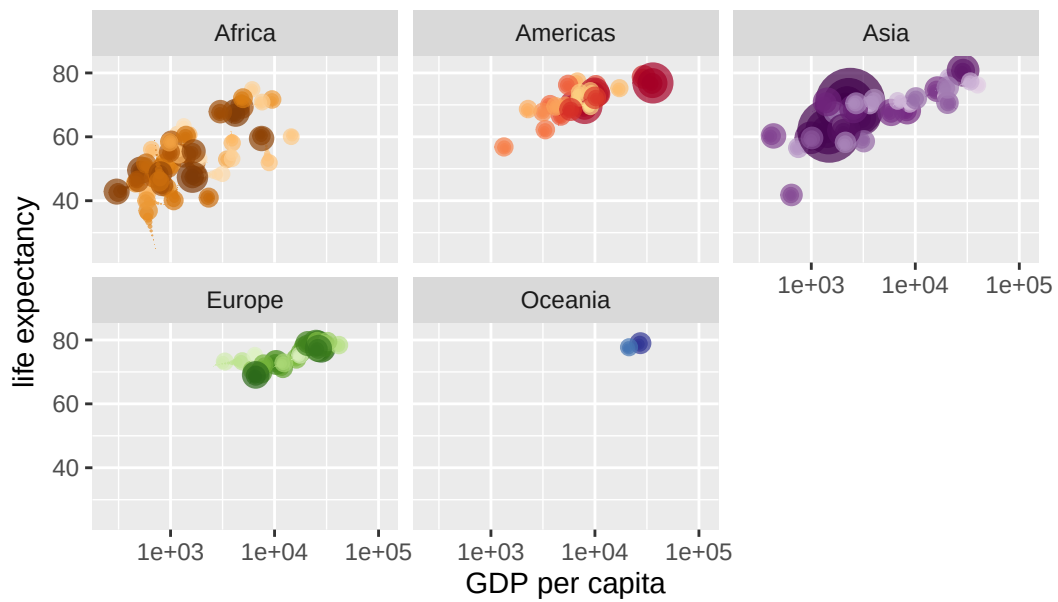
Year: 1996



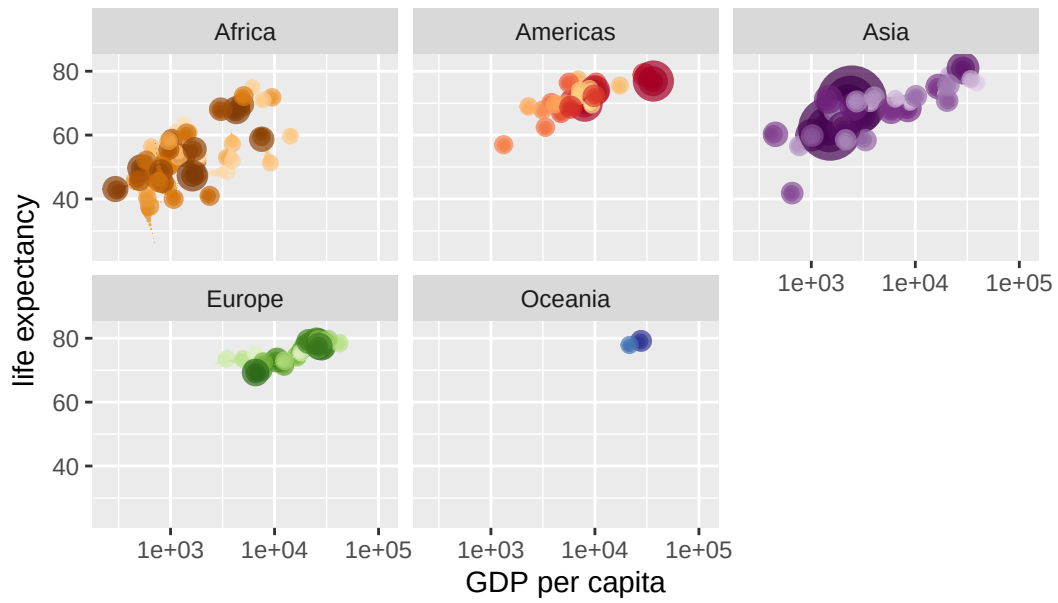
Year: 1997



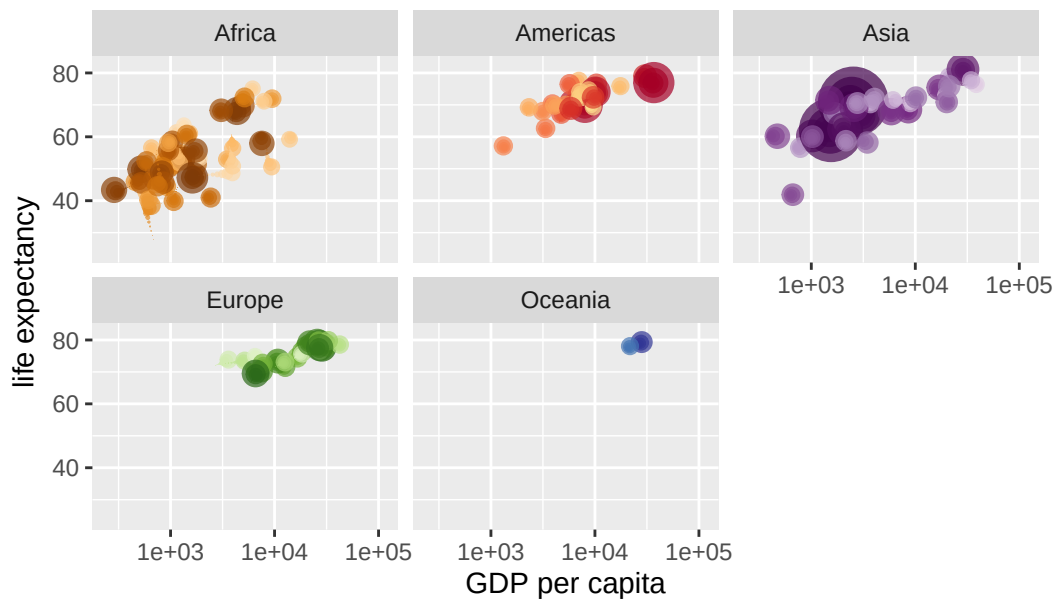
Year: 1998



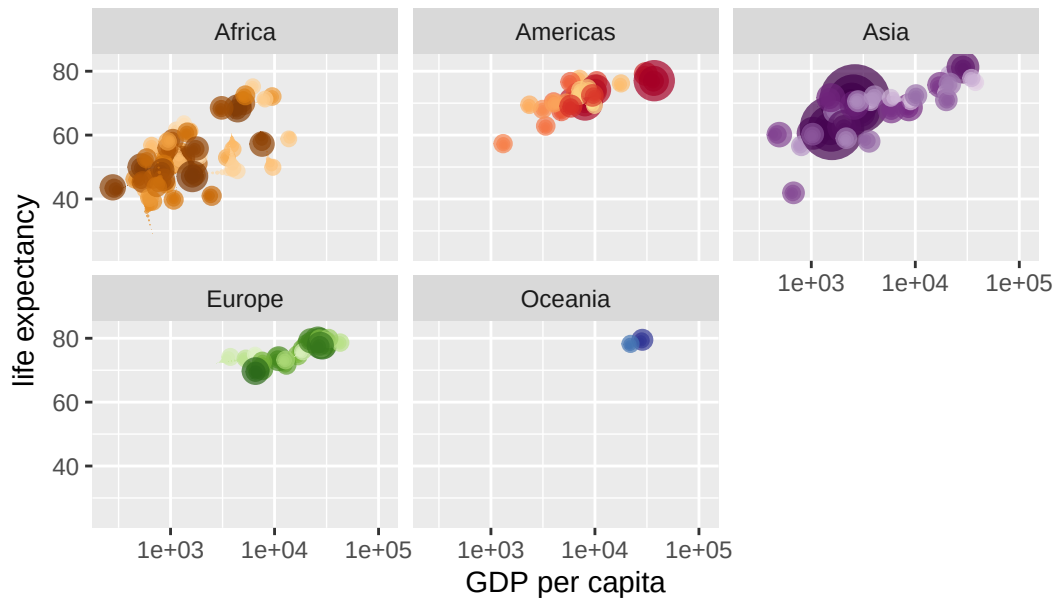
Year: 1998



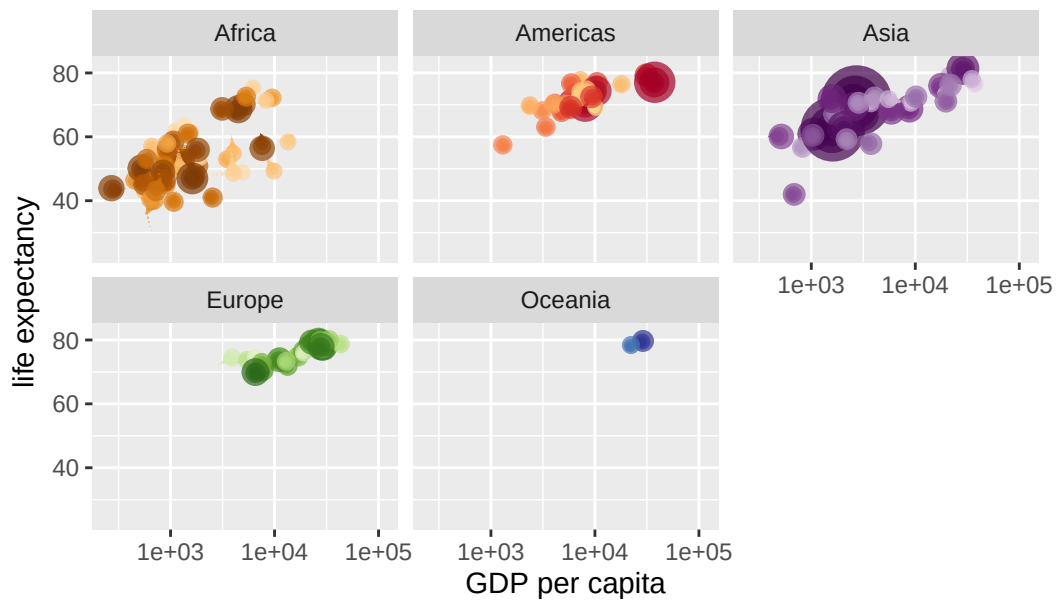
Year: 1999



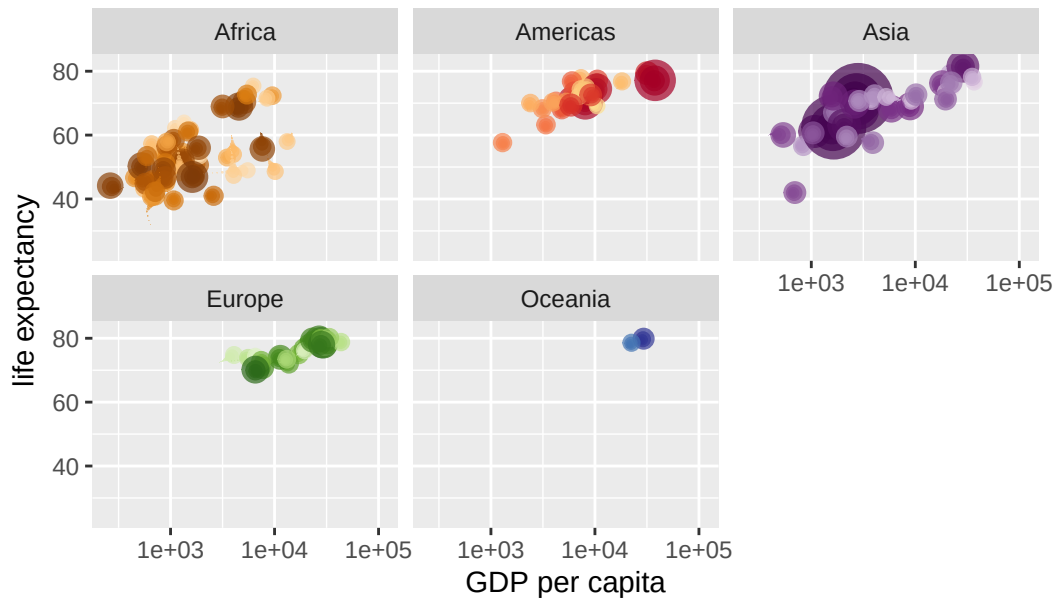
Year: 1999



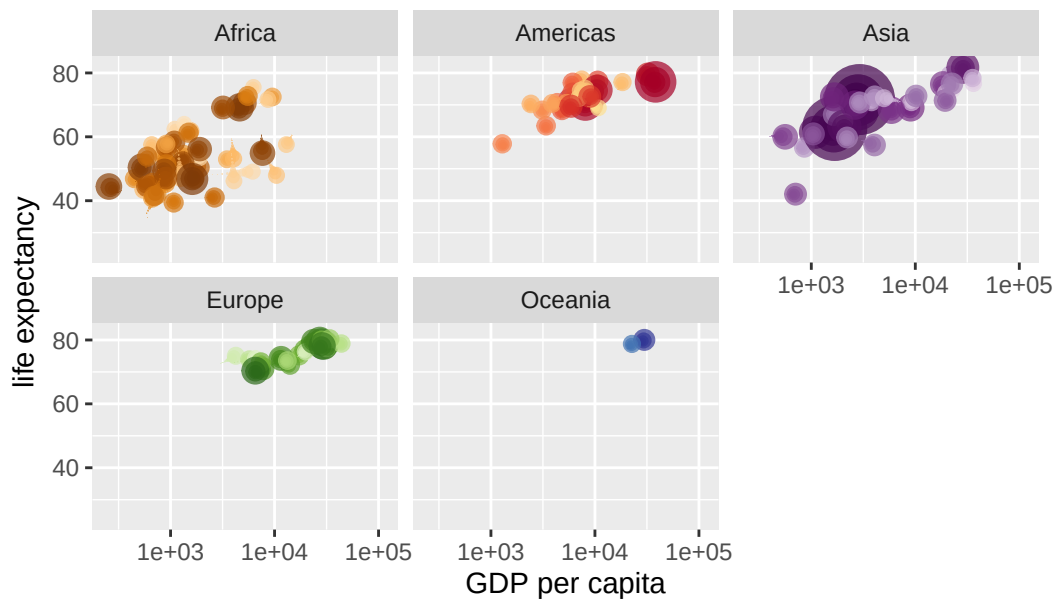
Year: 2000



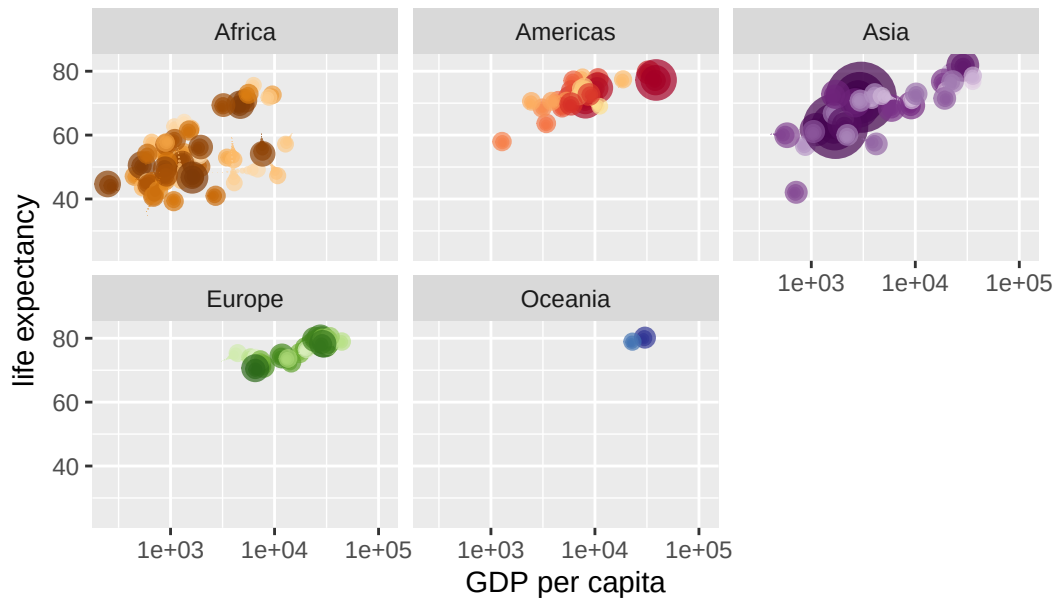
Year: 2000



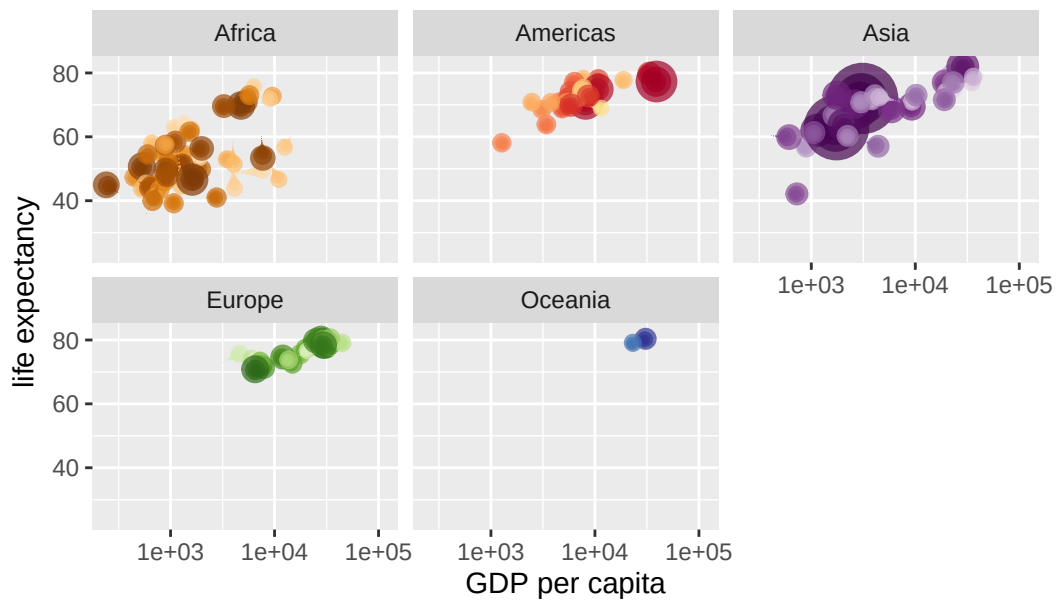
Year: 2001



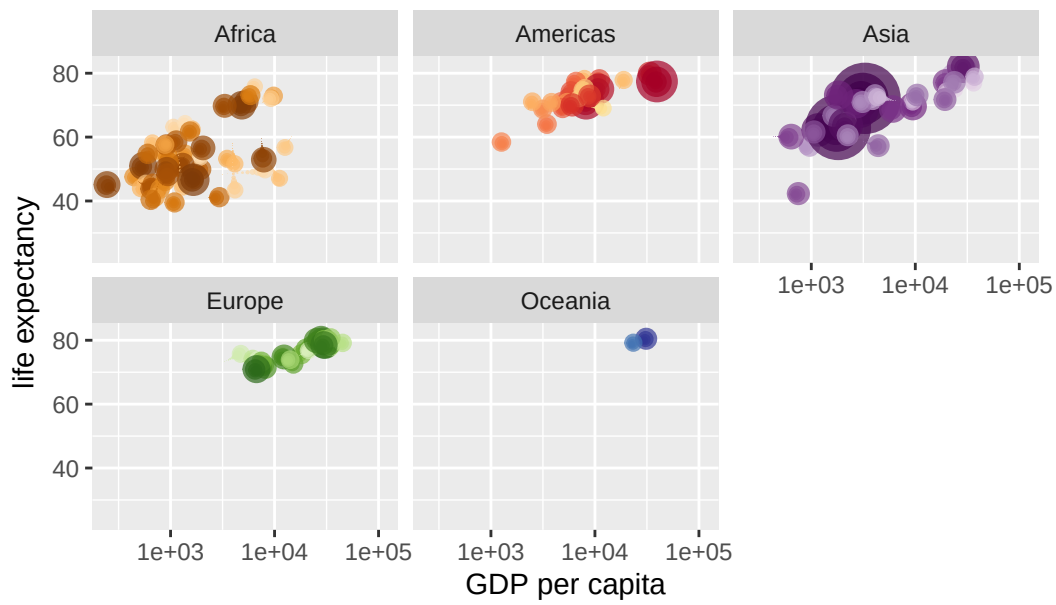
Year: 2001



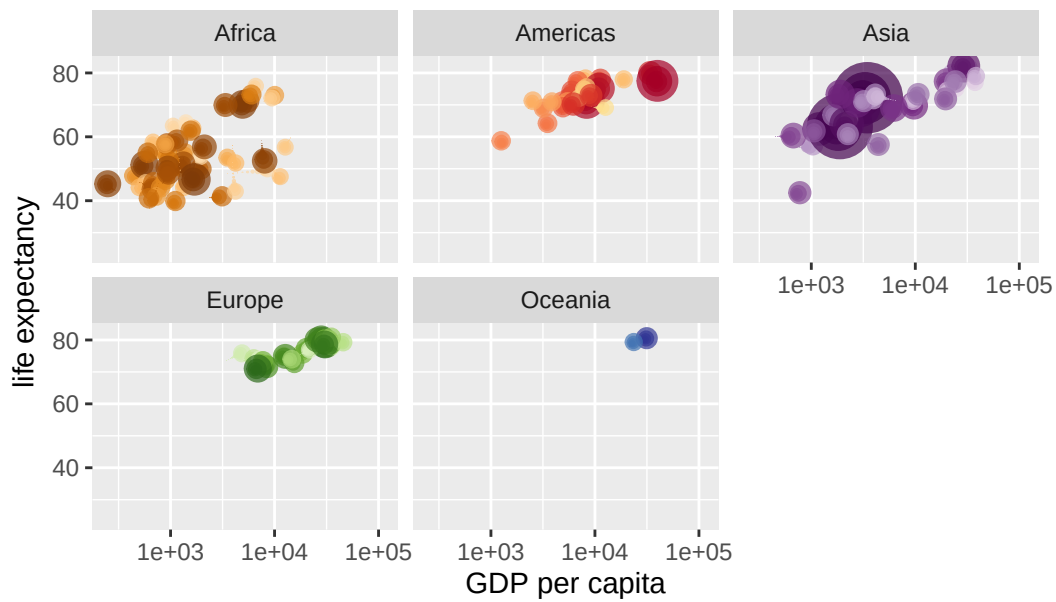
Year: 2002



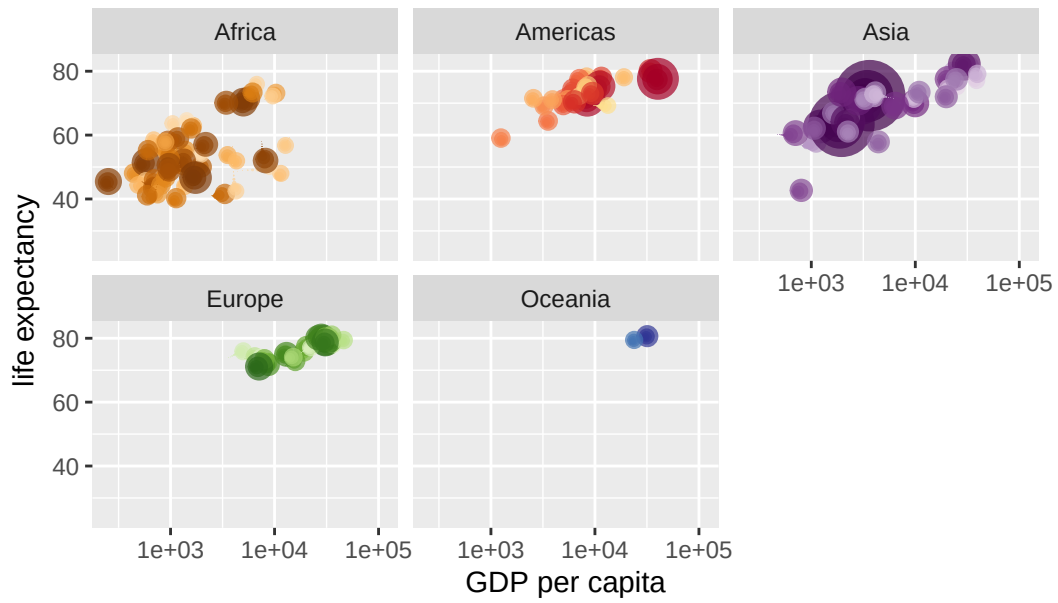
Year: 2003



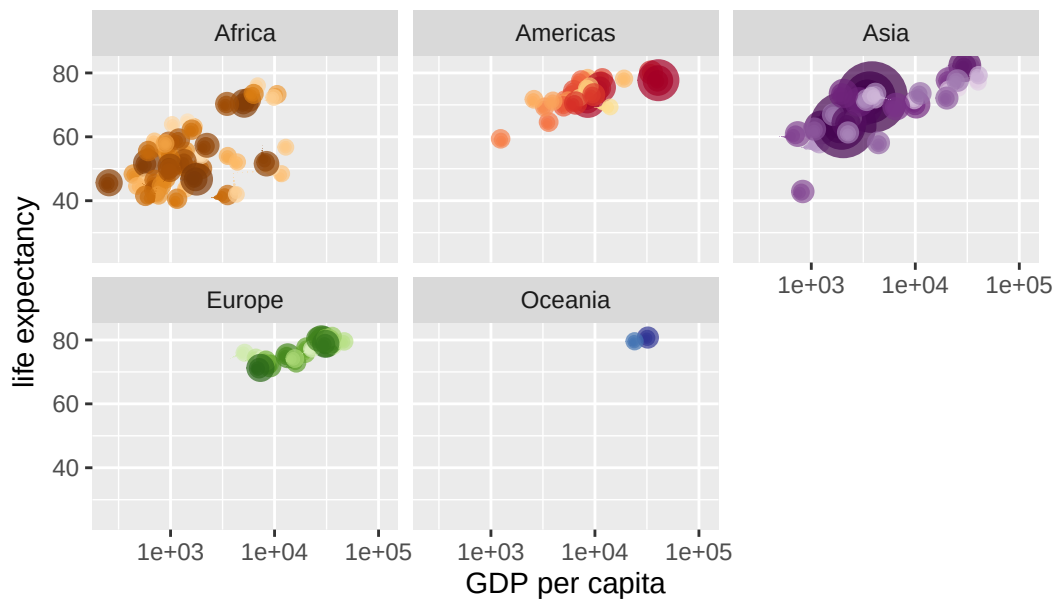
Year: 2003



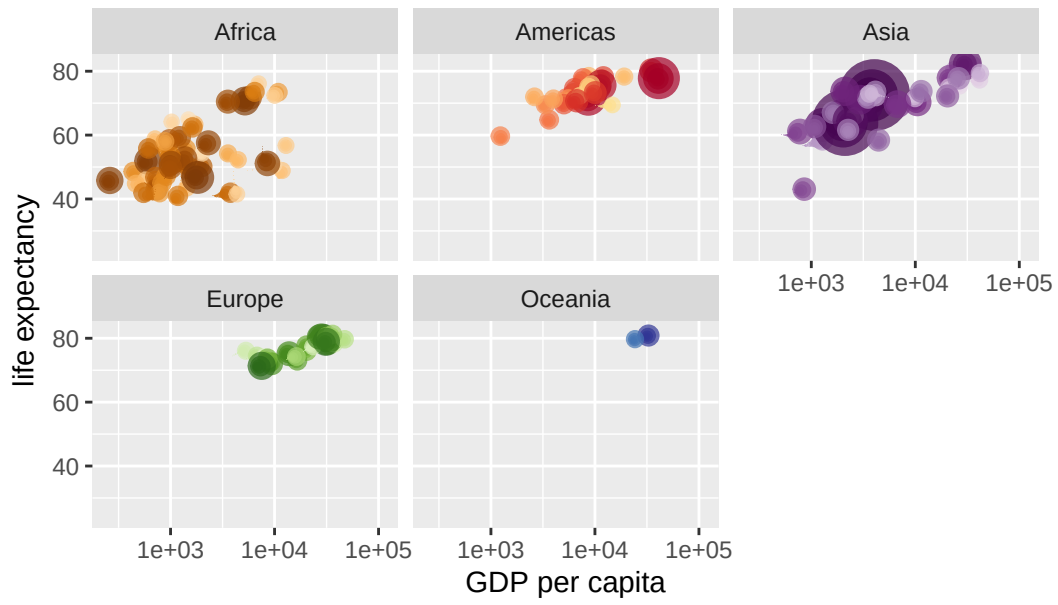
Year: 2004



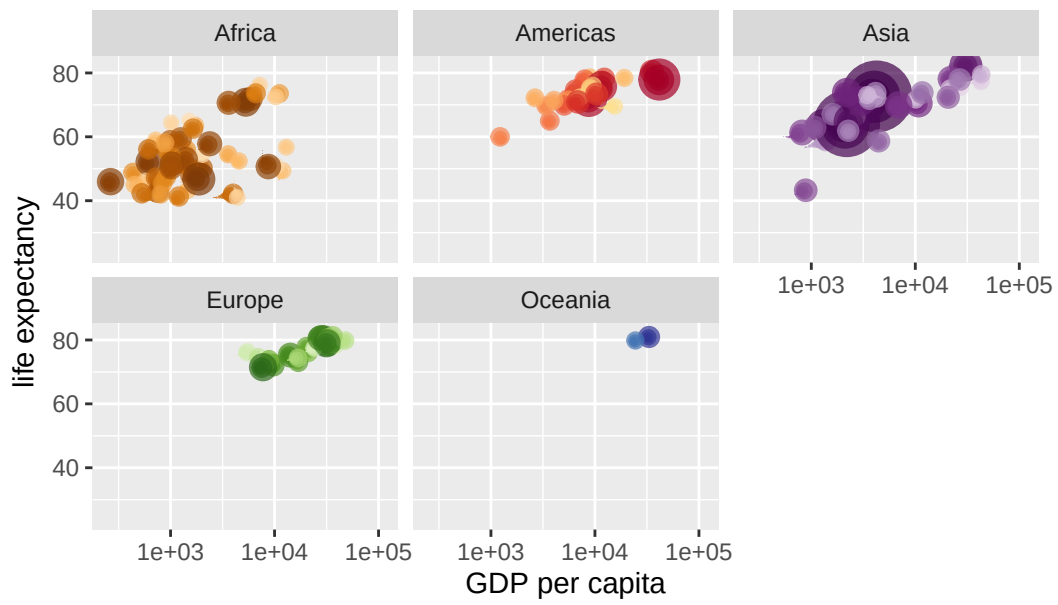
Year: 2004



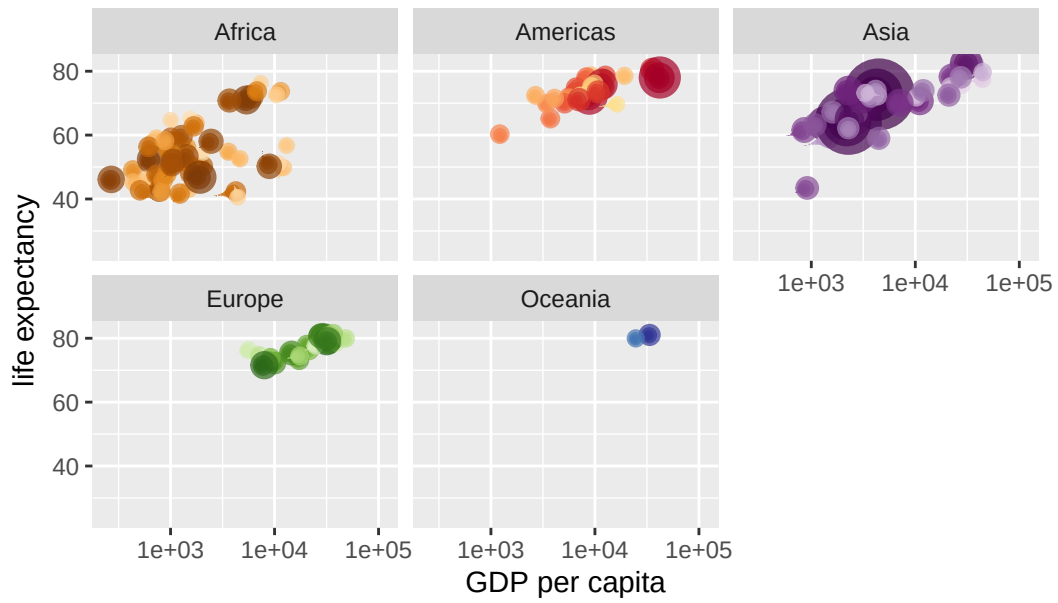
Year: 2005



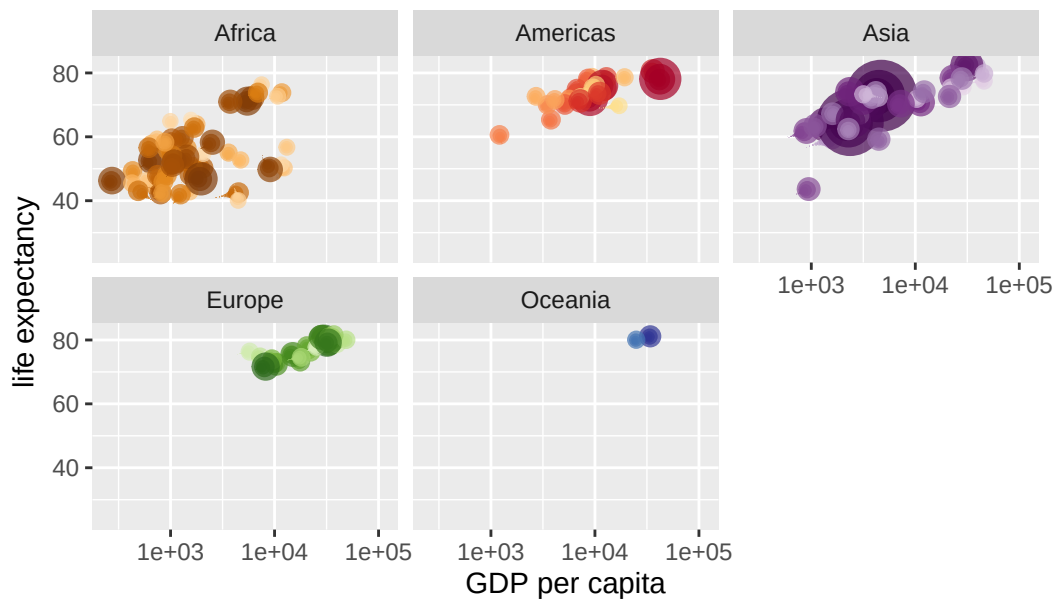
Year: 2005

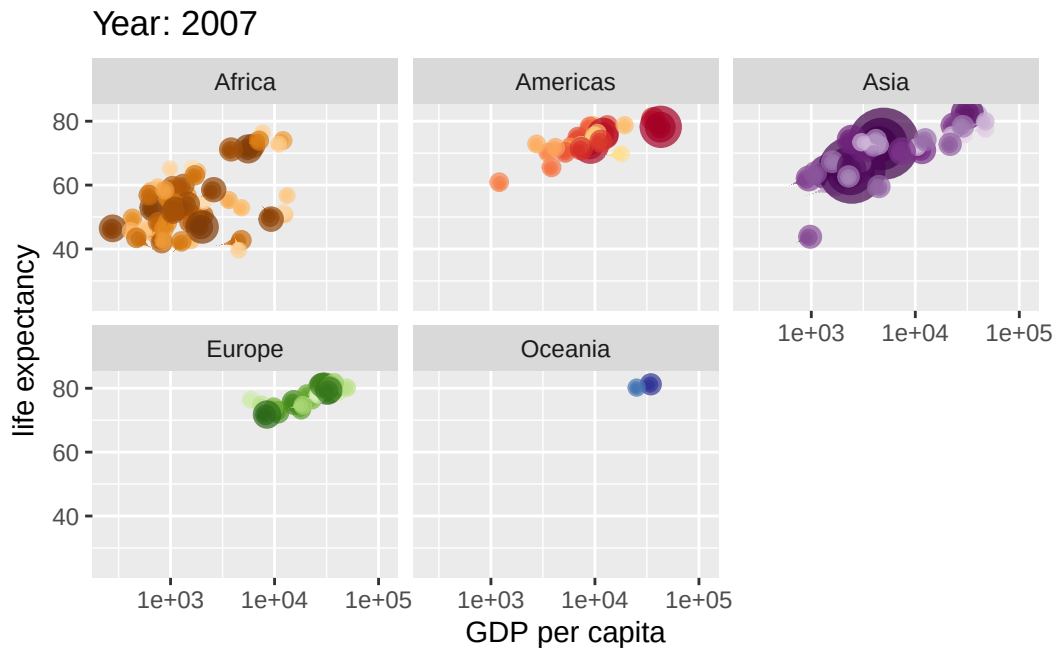


Year: 2006



Year: 2006





Part 10: Combining Plots

```
library(patchwork)

# Setup some example plots
p1 <- ggplot(mtcars) + geom_point(aes(mpg, disp))
p2 <- ggplot(mtcars) + geom_boxplot(aes(gear, disp, group = gear))
p3 <- ggplot(mtcars) + geom_smooth(aes(disp, qsec))
p4 <- ggplot(mtcars) + geom_bar(aes(carb))

# Use patchwork to combine them here:
(p1 | p2 | p3) /
  p4
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'

